CLEAN CORE MIGRATION COCKPIT

Custom Code Analysis & Decision Platform for SAP S/4HANA

20-DAY IMPLEMENTATION PLAN

Four team members · ABAP Cloud · CDS · RAP · Fiori Elements · ABAP Unit

Source limitation
No project PDF was attached or present in the workspace. This document is a detailed provisional plan derived from the written request, not a PDF extraction. Reconcile the requirements, policy rules, enums, workflow and architecture against the PDF before Day 2 contract freeze. The current checkout is AlgoViz 2.0 and is not the SAP cockpit project.

Prepared as a build plan, not a learning roadmap

All ABAP object names and SAP capability choices are proposed until verified in the assigned tenant on Day 1.

Contents

1. Provisional requirements extracted from the request . . . 3
2. Proposed final MVP scope . . . 3
3. Proposed technical conventions and Day-1 environment branch . . . 4
4. Shared data contract (proposed v1) . . . 5
5. Service, method, action, and validation contract . . . 12
6. Four-member ownership and handoff matrix . . . 15
7. Day 1–20 detailed execution plan . . . 15
DAY 1 — Verify real environment, access, and MVP gates . . . 16
DAY 2 — Freeze schema, enums, service contracts, synthetic inputs . . . 17
DAY 3 — Start the first thin backend/UI slice from the frozen contract . . . 18
DAY 4 — Activate the first inventory read service and connect the first list . . . 20
DAY 5 — Demonstrate the first thin end-to-end slice . . . 21
DAY 6 — Complete persistence, provenance, CSV template, and synthetic dataset . . . 22
DAY 7 — Implement initial parser, row validation, and import error reporting . . . 24
DAY 8 — Make imports idempotent and connect persisted evidence to classifier input . . . 25
DAY 9 — Implement deterministic rules, explanations, and conflict handling . . . 27
DAY 10 — Integrate and verify import → recommendation . . . 28
DAY 11 — Complete decision state model and legal-transition rules . . . 29
DAY 12 — Implement assignment, override, submit, and approval operations . . . 31
DAY 13 — Implement role authorization, reviewer separation, and audit events . . . 32
DAY 14 — Prove legal/illegal transitions, authorization, audit integrity, and stale-update behavior . . . 34
DAY 15 — Integrate workflow actions and audit history into the Fiori Object Page . . . 35
DAY 16 — Complete inventory filters, evidence display, recommendation detail, and UI errors . . . 36
DAY 17 — Build portfolio metrics and filters from supported read models . . . 38
DAY 18 — Integration/regression, fix high-priority defects, and retest . . . 39
DAY 19 — Finalize setup, architecture, test evidence, screenshots, and demo dataset . . . 40
DAY 20 — Run the demonstration, verify acceptance, and hand off limitations . . . 41
8. Exact artifact inventory (proposed names; ADT names finalized Day 1) . . . 43
9. Integration dependency matrix and daily handoffs . . . 45
10. Testing matrix (exact named inputs and expected results) . . . 46
11. Day-20 acceptance checklist . . . 47
12. Final demo sequence (10–12 minutes) . . . 48

Clean Core Migration Cockpit — 20-Day Implementation Plan

> **Source limitation / important:** No project PDF was attached or present in this checkout (`find` found no PDF under the repository or `/home/user`). The checkout itself is AlgoViz 2.0 (React/Vite), not a Clean Core SAP project. Therefore this is a **provisional implementation plan derived only from the requirements in the request**, not a claim that I inspected or extracted the PDF. Upload the PDF before treating the requirements, recommendations, enums, workflow, or architecture below as final; any PDF conflict wins. No SAP app code is being added to this unrelated React project.
>
> All technical names and project-specific policies are proposals until Day 1 verifies the tenant and Day 2 freezes the contract. The team must not claim an unsupported SAP capability works. Day 1 records the supported profile and gates the rest of the plan.

1. Provisional requirements extracted from the request

ID	Requirement stated in the request	Planned implementation	Source status
R1	Inventory custom objects by source system and expose them for analysis	`SourceSystem`, `CustomObject`, read-only CDS/RAP exposure, stable business key	Prompt only; confirm with PDF
R2	Ingest usage/evidence/findings/dependencies and retain provenance	`UsageFact`, `Evidence`, `Finding`, `Dependency`, `ImportBatch`, `ImportRow`; row-level validation and idempotent upsert	Prompt only; confirm required source formats/fields with PDF
R3	Produce a recommendation, rationale, rule trace, risk/priority, and missing-evidence result	Versioned `RuleSet`/`ClassificationRule`; deterministic classifier; immutable recommendation snapshots and match rows	Recommendation vocabulary/logic below is a proposed demo policy, not SAP guidance
R4	Record migration decisions, ownership, submission, approval, override reason, and history	`MigrationDecision` RAP root and append-only `DecisionAudit`; role and separation-of-duties checks	States/actions below are proposed pending PDF
R5	Browse inventory, evidence, recommendations, decisions, and portfolio metrics	Fiori Elements List Report/Object Page/portfolio page on the service actually supported by the tenant	Screen requirements from prompt; confirm PDF navigation and exact fields
R6	Use ABAP Cloud, ADT, CDS, RAP, Fiori Elements, and ABAP Unit where supported	Verify each capability in the assigned SAP environment before dependent work	No tenant/BTP access is available to this agent
R7	Four people keep the ownership boundaries in the request	M1 data/import; M2 rules/analytics; M3 workflow/security; M4 Fiori/integration/QA	Preserved; shared contracts are co-authored, not an ownership transfer
R8	Finish a demonstrable MVP in 20 working days	Day-by-day build/test order below, with explicit Day-1 stop/branch conditions	Schedule is a proposed student-team plan

What is not known without the PDF

The existing PDF architecture, actual required entities/fields, SAP product/version, required recommendation outcomes, business rule evidence, state names, approval roles, exact Fiori screens, existing acceptance criteria, and whether CSV upload is explicitly required cannot be verified. The contract below is consequently a disciplined **proposed MVP**, not a transcription of source requirements.

2. Proposed final MVP scope

1. Maintain source-system/custom-object inventory, period-based usage facts, evidence, findings, dependencies, import batches, and row errors.
2. Accept six documented CSV kinds (`SOURCE_SYSTEMS`, `CUSTOM_OBJECTS`, `USAGE_FACTS`, `EVIDENCE`, `FINDINGS`, `DEPENDENCIES`). Parse UTF-8 CSV, report row-level failures, preserve source/batch/row provenance, and make repeat imports idempotent. A valid-but-incomplete evidence record is retained with a warning; an invalid object reference is rejected.
3. Evaluate a fixed, versioned and editable demo rule set. Missing/unknown/stale required inputs, no match, or equal-priority conflicting outputs resolve safely to `MANUAL_REVIEW`. Recommendations are advisory snapshots; the cockpit never changes SAP custom code or executes a migration.
4. Create and manage one open decision per object, record an explicit owner, allow override only with a reason, submit, approve by a different principal, and preserve append-only audit events.

5. Deliver inventory List Report/Object Page, portfolio overview, recommendation explanation/matched rules, workflow actions, audit history, filters, and loading/empty/error states using the Fiori workflow Day 1 proves is supported.
6. Use synthetic data only. No live repository scanner, SAP standard replacement catalog, BTP service dependency, AI, automatic code remediation, or unsupported authentication is in scope.
7. If no supported persistence, RAP service exposure, user identity/authorization, or Fiori delivery path exists, report the feature as blocked and use only the smallest supported alternative listed in §3. Do not replace the backend with browser storage or imply that production authorization exists.

Proposed policy (must be reconciled with the PDF)

The following are explicitly **project-defined demo policy**, not SAP Clean Core rules: outcomes `KEEP`, `REMEDIATE`, `REPLACE`, `RETIRE`, `MANUAL_REVIEW`; required `CLOUD_READINESS` evidence and a 90-day usage window; evidence maximum age of 90 days; rule priorities; the risk score formula in §4. The demo dataset must label them as synthetic. If the PDF provides different outcomes, facts, thresholds, or source references, replace these values before Day 2 contract freeze.

3. Proposed technical conventions and Day-1 environment branch

Names and repository objects

- Proposed ABAP package: `ZCC_MIG_COCKPIT`; use the assigned customer package if that name, namespace, or software component is unavailable. Do not create objects in SAP namespace.
- Proposed prefix: `ZCC_` for custom persistence tables; `ZI_CC_` for interface/root CDS view entities; `ZC_CC_` for projection/consumption CDS; `ZCL_CC_` / `ZIF_CC_` for global classes/interfaces; `ZBP_I_...` for RAP behavior pools; `ZP_CC_...` for DCL access controls; `ZA_CC_...` for CDS abstract action-parameter entities; `ZUI_CC_...` for service definition/binding. Confirm repository name-length and namespace rules in ADT on Day 1. Names are proposed, not a claim of tenant availability.
- All ABAP artifacts are ADT repository objects, **not local filesystem files**. A BDEF is attached to its root entity; its behavior implementation is a behavior-pool class. ABAP Unit classes are local `FOR TESTING` classes inside the relevant class pool, not invented standalone files.
- UUID type: proposed `RAW(16)` (use `SYSUUID_X16` only if the release and DDL support it). UTC timestamp: proposed `UTCLONG` (substitute the release-supported UTC timestamp type). Use a released UUID generator and current-user API verified on Day 1. Optional business values persist as initial values when ABAP database semantics do not support SQL `NULL`; “nullable” below means semantically optional.
- Proposed contract package: `ZCC_MIG_COCKPIT`. If packages are split, preserve the ownership prefixes and grant only released package interfaces needed by consumers.
- Git holds the local docs/frontend, if any. Record the SAP transport/task or other tenant-supported delivery identifier for ADT objects. Do not claim an ADT object is in Git unless the tenant actually uses a supported Git integration.

Day-1 capability gate (no assumptions)

Record pass/fail and evidence for: assigned ABAP tenant/package; creation and activation of a table, CDS view entity, global class and ABAP Unit test; RAP managed root, action, validation, authorization handler and ETag/lock support; OData V4 UI binding (or supported V2); Fiori Elements preview/generation/deployment/launchpad; access to current principal and role assignment; CDS aggregation; supported data import path; and approved non-production synthetic-data seeding. The four Day-1 tasks create probes and record blockers.

Unavailable capability	Affected artifacts	Smallest supported alternative / stop condition
Custom persistence tables	All `ZCC_*` tables and RAP BOs	Use only an approved ABAP package/tenant with custom-table extensibility. There is no safe browser-only substitute for persisted decisions/audit. If none exists, stop backend implementation and report the MVP blocked.
CDS view entities / DDL sources	`ZI_CC_*`, `ZC_CC_*`, annotations	Use only the view/entity syntax supported by the ABAP Cloud language version. If a supported CDS variant exists, lock it on Day 2 and regenerate metadata; if no CDS/RAP-consumable entity exists, backend service/UI is blocked rather than switching to unreleased legacy constructs.
RAP managed behavior/actions	Decision/import/rules BDEFs and pools	If managed behavior is unavailable but supported unmanaged RAP is available, retain the same public OData/action contract and server validations in the unmanaged pool. If no RAP action/service path exists, workflow/import actions are blocked; do not mock them in Fiori.
ABAP Unit	All `ltcl_*` class-pool tests	Use the tenant-supported ABAP test runner if the standard ADT launch differs. If automated ABAP tests are not supported at all, record that acceptance blocker and do not claim the test matrix passed.
OData V4 UI binding	`ZUI_CC_COCKPIT_V4`, generated FE manifest/data source	If the tenant supports a Fiori Elements-compatible OData V2 binding, use proposed `ZUI_CC_COCKPIT_V2` and regenerate the app against its metadata. If neither is supported, use the tenant’s supported FE preview only if it supports all required actions; otherwise UI delivery is blocked.

Unavailable capability	Affected artifacts	Smallest supported alternative / stop condition
Local Fiori Tools/BAS or deployment	`ui/cc-migration-cockpit/**`	Use the tenant-supported Fiori Elements preview/runtime and CDS annotations; do not substitute React or claim a deployed app. Record any missing custom upload UI.
RAP authorization/role mapping	`ZP_CC_*`, behavior authorization methods, approval actions	Use the environment's supported global-role authorization if it exists; retain owner and separation-of-duties checks in RAP. If no authenticated principal/role source exists, disable mutation/approval outside an isolated dev tenant and mark security acceptance blocked—never hard-code users/passwords.
Instance authorization or ETag	Decision read/actions and stale-update test	Use supported global roles plus owner checks; serialize/lock actions if supported. If neither stale-update guard works, disable concurrent mutation and document the limitation; do not claim concurrency safety.
CSV file upload / large action parameter	Optional Fiori upload extension and `ImportCsv` action	Keep parser/validator in ABAP. Use supported RAP `ImportRow` staging CRUD plus `processBatch` action as the smallest repeatable API alternative. The app must label that it stages rows rather than uploading a file. If neither route is supported, use ABAP Unit for repeatable ingestion tests and mark interactive CSV import blocked.
CDS aggregate functions	`ZC_CC_PORTFOLIO`	Implement the same explicitly defined counts in a read-only ABAP query class exposed by supported RAP service; no external analytics service is assumed.

4. Shared data contract (proposed v1)

Freeze rule: Day 2 produces `docs/clean-core/contracts/mvp-contract-v1.md`. No member changes a shared name/type/enum after freeze without a same-day contract change, consumer notification, and regenerated tests. Foreign-key enforcement is done by supported DDIC/association constraints where available and always by importer/service validation. Composite business uniqueness is `(SourceSystemUUID,ObjectType,ObjectName)` for objects; other imported records use `(entity kind, SourceSystemUUID, SourceRecordKey)`. If unique indexes/locks are unavailable, Day 1 must select serialized single-writer import and record that limitation.

Types: `UUID16` = `RAW(16)` / verified `SYSUUID_X16`; `TS_UTC` = verified UTC timestamp; `CHAR(n)`, `INT4`, `INT8`, `DEC(15,3)`, `DATS`. Persist booleans as `CHAR(1)` with `X`/space; use ABAP `abap_bool` only for method parameters where supported. UUID primary keys are non-null. Optional fields may be initial. The table below is the complete proposed MVP field list; do not add per-member variants.

Entity / field	ABAP-compatible type	Key?	Nullable?	Purpose	Owner → consumers
SourceSystem `SourceSystemUUID`	UUID16	PK	No	Internal source key	M1 → M2,M3,M4
`SystemCode`	CHAR(20)	Unique business key	No	Stable external system code	M1 → all
`Description`	CHAR(80)	No	Yes	Display label	M1 → M4
`ProductVersion`	CHAR(20)	No	Yes	Imported source metadata only	M1 → M2,M4
`LandscapeCode`	CHAR(10)	No	No	`DEV/TEST/PROD/UNKNOWN` proposed labels	M1 → M4
`CreatedAt`, `CreatedBy`	TS_UTC, CHAR(40)	No	No	Provenance for source record	M1 → M4
CustomObject `ObjectUUID`	UUID16	PK	No	Internal object key	M1 → all
`SourceSystemUUID`	UUID16	FK	No	Parent system	M1 → all
`ObjectType`	CHAR(10)	Natural-key part	No	Imported repository object type	M1 → M2,M4
`ObjectName`	CHAR(40)	Natural-key part	No	Imported object name	M1 → M2,M4
`PackageName`	CHAR(30)	No	Yes	Source package label	M1 → M4
`SoftwareComponent`	CHAR(30)	No	Yes	Imported grouping/filter	M1 → M2,M4
`Description`	CHAR(80)	No	Yes	Display text	M1 → M4
`OwnerUser`	CHAR(40)	No	Yes	Imported object owner, not login credential	M1 → M3,M4
`SourceRecordKey`	CHAR(64)	Unique import key	No	Stable external row ID	M1 → M2
`ImportBatchUUID`	UUID16	FK	No	Latest accepted source batch	M1 → M4
`ImportedAt`	TS_UTC	No	No	Latest source update time	M1 → M4
UsageFact `UsageUUID`	UUID16	PK	No	Usage snapshot key	M1 → M2,M4
`ObjectUUID`	UUID16	FK	No	Referenced inventory object	M1 → M2,M4

Entity / field	ABAP-compatible type	Key?	Nullable?	Purpose	Owner → consumers
PeriodFrom, PeriodTo	DATS	No	No	Inclusive measurement window	M1 → M2,M4
ExecutionCount	INT8	No	No	Non-negative observed count	M1 → M2
LastUsedAt	TS_UTC	No	Yes	Last observed use, if source supplied it	M1 → M2,M4
CapturedAt	TS_UTC	No	No	Freshness/provenance timestamp	M1 → M2,M4
SourceName, SourceReference	CHAR(40), CHAR(120)	No	No, Yes	Producer and source record location	M1 → M2,M4
SourceRecordKey	CHAR(64)	Unique import key	No	Idempotent source-row key	M1 → M2
ImportBatchUUID	UUID16	FK	No	Import provenance	M1 → M4
Evidence EvidenceUUID	UUID16	PK	No	Evidence row key	M1 → M2,M4
ObjectUUID	UUID16	FK	No	Referenced inventory object	M1 → M2,M4
EvidenceCode	CHAR(30)	Business-key part	No	E.g. proposed CLOUD_READINESS, ALTERNATIVE_CANDIDATE	M1 → M2,M4
ValueText	CHAR(80)	No	Yes	Enumerated/text evidence value	M1 → M2,M4
ValueNumber	DEC(15,3)	No	Yes	Numeric evidence value	M1 → M2
ValueDate	DATS	No	Yes	Date evidence value	M1 → M2
QualityCode	CHAR(12)	No	No	COMPLETE/INCOMPLETE/UNKNOWN	M1 → M2,M4
CapturedAt	TS_UTC	No	No	Freshness/provenance	M1 → M2,M4
SourceName, SourceReference	CHAR(40), CHAR(120)	No	No, Yes	Provenance source	M1 → M4
SourceRecordKey	CHAR(64)	Unique import key	No	Idempotent source-row key	M1 → M2
ImportBatchUUID	UUID16	FK	No	Import provenance	M1 → M4
Finding FindingUUID	UUID16	PK	No	Finding row key	M1 → M2,M4
ObjectUUID	UUID16	FK	No	Affected object	M1 → M2,M4
FindingCode, SeverityCode	CHAR(30), CHAR(10)	No	No	Source finding ID and source-normalized label; not SAP severity guidance	M1 → M2,M4
Summary	CHAR(255)	No	No	Imported explanation	M1 → M2,M4
DetectedAt	TS_UTC	No	Yes	Source observation time	M1 → M2,M4
SourceName, SourceReference	CHAR(40), CHAR(120)	No	No, Yes	Provenance source	M1 → M4
SourceRecordKey	CHAR(64)	Unique import key	No	Idempotent source-row key	M1 → M2
ImportBatchUUID	UUID16	FK	No	Import provenance	M1 → M4
Dependency DependencyUUID	UUID16	PK	No	Edge key	M1 → M2,M4
SourceObjectUUID, TargetObjectUUID	UUID16 each	FK	No	Directed source → target; MVP requires both inventory records	M1 → M2,M4
DependencyType	CHAR(20)	No	No	Imported edge category	M1 → M2,M4
SourceName, SourceReference	CHAR(40), CHAR(120)	No	No, Yes	Provenance	M1 → M4
SourceRecordKey	CHAR(64)	Unique import key	No	Idempotent edge key	M1 → M2
ImportBatchUUID	UUID16	FK	No	Import provenance	M1 → M4
ImportBatch BatchUUID	UUID16	PK	No	Batch key	M1 → M4
ImportKind	CHAR(20)	No	No	Six CSV kinds listed above	M1 → M4
BatchStatus	CHAR(20)	No	No	NEW/PROCESSING/COMPLETED/COMPLETED_WITH_ERRORS/FAILED	M1 → M4
FileName	CHAR(128)	No	Yes	Submitted filename (no path/secret)	M1 → M4

Entity / field	ABAP-compatible type	Key?	Nullable?	Purpose	Owner → consumers
SubmittedBy, SubmittedAt, CompletedAt	CHAR(40), TS_UTC, TS_UTC	No	Submitted fields no; completion yes	Actor/time provenance	M1 → M3,M4
TotalRows, AcceptedRows, WarningRows, RejectedRows, DuplicateRows, UpdatedRows	INT4 each	No	No	Reconciled batch counts	M1 → M4
ErrorSummary	CHAR(255)	No	Yes	Batch-level failure only; row detail stays in ImportRow	M1 → M4
ImportRow ImportRowUUID	UUID16	PK	No	Row-result key	M1 → M4
BatchUUID	UUID16	FK	No	Parent batch	M1 → M4
RowNumber	INT4	Unique in batch	No	1-based physical/data row number	M1 → M4
EntityKind, SourceRecordKey	CHAR(20), CHAR(64)	No	Yes	Parsed kind and source key	M1 → M4
RawRow	CHAR(512)	No	Yes	Bounded row for error inspection; over-limit row rejected	M1 → M4
RowStatus	CHAR(20)	No	No	PERSISTED/UPDATED/ACCEPTED_WARNING/SKIPPED_DUPLICATE/REJECTED	M1 → M4
ErrorCode, ErrorText	CHAR(30), CHAR(255)	No	Yes	Stable machine code and human message	M1 → M4
ResolvedUUID	UUID16	No	Yes	Persisted entity key on success	M1 → M4
RuleSet RuleSetUUID	UUID16	PK	No	Versioned rule-set key	M2 → M1,M3,M4
RuleSetCode, Version	CHAR(20), CHAR(10)	Unique pair	No	Stable code and immutable version	M2 → M3,M4
Status	CHAR(10)	No	No	DRAFT/ACTIVE/RETIRED	M2 → M3,M4
Description	CHAR(120)	No	No	Policy description with demo disclaimer	M2 → M4
EffectiveFrom, EffectiveTo, PublishedAt	TS_UTC each	No	End/published may be initial	Version lifecycle	M2 → M3,M4
PublishedBy	CHAR(40)	No	Yes until publish	Authenticated publisher snapshot from supported platform identity	M2 → M3
CreatedAt, CreatedBy	TS_UTC, CHAR(40)	No	No	Configuration provenance	M2 → M3
Classification Rule RuleUUID	UUID16	PK	No	Rule key	M2 → M3,M4
RuleSetUUID	UUID16	FK	No	Parent version	M2 → M3,M4
RuleID	CHAR(30)	Unique within version	No	Stable trace ID	M2 → M3,M4
Priority	INT4	No	No	Positive integer; lower number wins	M2 → M3
ConditionCode, InputCode, Operator, CompareValue	CHAR(30), CHAR(30), CHAR(12), CHAR(80)	No	Condition/input/operator required; compare may be initial for IS_MISSING	Allowlisted, non-dynamic rule expression	M2 → M3
OutcomeCode	CHAR(20)	No	No	One agreed recommendation outcome	M2 → M3,M4
ExplanationText	CHAR(255)	No	No	Plain-language rule rationale	M2 → M4
IsActive	CHAR(1)	No	No	Rule participation flag (X/space)	M2 → M3
Recommendation RecommendationUUID	UUID16	PK	No	Evaluation snapshot key	M2 → M1,M3,M4
ObjectUUID	UUID16	FK	No	Evaluated object	M2 → M1,M3,M4
RuleSetUUID, RuleSetVersion	UUID16, CHAR(10)	FK / snapshot	No	Exact policy version used	M2 → M3,M4
EvaluatedAt	TS_UTC	No	No	Evaluation time	M2 → M3,M4
OutcomeCode	CHAR(20)	No	No	Advisory proposed outcome	M2 → M3,M4
EvidenceStatus	CHAR(15)	No	No	COMPLETE/INCOMPLETE/STALE/UNKNOWN	M2 → M4
RiskScore	INT4	No	No	Demo triage score, 0–100	M2 → M4
PriorityBand	CHAR(6)	No	No	LOW/MEDIUM/HIGH, from formula below	M2 → M4
Rationale, MissingEvidenceCodes	CHAR(255) each	No	Rationale no; missing list may be initial	Explanation and concise missing-input list	M2 → M4

Entity / field	ABAP-compatible type	Key?	Nullable?	Purpose	Owner → consumers
RecommendationMatch MatchUUID	UUID16	PK	No	Trace row key	M2 → M4
RecommendationUUID	UUID16	FK	No	Snapshot parent	M2 → M4
RuleUUID	UUID16	FK	Yes	Null only for system fallback	M2 → M4
RuleID	CHAR(30)	No	No	Matched rule or SYSTEM_FALLBACK	M2 → M4
Priority	INT4	No	No	Applied priority; 0 for preflight fallback	M2 → M4
OutcomeCode	CHAR(20)	No	No	Rule/fallback result	M2 → M4
ExplanationText	CHAR(255)	No	No	Human-readable matched-rule trace	M2 → M4
IsSelected	CHAR(1)	No	No	Marks final winning rule(s) (X/space)	M2 → M4
MigrationDecision DecisionUUID	UUID16	PK	No	Decision key	M3 → M4
ObjectUUID, RecommendationUUID	UUID16 each	FK	No	Object and recommendation snapshot being decided	M3 → M2,M4
DecisionStatus	CHAR(16)	No	No	Proposed DRAFT/SUBMITTED/APPROVED	M3 → M4
DecisionOutcome	CHAR(20)	No	No	Current chosen outcome; may equal recommendation	M3 → M4
IsOverride	CHAR(1)	No	No	True only after valid override (X/space)	M3 → M4
OverrideReason	CHAR(255)	No	Yes	Mandatory when IsOverride = X	M3 → M4
OwnerUser	CHAR(40)	No	Yes in draft	Decision owner from authenticated user directory	M3 → M4
SubmittedBy, ApprovedBy	CHAR(40) each	No	Yes until set	Principal snapshots	M3 → M4
SubmittedAt, ApprovedAt	TS_UTC each	No	Yes until set	Workflow timestamps	M3 → M4
CreatedAt, CreatedBy, LastChangedAt, LastChangedBy	TS_UTC, CHAR(40), TS_UTC, CHAR(40)	No	No	ETag/audit support	M3 → M4
DecisionAudit AuditUUID	UUID16	PK	No	Audit event key	M3 → M4
DecisionUUID	UUID16	FK	No	Decision parent	M3 → M4
EventCode	CHAR(20)	No	No	CREATED/OWNER_ASSIGNED/OVERRIDDEN/SUBMITTED/APPROVED	M3 → M4
OldStatus, NewStatus	CHAR(16) each	No	Old may be initial	Before/after status	M3 → M4
OldOutcome, NewOutcome	CHAR(20) each	No	Old may be initial	Before/after outcome	M3 → M4
ActorUser	CHAR(40)	No	No	Authenticated platform principal	M3 → M4
OccurredAt	TS_UTC	No	No	Server event time	M3 → M4
Reason	CHAR(255)	No	Yes except override	User-supplied reason/comment	M3 → M4
CorrelationID	UUID16	No	Yes	Group related events where supported	M3 → M4
Portfolio read model SystemCode	CHAR(20)	Composite key	No	Source system dimension	M2 → M4
SoftwareComponent	CHAR(30)	Composite key	Yes (blank group allowed)	Component dimension	M2 → M4
ObjectType	CHAR(10)	Composite key	No	Object type dimension	M2 → M4
TotalObjectCount, KeepCount, RemediateCount, ReplaceCount, RetireCount, ManualReviewCount	INT4 each	No	No	Current object/outcome counts; no-latest-recommendation counts as MANUAL_REVIEW	M2 → M4
IncompleteEvidenceCount, OpenDecisionCount	INT4 each	No	No	Current incomplete/manual-review population; DRAFT+SUBMITTED objects	M2 → M4

Entity / field	ABAP-compatible type	Key?	Nullable?	Purpose	Owner → consumers
HighPriorityCount, MediumPriorityCount, LowPriorityCount	INT4 each	No	No	Current per-object priority-band counts; no-evaluation fallback has score 50/MEDIUM	M2 → M4

Relationships and shared enums

- SourceSystem 1–N CustomObject; CustomObject 1–N UsageFact/Evidence/Finding; CustomObject 1–N Dependency as source and target; ImportBatch 1–N ImportRow and accepted records; RuleSet 1–N ClassificationRule; CustomObject 1–N Recommendation; Recommendation 1–N RecommendationMatch; CustomObject 1–N MigrationDecision; MigrationDecision 1–N DecisionAudit.
- Imported references use system_code + object_type + object_name; importer resolves them to UUIDs. Dependency targets must exist in the MVP; unresolved/external target rows are rejected as UNKNOWN_TARGET_OBJECT, not silently created.
- Recommendation enum (proposed): KEEP, REMEDIATE, REPLACE, RETIRE, MANUAL_REVIEW. MANUAL_REVIEW is a safe fallback and must never be changed to a more specific outcome because an input is missing.
- Decision states (proposed): DRAFT → SUBMITTED → APPROVED. No reject/cancel/reopen action is added until the PDF requires it. Only DRAFT can be assigned/overridden; only SUBMITTED can be approved.
- Import row result values are exactly those in the field table. Structural errors are REJECTED; exact repeat rows are SKIPPED_DUPLICATE; changed payload under the same stable source key is UPDATED; valid evidence with blank/unknown value is retained as ACCEPTED_WARNING and classified as incomplete. Batch counters are mutually exclusive: AcceptedRows=PERSISTED, UpdatedRows=UPDATED, WarningRows=ACCEPTED_WARNING, DuplicateRows=SKIPPED_DUPLICATE, RejectedRows=REJECTED, and TotalRows is their sum. A source-system row upserts by SystemCode; an object with the same system/type/name but a different SourceRecordKey is DUPLICATE_CONFLICT, not a second object.

Exact CSV wire format (proposed v1)

All six local templates live at exact paths under docs/clean-core/import-template/. ImportKind matches each filename stem in uppercase. Encoding is UTF-8, delimiter comma, quote ", doubled quote escapes a quote, headers/order are exact, dates use YYYY-MM-DD, UTC timestamps use YYYY-MM-DDThh:mm:ssZ, integers use ASCII digits/no thousands separators, decimals use a period, embedded line breaks are not allowed, and each raw row is limited to 512 characters. The server derives UUIDs, ImportBatchUUID, RowStatus, QualityCode, error fields and timestamps; CSV cannot set them. source_systems.csv is idempotent by system_code; all other rows require source_record_key.

Exact file / ImportKind	Exact header line	Row reference and special validation
source_systems.csv / SOURCE_SYSTEMS	system_code,description,product_version,landscape_code	system_code unique; proposed landscape values DEV, TEST, PROD, UNKNOWN
custom_objects.csv / CUSTOM_OBJECTS	system_code,object_type,object_name,package_name,software_component,description,owner_user,source_record_key	SourceSystem must exist; object key is system+type+name
usage_facts.csv / USAGE_FACTS	system_code,object_type,object_name,period_from,period_to,execution_count,last_used_at,captured_at,source_name,source_reference,source_record_key	Object must exist; count ≥0; period_from ≤ period_to; last_used_at may be blank
evidence.csv / EVIDENCE	system_code,object_type,object_name,evidence_code,value_text,value_number,value_date,captured_at,source_name,source_reference,source_record_key	Exactly one typed value for complete evidence; all three blank or an allowed unknown is ACCEPTED_WARNING; two or three simultaneously populated value columns are rejected INVALID_EVIDENCE_VALUE; QualityCode is server-derived
findings.csv / FINDINGS	system_code,object_type,object_name,finding_code,severity_code,summary,detected_at,source_name,source_reference,source_record_key	Object must exist; preserve supplied severity code; normalize only under approved v1 mapping
dependencies.csv / DEPENDENCIES	system_code,source_object_type,source_object_name,target_object_type,target_object_name,dependency_type,source_name,source_reference,source_record_key	Both objects must exist in same source system for v1; otherwise reject UNKNOWN_OBJECT/UNKNOWN_TARGET_OBJECT

Clean-fixture import order: source systems → custom objects → usage/evidence/findings/dependencies. A dependency on an object in another system is outside proposed v1; add an explicit target-system column only through a contract change if the PDF requires cross-system edges.

Proposed evidence/rule policy and deterministic formulas

- Demo required inputs: latest CLOUD_READINESS (PASS/FAIL/UNKNOWN) captured no more than 90 days before AsOfUtc and not in the future; plus a usage fact whose inclusive measurement period is at least 90 calendar days (PeriodTo - PeriodFrom + 1 ≥ 90) and whose CapturedAt is no more than 90 days before AsOfUtc and not in the future. The measured period may be historical; the capture timestamp must be fresh. Never coerce an unknown count or blank LastUsedAt to zero. For each EvidenceCode select greatest CapturedAt, then lexicographically smallest SourceRecordKey, then UUID. For UsageFact, filter to eligible periods with fresh capture, then select greatest PeriodTo, greatest CapturedAt, lexicographically smallest SourceRecordKey, then UUID. A newest unknown evidence row never falls back to an older PASS. ALTERNATIVE_CANDIDATE=YES is an explicit synthetic/imported indicator, not a claim about SAP standard functionality. All 90-day thresholds are project policy only.
- Demo rules: D00_MISSING (priority 1, required evidence absent/unknown/stale → MANUAL_REVIEW); D10_READINESS_FAIL (10, CLOUD_READINESS=FAIL → REMEDIATE); D20_ALT_CONFIRMED (20,

ALTERNATIVE_CANDIDATE=YES → REPLACE); D30_ZERO_USAGE (30, 90-day count 0 → RETIRE); D40_READINESS_PASS (40, CLOUD_READINESS=PASS → KEEP). Lower number wins. Same winning priority + different outcomes returns MANUAL_REVIEW, traces all tied rules, and does not select one arbitrarily. Same-priority same-outcome rules are all traced. No match → MANUAL_REVIEW with SYSTEM_FALLBACK. Unknown required values are missing; unknown optional values do not assert a rule.

- Freshness is checked against an explicit AsOfUtc input (inject a fixed clock in tests): AsOfUtc - CapturedAt > 90 days is stale; exactly 90 days is fresh. A future capture time is UNKNOWN and does not match a positive rule. EvidenceStatus precedence when several inputs are bad is UNKNOWN (unknown/future required value) → STALE (otherwise any stale required value) → INCOMPLETE (otherwise a required row/value is absent) → COMPLETE; MissingEvidenceCodes still lists every issue.
- **Demo triage score only:** RiskScore = min(100, (EvidenceStatus is INCOMPLETE/STALE/UNKNOWN ? 50 : 0) + 20 * min(3, count(Finding.SeverityCode in {HIGH,ERROR})) + 5 * min(4, dependency_count)). PriorityBand = HIGH for score ≥70, MEDIUM for 40–69, LOW below 40. This is not SAP risk scoring. Explain that label in UI and docs; do not use the score as an SAP fact.

Complete proposed v1 rule catalog (all rules are project-defined demo policy)

RuleID / priority	Preconditions and evidence inputs	Decision logic	Output	Exact explanation text	Missing-data behavior	Conflict behavior	Positive/negative test
D00_MISSING / 1	Any required CLOUD_READINESS or valid 90-day UsageFact is absent, unknown, future-dated or stale; input code EVIDENCE_COMPLETE	EVIDENCE_COMPLETE EQ FALSE (safety preflight)	MANUAL_REVIEW	Required demo evidence is missing, unknown, future-dated, or older than the configured policy window; recommendation withheld.	Lists each missing/stale/unknown code; takes precedence over every positive rule	Highest-priority safety result; never tie-break into a positive recommendation	Remove readiness → MANUAL_REVIEW; both fresh required inputs → D00 does not match
D10_READINESS_FAIL / 10	Required evidence complete; CLOUD_READINESS value is known and fresh	CLOUD_READINESS EQ FAIL	REMEDIATE	Synthetic demo policy: imported CLOUD_READINESS is FAIL; review the source finding before choosing an action.	If required evidence is not complete, D00 returns MANUAL_REVIEW first	If tied at priority 10 with a different outcome, MANUAL_REVIEW; lower-number D00 safety gate still wins	FAIL → REMEDIATE; PASS → D10 does not match
D20_ALT_CONFIRMED / 20	Required evidence complete; explicit ALTERNATIVE_CANDIDATE evidence is YES	ALTERNATIVE_CANDIDATE EQ YES	REPLACE	Synthetic demo indicator marks an alternative candidate; confirm the actual option with an authorized human.	Optional indicator absent/unknown means no match, not an inferred YES; missing required evidence still triggers D00	Same-priority different output → MANUAL_REVIEW; higher-priority matching rule wins	YES with readiness PASS/usage >0 → REPLACE; NO → no D20 match
D30_ZERO_USAGE / 30	Required evidence complete; a usage fact covers the configured 90-day period	USAGE_COUNT_90D EQ 0	RETIRE	Synthetic demo policy: no executions were recorded in the configured usage window; confirm retention requirements before deciding.	Missing/stale usage triggers D00, not zero; unknown count is never coerced to 0	Same-priority different output → MANUAL_REVIEW; higher-priority matching rule wins	Count 0 with readiness PASS/candidate NO → RETIRE; count 25 → no D30 match
D40_READINESS_PASS / 40	Required evidence complete; CLOUD_READINESS value is known and fresh	CLOUD_READINESS EQ PASS	KEEP	Synthetic demo policy: imported CLOUD_READINESS is PASS; this is an advisory candidate, not an approval.	Missing/unknown/stale required data triggers D00 first	Same-priority different output → MANUAL_REVIEW; any lower numeric priority that matches wins	PASS + usage 25 + candidate NO → KEEP; FAIL → no D40 match

Exact configuration rows stored in ClassificationRule are:

RuleID	ConditionCode	InputCode	Operator	CompareValue	Priority	OutcomeCode
D00_MISSING	EVIDENCE_GATE	EVIDENCE_COMPLETE	EQ	FALSE	1	MANUAL_REVIEW
D10_READINESS_FAIL	COMPARE	CLOUD_READINESS	EQ	FAIL	10	REMEDIATE
D20_ALT_CONFIRMED	COMPARE	ALTERNATIVE_CANDIDATE	EQ	YES	20	REPLACE
D30_ZERO_USAGE	COMPARE	USAGE_COUNT_90D	EQ	0	30	RETIRE
D40_READINESS_PASS	COMPARE	CLOUD_READINESS	EQ	PASS	40	KEEP

Fixed synthetic fixture and exact classifier oracle

Use the same clock for deterministic integration tests: AsOfUtc=2026-10-13T00:00:00Z. All fixture evidence/usage rows are captured at 2026-10-12T12:00:00Z; the proposed usage period is inclusive 2026-07-15 through 2026-10-12 (90 days in this fixture). These are generated demo dates, not production observations. Create five CLAS objects in system DEMO01 and use the exact CSV schemas above.

ObjectName	CLOUD_READINESS	ALTERNATIVE_CANDIDATE	UsageFact (90 days)	Other fixture rows	Expected v1 outcome
ZCL_DEMO_KEEP	PASS, complete/fresh	NO, complete/fresh	ExecutionCount=25, complete/fresh	none	KEEP, D40_READINESS_PASS
ZCL_DEMO_REMEDIATE	FAIL, complete/fresh	NO, complete/fresh	ExecutionCount=25, complete/fresh	one synthetic HIGH finding and one dependency to ZCL_DEMO_KEEP	REMEDIATE, D10_READINESS_FAIL
ZCL_DEMO_REPLACE	PASS, complete/fresh	YES, complete/fresh	ExecutionCount=25, complete/fresh	none	REPLACE, D20_ALT_CONFIRMED (priority 20 beats D40)
ZCL_DEMO_RETIRE	PASS, complete/fresh	NO, complete/fresh	ExecutionCount=0, complete/fresh; LastUsedAt blank	none	RETIRE, D30_ZERO_USAGE (priority 30 beats D40)
ZCL_DEMO_UNKNOWN	UNKNOWN/accepted warning	absent	ExecutionCount=25, complete/fresh	none	MANUAL_REVIEW, EvidenceStatus=UNKNOWN and unknown readiness listed

source_systems.csv contains DEMO01,Synthetic demo source,DEMO-1,TEST. All object rows use package ZDEMO, component DEMO, owner label SYNTHETIC_OWNER (not a login), and stable keys OBJ-KEEP-001, OBJ-REMEDIATE-001, OBJ-REPLACE-001, OBJ-RETIRE-001, OBJ-UNKNOWN-001. The synthetic finding/dependency source keys are FND-REMEDIATE-001 and DEP-REMEDIATE-KEEP-001. All SourceName values are SYNTHETIC_DEMO; SourceReference is fixture-v1. The five outcomes are test-policy outputs only. Use these exact fixture file contents (the header lines must match §4):

docs/clean-core/demo/source_systems.csv

```
system_code,description,product_version,landscape_code
DEMO01,Synthetic demo source,DEMO-1,TEST
```

docs/clean-core/demo/custom_objects.csv

```
system_code,object_type,object_name,package_name,software_component,description,owner_user,source_record_key
DEMO01,CLAS,ZCL_DEMO_KEEP,ZDEMO,DEMO,KEEP synthetic fixture,SYNTHETIC_OWNER,OBJ-KEEP-001
DEMO01,CLAS,ZCL_DEMO_REMEDIATE,ZDEMO,DEMO,REMEDIATE synthetic fixture,SYNTHETIC_OWNER,OBJ-REMEDIATE-001
DEMO01,CLAS,ZCL_DEMO_REPLACE,ZDEMO,DEMO,REPLACE synthetic fixture,SYNTHETIC_OWNER,OBJ-REPLACE-001
DEMO01,CLAS,ZCL_DEMO_RETIRE,ZDEMO,DEMO,RETIRE synthetic fixture,SYNTHETIC_OWNER,OBJ-RETIRE-001
DEMO01,CLAS,ZCL_DEMO_UNKNOWN,ZDEMO,DEMO,Unknown evidence fixture,SYNTHETIC_OWNER,OBJ-UNKNOWN-001
```

docs/clean-core/demo/usage_facts.csv

```
system_code,object_type,object_name,period_from,period_to,execution_count,last_used_at,captured_at,source_name,source_reference,source_record_key
DEMO01,CLAS,ZCL_DEMO_KEEP,2026-07-15,2026-10-12,25,2026-10-11T10:00:00Z,2026-10-12T12:00:00Z,SYNTHETIC_DEMO,fixture-v1,USE-KEEP-001
DEMO01,CLAS,ZCL_DEMO_REMEDIATE,2026-07-15,2026-10-12,25,2026-10-11T10:00:00Z,2026-10-12T12:00:00Z,SYNTHETIC_DEMO,fixture-v1,USE-REMEDIATE-001
DEMO01,CLAS,ZCL_DEMO_REPLACE,2026-07-15,2026-10-12,25,2026-10-11T10:00:00Z,2026-10-12T12:00:00Z,SYNTHETIC_DEMO,fixture-v1,USE-REPLACE-001
DEMO01,CLAS,ZCL_DEMO_RETIRE,2026-07-15,2026-10-12,0,,2026-10-12T12:00:00Z,SYNTHETIC_DEMO,fixture-v1,USE-RETIRE-001
DEMO01,CLAS,ZCL_DEMO_UNKNOWN,2026-07-15,2026-10-12,25,2026-10-11T10:00:00Z,2026-10-12T12:00:00Z,SYNTHETIC_DEMO,fixture-v1,USE-UNKNOWN-001
```

docs/clean-core/demo/evidence.csv

```
system_code,object_type,object_name,evidence_code,value_text,value_number,value_date,captured_at,source_name,source_reference,source_record_key
DEMO01,CLAS,ZCL_DEMO_KEEP,CLOUD_READINESS,PASS,,,2026-10-12T12:00:00Z,SYNTHETIC_DEMO,fixture-v1,EVD-KEEP-READINESS
DEMO01,CLAS,ZCL_DEMO_KEEP,ALTERNATIVE_CANDIDATE,NO,,,2026-10-12T12:00:00Z,SYNTHETIC_DEMO,fixture-v1,EVD-KEEP-ALTERNATIVE
DEMO01,CLAS,ZCL_DEMO_REMEDIATE,CLOUD_READINESS,FAIL,,,2026-10-12T12:00:00Z,SYNTHETIC_DEMO,fixture-v1,EVD-REMEDIATE-READINESS
DEMO01,CLAS,ZCL_DEMO_REMEDIATE,ALTERNATIVE_CANDIDATE,NO,,,2026-10-12T12:00:00Z,SYNTHETIC_DEMO,fixture-v1,EVD-REMEDIATE-ALTERNATIVE
DEMO01,CLAS,ZCL_DEMO_REPLACE,CLOUD_READINESS,PASS,,,2026-10-12T12:00:00Z,SYNTHETIC_DEMO,fixture-v1,EVD-REPLACE-READINESS
DEMO01,CLAS,ZCL_DEMO_REPLACE,ALTERNATIVE_CANDIDATE,YES,,,2026-10-12T12:00:00Z,SYNTHETIC_DEMO,fixture-v1,EVD-REPLACE-ALTERNATIVE
DEMO01,CLAS,ZCL_DEMO_RETIRE,CLOUD_READINESS,PASS,,,2026-10-12T12:00:00Z,SYNTHETIC_DEMO,fixture-v1,EVD-RETIRE-READINESS
DEMO01,CLAS,ZCL_DEMO_RETIRE,ALTERNATIVE_CANDIDATE,NO,,,2026-10-12T12:00:00Z,SYNTHETIC_DEMO,fixture-v1,EVD-RETIRE-ALTERNATIVE
DEMO01,CLAS,ZCL_DEMO_UNKNOWN,CLOUD_READINESS,UNKNOWN,,,2026-10-12T12:00:00Z,SYNTHETIC_DEMO,fixture-v1,EVD-UNKNOWN-READINESS
```

docs/clean-core/demo/findings.csv

```
system_code,object_type,object_name,finding_code,severity_code,summary,detected_at,source_name,source_reference,source_record_key
DEMO01,CLAS,ZCL_DEMO_REMEDIATE,DEMO_FINDING,HIGH,Synthetic demo finding only,2026-10-12T12:00:00Z,SYNTHETIC_DEMO,fixture-v1,FND-REMEDIATE-001
```

docs/clean-core/demo/dependencies.csv

```
system_code,source_object_type,source_object_name,target_object_type,target_object_name,dependency_type,source_name,source_reference,source_record_key
DEMO01,CLAS,ZCL_DEMO_REMEDIATE,CLAS,ZCL_DEMO_KEEP,CALLS,SYNTHETIC_DEMO,fixture-v1,DEP-REMEDIATE-KEEP-001
```

Supported v1 ConditionCode values are EVIDENCE_GATE and COMPARE. Supported input codes are EVIDENCE_COMPLETE (boolean), CLOUD_READINESS (PASS/FAIL/UNKNOWN), ALTERNATIVE_CANDIDATE (YES/NO/UNKNOWN), and USAGE_COUNT_90D (nonnegative integer). Supported comparison operators are EQ, NE, GT, GE, LT, LE, IS_MISSING, IS_UNKNOWN; unknown condition/input/operator causes publish rejection rather than being interpreted dynamically. A rule-set with complete required data but no matching active rule produces MANUAL_REVIEW and SYSTEM_FALLBACK.

5. Service, method, action, and validation contract

Operation	Input	Output	Validation / error cases	Owner
ZIF_CC_CLASSIFIER~EVALUATE_OBJECT	TY_EVAL_INPUT: object UUID, AsOfUtc, evidence rows, usage facts, findings, dependencies, active rule set/rules	TY_EVAL_RESULT: outcome, rule-set code/version, evidence status, score/band, rationale, missing codes; TY_T_MATCH trace rows	Reject unknown object or absent active rule set; missing/stale/unknown → MANUAL_REVIEW; equal-priority conflict → MANUAL_REVIEW; same input and timestamp must be deterministic	M2
ZCL_CC_RECOMMENDATION_SERVICE~RECALCULATE_OBJECTS	unique object UUID table + explicit evaluation time	persisted recommendation IDs and output summaries	Skip/reject deleted object; persist exact rule version and all match rows in same supported RAP LUW; no independent COMMIT inside a behavior pool	M2; called by M1 batch integration
Import processBatch RAP action	Batch UUID (instance action); staged row records	Batch status and total/persisted/warning/rejected/duplicate/updated counts	Header/version mismatch, malformed quoting/column count, missing mandatory value, invalid enum/negative count, unknown source/object, duplicate conflict; client cannot set row result fields	M1
importCsv RAP action (conditional only)	ImportKind, FileName, CsvText via ZA_CC_CSV_UPLOAD	Same batch summary plus BatchUUID	Create only if Day 1 proves RAP string parameter, FE extension and payload limit; reject malformed/header/row-size issues	M1
Standard read GET entities	OData filters/sort/page on object, evidence, usage, finding, dependency, recommendation, match, decision, audit, portfolio	Bound entity rows	Apply DCL/authorization; validate supported filter fields; no client-side secret or local-only data substitute	M1 supplies data views; M2 recommendation/portfolio views; M3 decision/audit
Create decision	Object UUID + current recommendation UUID	new DRAFT decision	Both records must exist; no existing open decision for that object; outcome copied from snapshot; actor is authenticated principal	M3
assignOwner	owner user name (40 chars)	updated decision	DRAFT only; caller has decision-manager/owner-assignment permission; target resolves in supported identity context	M3
overrideRecommendation	NewOutcome, Reason (1–255 nonblank chars)	updated decision and audit event	DRAFT only; allowed outcome; nonblank reason; authorized actor; changes decision outcome, never changes recommendation snapshot	M3
submitDecision	optional comment ≤255	status SUBMITTED	DRAFT, owner assigned, recommendation still linked, override reason present if overridden, caller has Analyst role and equals OwnerUser	M3
approveDecision	optional comment ≤255	status APPROVED	SUBMITTED, caller is reviewer, caller ≠ SubmittedBy, authorized current instance, ETag/lock matches	M3
publish RuleSet action	RuleSet UUID instance	status ACTIVE, version immutable	Draft only; unique code/version; supported condition/operator/outcome; effective time valid; active rule set selection unambiguous	M2
Portfolio read	source system/component/object type/recommendation/status filters	per-group counts and demo scores	same population/date filter for every count; empty portfolio returns zeros/no row per agreed query shape	M2

Proposed action-role matrix (map aliases only to real tenant roles)

COCKPIT_ANALYST, COCKPIT_REVIEWER, and COCKPIT_ADMIN are project aliases, not SAP-delivered role names. Day 1/13 map them to the actual supported user/role mechanism or leave the action blocked. Authenticated identity is supplied by the SAP environment; RAP performs authorization. No local passwords, hard-coded user lists or UI-only protection.

Operation	Allowed role/principal	Additional instance rule
Read inventory/evidence/recommendation	Authenticated Analyst, Reviewer, Admin within tenant access scope	Apply supported DCL; no anonymous reads unless explicitly approved
Create/assign/override/submit decision	Analyst; Admin may assign but does not implicitly approve	assignOwner DRAFT only; overrideRecommendation DRAFT only; submitDecision DRAFT and caller must equal OwnerUser
Approve decision	Reviewer	SUBMITTED only; caller must differ from SubmittedBy; reviewer cannot override submitted result
Create/edit/publish RuleSet	Admin/rule administrator	Draft only; published versions immutable
processBatch / conditional importCsv	Analyst or designated importer role	Non-production source scope; client cannot set row status, actor or persistence UUID
Read decision audit	Analyst/Reviewer/Admin according to tenant scope	Read-only; ordinary update/delete unavailable

RAP decision action contract (v1)

Action	Input	Allowed initial state	Allowed role	Required validations	State/result change	Audit event	Failure behavior	Positive / negative test
Standard create on ZI_CC_DECISION	ObjectUUID, RecommendationUUID	No open decision exists	Analyst	Both keys exist; no DRAFT/SUBMITTED for object; copy recommendation outcome; ignore client state/actor/time	Create DRAFT	CREATED	Reject with stable OPEN_DECISION_EXISTS/REFERENCE_NOT_FOUND; no partial row	Create valid draft; missing snapshot rejected
assignOwner	OwnerUserName	DRAFT	Analyst; Admin may assign	Target resolves using supported identity directory; caller authorized	OwnerUser updated; status stays DRAFT	OWNER_ASSIGNED	Invalid user/role/state leaves decision unchanged	Valid user assigned; invalid user and APPROVED state rejected
overrideRecommendation	NewOutcome, Reason	DRAFT	Assigned OwnerUser with Analyst role	Outcome in enum; reason 1–255 nonblank; caller matches OwnerUser; recommendation snapshot unchanged	DecisionOutcome changes; IsOverride=X; status stays DRAFT	OVERRIDDEN with exact reason and before/after outcomes	Blank reason/unauthorized actor/invalid outcome leaves state unchanged and writes no success event	Valid reason updates; blank reason rejected
submitDecision	Optional Comment	DRAFT	OwnerUser with Analyst role	Caller equals OwnerUser; owner set; recommendation linked; override reason present when IsOverride=X	Status SUBMITTED; server sets SubmittedBy/SubmittedAt	SUBMITTED	Wrong state/no owner/missing reason denied; no transition/audit on failure	Assigned owner submits; ownerless draft and non-owner call rejected
approveDecision	Optional Comment	SUBMITTED	Reviewer	Reviewer differs from SubmittedBy; role and instance allowed; ETag/lock current	Status APPROVED; server sets ApprovedBy/ApprovedAt	APPROVED	Self-approval/unauthorized/stale state denied; no status change/event	Different reviewer approves; submitter direct-call denied
processBatch (import BO)	Batch UUID; rows already staged	NEW	Analyst/import role	Header, row, reference, idempotency and payload checks; caller cannot set row result fields	Batch reaches COMPLETED/COMPLETED_WITH_ERRORS/FAILED; row records carry outcomes	ImportBatch/ImportRow status and source provenance (not DecisionAudit)	Invalid batch remains inspectable; rejected rows do not partially write facts	Valid batch completes; malformed/unknown reference gets row error
importCsv (conditional import BO)	ImportKind, FileName, CsvText	No batch exists	Analyst/import role	Day-1 proof required; UTF-8/header/payload cap and all row checks; no caller-supplied status/UUID	Creates batch and row outcomes using same importer	Batch/row provenance, no DecisionAudit	Unsupported profile has no action; malformed input returns row/batch errors	Valid file yields same output as staging; over-limit file is rejected
publish (RuleSet BO)	RuleSet UUID	DRAFT	Admin/rule administrator	Unique code/version; allowlisted codes/operators/outcomes; valid dates; unambiguous active version	RuleSet ACTIVE and immutable	PublishedAt and server-derived PublishedBy; no DecisionAudit event	Invalid rule set remains DRAFT with validation message	Valid v1 publish succeeds; unsupported operator/duplicate version rejected

Decision actions and processBatch are enforced in RAP, not merely hidden by the UI. Audit writes are part of the successful transaction. If the tenant cannot provide a trusted principal or role mapping, the relevant action is disabled and acceptance remains blocked.

ABAP interface type details

ZIF_CC_CLASSIFIER owns public TY_EVAL_INPUT, TY_EVIDENCE_INPUT, TY_USAGE_INPUT, TY_FINDING_INPUT, TY_DEPENDENCY_INPUT, TY_RULE_INPUT, TY_MATCH, TY_T_MATCH, and TY_EVAL_RESULT; AsOfUtc is explicit. EVALUATE_OBJECT is pure (no clock/database reads). ZCL_CC_RECOMMENDATION_SERVICE owns database reads/writes and invokes the pure classifier. M1 calls only its released ZIF_CC_RECOMMENDATION_SERVICE contract; it does not write recommendation tables. Rule expressions are allowlisted operators/codes—never dynamic ABAP, user code, or SQL built from CSV.

Exact ABAP service structures and action parameters

Structure / element	Type	Required?	Contract meaning
TY_EVAL_INPUT-ObjectUUID	UUID16	Yes	Object being classified
TY_EVAL_INPUT-AsOfUtc	TS_UTC	Yes	Injected evaluation time; no implicit system clock
TY_EVAL_INPUT-RuleSetUUID	UUID16	Yes	Exact active RuleSet version selected by recommendation service
TY_EVAL_INPUT-RuleSetCode, RuleSetVersion	CHAR(20), CHAR(10)	Yes	Version labels copied to pure result
TY_EVAL_INPUT-Evidence	STANDARD TABLE OF TY_EVIDENCE_INPUT	Yes (may be empty)	All latest evidence rows for this object
TY_EVAL_INPUT-UsageFacts	STANDARD TABLE OF TY_USAGE_INPUT	Yes (may be empty)	Usage snapshots with period/capture timestamps
TY_EVAL_INPUT-Findings	STANDARD TABLE OF TY_FINDING_INPUT	Yes (may be empty)	Severity codes for risk formula
TY_EVAL_INPUT-Dependencies	STANDARD TABLE OF TY_DEPENDENCY_INPUT	Yes (may be empty)	Dependency rows for risk formula
TY_EVAL_INPUT-Rules	STANDARD TABLE OF TY_RULE_INPUT	Yes	Active rules in deterministic order
TY_EVIDENCE_INPUT	EvidenceCode CHAR(30), ValueText CHAR(80), ValueNumber DEC(15,3), ValueDate DATS, QualityCode CHAR(12), CapturedAt TS_UTC	Per row	Typed values; one effective value type per record
TY_USAGE_INPUT	PeriodFrom DATS, PeriodTo DATS, ExecutionCount INT8, LastUsedAt TS_UTC, CapturedAt TS_UTC	Per row	Period and freshness evidence
TY_FINDING_INPUT	FindingCode CHAR(30), SeverityCode CHAR(10)	Per row	Imported normalized source code; only HIGH/ERROR affect proposed score
TY_DEPENDENCY_INPUT	DependencyType CHAR(20)	Per row	One dependency contributes 5 points up to four
TY_RULE_INPUT	RuleUUID UUID16, RuleID CHAR(30), Priority INT4, ConditionCode CHAR(30), InputCode CHAR(30), Operator CHAR(12), CompareValue CHAR(80), OutcomeCode CHAR(20), ExplanationText CHAR(255), IsActive CHAR(1)	Per rule	Frozen allowlisted rule configuration
TY_EVAL_RESULT	ObjectUUID UUID16, RuleSetUUID UUID16, RuleSetCode CHAR(20), RuleSetVersion CHAR(10), OutcomeCode CHAR(20), EvidenceStatus CHAR(15), RiskScore INT4, PriorityBand CHAR(6), Rationale CHAR(255), MissingEvidenceCodes CHAR(255), MatchedRuleCount INT4	All except missing-code list may be empty	Pure evaluation result; no DB-generated RecommendationUUID
TY_MATCH / TY_T_MATCH row	RuleUUID UUID16 optional, RuleID CHAR(30), Priority INT4, OutcomeCode CHAR(20), ExplanationText CHAR(255), IsSelected CHAR(1)	One row per matched/fallback rule	Pure trace; recommendation service assigns MatchUUID/RecommendationUUID
TY_RECALCULATE_RESULT	ObjectUUID UUID16, RecommendationUUID UUID16, OutcomeCode CHAR(20), RuleSetVersion CHAR(10), EvidenceStatus CHAR(15), MatchedRuleCount INT4, ErrorCode CHAR(30), ErrorText CHAR(255)	ObjectUUID required	M1 receives one result per unique object; error fields set only on that object's failure
TY_IMPORT_ROW_INPUT	BatchUUID UUID16, RowNumber INT4, EntityKind CHAR(20), RawRow CHAR(512)	Required for staged fallback	Client-stageable values only; first physical row is the exact CSV header
TY_IMPORT_ROW_RESULT	RowNumber INT4, RowStatus CHAR(20), ErrorCode CHAR(30), ErrorText CHAR(255), ResolvedUUID UUID16	One per physical data row	Backend-derived outcome; status/error cannot be client-authored
TY_BATCH_SUMMARY	BatchUUID UUID16, BatchStatus CHAR(20), TotalRows INT4, AcceptedRows INT4, WarningRows INT4, RejectedRows INT4, DuplicateRows INT4, UpdatedRows INT4	All required	Mutually exclusive counters reconcile to TotalRows

ZIF_CC_IMPORTER owns TY_IMPORT_ROW_INPUT, TY_IMPORT_ROW_RESULT, TY_BATCH_SUMMARY, and these methods: PARSE_CSV(ImportKind, CsvText OR staged raw rows) → parsed rows/header errors; VALIDATE_ROW(ImportKind, RowNumber, RawRow) → typed row result; PROCESS_BATCH(BatchUUID) → TY_BATCH_SUMMARY + row results. In staging mode, ImportRow.RowNumber=1 carries the exact header and following rows retain their physical line numbers; embedded line breaks are prohibited by CSV v1.

ZIF_CC_CLASSIFIER~EVALUATE_OBJECT imports IS_INPUT TYPE TY_EVAL_INPUT and returns ES_RESULT TYPE TY_EVAL_RESULT plus ET_MATCHES TYPE TY_T_MATCH. ZIF_CC_RECOMMENDATION_SERVICE~RECALCULATE_OBJECTS imports a unique UUID table and IV_AS_OF_UTC; exports a result row per input object. GET_LATEST_FOR_OBJECT takes ObjectUUID and returns the latest Recommendation plus matches or NOT_FOUND; GET_BY_UUID returns an immutable historical snapshot. Internal services use typed RAP reported/failed messages or the agreed error fields, never free-form exception text as a status code.

Abstract action parameter entity	Fields and types	UI/backend validation
ZA_CC_ASSIGN_OWNER	OwnerUserName CHAR(40)	Required, nonblank, resolves through supported identity context
ZA_CC_OVERRIDE	NewOutcome CHAR(20), Reason CHAR(255)	Both required; reason 1–255 non-whitespace chars
ZA_CC_SUBMIT	Comment CHAR(255)	Optional; max 255
ZA_CC_APPROVE	Comment CHAR(255)	Optional; max 255
ZA_CC_CSV_UPLOAD (conditional only)	ImportKind CHAR(20), FileName CHAR(128), CsvText STRING	Create only if Day 1 proves action-type/runtime size support; hard cap 256 KB proposed; otherwise this entity does not exist
ImportBatch create (fallback)	ImportKind CHAR(20), FileName CHAR(128)	Server initializes status/actor/time/counts
ImportRow stage create (fallback)	BatchUUID UUID16, RowNumber INT4, EntityKind CHAR(20), RawRow CHAR(512)	Server owns row status/error/resolved UUID; processBatch validates row before persistence

Exact stable error codes: import MALFORMED_CSV, HEADER_MISMATCH, COLUMN_COUNT_MISMATCH, ROW_TOO_LONG, MISSING_REQUIRED_FIELD, INVALID_ENUM_VALUE, INVALID_EVIDENCE_VALUE, INVALID_EXECUTION_COUNT, INVALID_PERIOD, UNKNOWN_SOURCE_SYSTEM, UNKNOWN_OBJECT, UNKNOWN_TARGET_OBJECT, DUPLICATE_CONFLICT, IMPORT_KIND_MISMATCH, BATCH_ALREADY_PROCESSED; workflow REFERENCE_NOT_FOUND, OPEN_DECISION_EXISTS, OWNER_REQUIRED, USER_NOT_FOUND, OVERRIDE_REASON_REQUIRED, INVALID_OUTCOME, INVALID_TRANSITION, UNAUTHORIZED, SELF_APPROVAL, ETAG_CONFLICT; rules NO_ACTIVE_RULESET, INVALID_RULE_CONFIGURATION. A no-match is a valid MANUAL_REVIEW result, not a technical error.

Proposed Fiori screen contract

Route/page	Fiori artifact	Entity/service consumed	Display/filter/action contract
CustomObjects List Report	Generated FE List Report on ZC_CC_CUSTOM_OBJECT	ZUI_CC_COCKPIT	Search ObjectName; filter SystemCode/ObjectType/SoftwareComponent/OwnerUser/Outcome/PriorityBand as metadata allows; navigate by ObjectUUID
CustomObjects Object Page	Generated FE Object Page	CustomObject associations to facts/recommendation/decision	Header identity; sections for source, usage, evidence, findings, dependencies, recommendation/matches, decision, audit; show explicit empty/missing states
ImportBatches report/object page	Generated FE page or supported import extension	ZC_CC_IMPORT_BATCH, ZC_CC_IMPORT_ROW	Create/process supported batch; show counts and row-level status/error; upload button exists only under verified file-action profile
Portfolio	FE chart/list annotations or generator-supported custom page	ZC_CC_PORTFOLIO	Total/outcome/incomplete/open/priority metrics; filter by source/component/type/outcome/priority; drill-down reuses same filter set

6. Four-member ownership and handoff matrix

Member	Owns and creates	May consume	Must hand off
M1 — ABAP Data & Import	Tables/views for SourceSystem, CustomObject, UsageFact, Evidence, Finding, Dependency, ImportBatch/Row; CSV templates/parser/validator/import behavior; provenance/idempotency	M2 classifier/recommendation service interface; M3 security annotations; M4 UI read service	Activated inventory/evidence CDS fields Day 4; import API/error contract Day 7; idempotent persisted evidence Day 8; affected-object list/recalc call Day 10
M2 — Rules Engine & Analytics	RuleSet/ClassificationRule, type/classifier interfaces, recommendation snapshots/matches, explanation/completeness/freshness, risk/priority, portfolio read model	M1 activated entities; M3 decision context; M4 service/UI contract	ZIF_CC_CLASSIFIER Day 3; completeness interface Day 8; outputs/rule trace Day 9; import-triggered recommendation service Day 10; portfolio fields Day 17
M3 — RAP Workflow & Security	MigrationDecision root/BDEF/pool/actions, authorization/DCL/roles integration, DecisionAudit, workflow service exposure	M1 object keys; M2 recommendation snapshot; M4 action UX requirements	State/action contract Day 2; decision root Day 3; actions Day 12; authorization/audit Day 13; activated decision/audit metadata Day 15
M4 — Fiori UI, QA & Integration	Supported Fiori Elements app/configuration/annotations, pages/filters/navigation/errors, E2E/regression evidence, demo and presentation docs	Activated M1/M2 read entities and M3 action/service metadata	Integration checklist Day 3; list page Day 4; first slice Day 5; final workflow UI Day 15; test/screenshot/demo evidence Days 18–20

Checkpoint rule: 15-minute daily API/schema checkpoint. M1 owns data-contract changes; M2 rule outputs; M3 lifecycle/security; M4 frontend routing/annotations. The owner raises a written change in docs/clean-core/contracts/change-log.md, lists affected consumers/tests, and gets all impacted owners' approval before editing shared artifacts. Blockers go in docs/clean-core/blockers.md with owner, next action, and due day; never silently rename a field.

7. Day 1–20 detailed execution plan

Every task below is actionable only under the Day-1 capability profile. Exact object names remain proposed until ADT confirms package, namespace, language version, and object-type availability. Backend "commit" means record the supported transport/task; local Git commits are for local docs/UI only.

DAY 1 — Verify real environment, access, and MVP gates

M1-D01-T01 — ABAP data/persistence probe

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** Local `docs/clean-core/environment/day1-data-capability.md`; ADT probe table `ZCC_D1_PROBE`, root CDS `ZI_CC_D1_PROBE`, class `ZCL_CC_D1_PROBE` with local `FOR TESTING` class `ltcl_probe` in the assigned package.
- **PURPOSE:** Prove the assigned developer can create/activate customer persistence, CDS and a class/test in the actual tenant, not infer it from this repository.
- **PRECONDITIONS:** SAP tenant, ADT connection, user/package assignment; M1 records URL/tenant alias only, never credentials.
- **IMPLEMENTATION STEPS:** 1. Create/use package. 2. Define key `PROBE_ID CHAR(8)` and `PROBE_VALUE CHAR(20)`. 3. Define `ZI_CC_D1_PROBE` as a root view entity suitable for the RAP probe. 4. Add `ZCL_CC_D1_PROBE=>READ_PROBE` and local ABAP Unit `probe_table_round_trip` using supported test transaction semantics. 5. Record object types, release checks, unique-key/lock support, seed path, and transport ID.
- **FIELDS / METHODS / ACTIONS:** `PROBE_ID`, `PROBE_VALUE`; method `READ_PROBE`; local test `probe_table_round_trip`.
- **EXPECTED BEHAVIOR:** One fixture is readable through CDS and the class; activation/test succeed without SAP namespace changes.
- **TEST CASES:** Insert `P1/OK`; expect exactly one row with `PROBE_VALUE=OK`; duplicate key behavior recorded.
- **ACCEPTANCE CRITERIA:** Table, root CDS, class and test all activate/pass; evidence states object types, package, transport and blockers. If persistence is unavailable, stop table-dependent work and apply §3 alternative.
- **DEPENDENCIES:** Assigned SAP environment; M3 can reuse the probe CDS for RAP verification after M1 activates it.
- **COMMIT / DELIVERABLE:** `day1-data-capability.md`, probe object list and transport/task identifier.

M2-D01-T01 — Language, types, and unit-test probe

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** ADT class `ZCL_CC_D1_RULE_PROBE` with local test class; local `docs/clean-core/environment/day1-rules-capability.md`.
- **PURPOSE:** Verify ABAP Cloud language version, global class activation, ABAP Unit, type support and released UUID/time APIs needed by classification.
- **PRECONDITIONS:** Assigned package and M1 package result; no production data.
- **IMPLEMENTATION STEPS:** 1. Add public method `NORMALIZE_INPUT`. 2. Use proposed UUID16, `INT8`, `DEC(15,3)`, `DATS`, UTC timestamp, and ABAP_BOOL in a small local type. 3. Add `FOR TESTING` assertions for normalization and comparison. 4. Record syntax/release failures and supported replacements in Day-1 matrix.
- **FIELDS / METHODS / ACTIONS:** `NORMALIZE_INPUT`; local test `normalizes_known_code`.
- **EXPECTED BEHAVIOR:** `pass` normalizes to `PASS`; numeric zero stays zero; timestamp comparison is UTC and deterministic.
- **TEST CASES:** `pass → PASS`, `0 → 0`, missing → explicit unknown; expected all assertions pass.
- **ACCEPTANCE CRITERIA:** Class activates, ABAP Unit passes, no unreleased API is retained; type fallbacks are agreed before Day 2.
- **DEPENDENCIES:** Package/language version from M1; M1's probe only if shared package access is required.
- **COMMIT / DELIVERABLE:** `day1-rules-capability.md`, class/test names, transport/task ID.

M3-D01-T01 — RAP, service, identity, and authorization probe

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** After M1 activates `ZI_CC_D1_PROBE`, proposed BDEF `ZI_CC_D1_PROBE`, behavior pool `ZBP_I_CC_D1_PROBE`, service definition `ZUI_CC_D1_PROBE`, and binding `ZUI_CC_D1_PROBE_V4` or supported fallback `ZUI_CC_D1_PROBE_V2`; local `day1-rap-security-capability.md`.
- **PURPOSE:** Prove which RAP operations and authenticated-user/role mechanisms exist before scheduling decisions or approvals.
- **PRECONDITIONS:** M1 probe CDS activated; a non-production test principal and tenant-admin contact.
- **IMPLEMENTATION STEPS:** 1. Add minimal managed create/read root behavior. 2. Add one `ping` action or simple create validation. 3. Activate service definition/binding. 4. Verify preview/test call, current-principal API, role assignment source, global/instance authorization hook, lock/ETag and stale-update response. 5. Record each as supported/unsupported; remove no shared object without owner approval.
- **FIELDS / METHODS / ACTIONS:** Probe create; `ping`; validation/auth hooks as available.
- **EXPECTED BEHAVIOR:** Authorized test user reads/creates one probe; missing role is rejected if supported; stale version behavior is observed or marked unavailable.
- **TEST CASES:** Create `P2`; call with allowed role; call without role; replay old ETag. Record actual HTTP/ADT results, not assumed outputs.

- **ACCEPTANCE CRITERIA:** RAP/service version and principal/role mechanism are documented. If no trusted identity/authorization exists, approve only a dev-only profile and mark production workflow blocked.
- **DEPENDENCIES:** M1 probe; tenant-provided role assignment and test user.
- **COMMIT / DELIVERABLE:** Capability matrix, service metadata/preview evidence, transport/task ID; no secret or token committed.

M4-D01-T01 — Fiori workflow and deployment probe

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** Local `docs/clean-core/environment/day1-fiori-capability.md` and canonical `docs/clean-core/environment/day1-capability-matrix.md`; if supported, generated `ui/cc-migration-cockpit/` Fiori Elements skeleton from the probe service.
- **PURPOSE:** Determine the actual Fiori Elements generation, OData consumption, preview, authentication handoff, and deployment path.
- **PRECONDITIONS:** M3 probe service metadata/binding and permitted BAS/Fiori Tools/ADT/launchpad access.
- **IMPLEMENTATION STEPS:** 1. Run the tenant-approved FE generator/preview against probe metadata. 2. Verify List Report render, route, user sign-in, action metadata, and allowed deployment target. 3. Record generated file paths/template/version. 4. Collect M1–M3 probe reports into `day1-capability-matrix.md`, select one supported profile, and list blockers/fallbacks. 5. If generator/deployment is unavailable, test only the tenant's supported FE preview and document limits; do not create a React replacement.
- **FIELDS / METHODS / ACTIONS:** Probe `PROBE_ID`, `PROBE_VALUE`; generated manifest data source and route.
- **EXPECTED BEHAVIOR:** A supported UI displays the one probe row under the actual preview/deployment URL; browser request does not depend on localhost.
- **TEST CASES:** Load list; refresh direct route; unauthenticated/expired session behavior; service unavailable state.
- **ACCEPTANCE CRITERIA:** One supported profile is chosen: FE local project or supported FE preview. Record exact generator file paths and deployment mechanism; if none, UI delivery is blocked.
- **DEPENDENCIES:** M3 activated service binding and M1 probe row.
- **COMMIT / DELIVERABLE:** Capability report; generated skeleton commit only if it belongs to the supported app workflow.

DAY 2 — Freeze schema, enums, service contracts, synthetic inputs

M1-D02-T01 — Freeze data and CSV schema

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** `docs/clean-core/contracts/data-v1.md`, to be merged into `docs/clean-core/contracts/mvp-contract-v1.md` by M4.
- **PURPOSE:** Make every persistence field, key, relationship, CSV header, and error code shared before parallel coding.
- **PRECONDITIONS:** Day-1 data capability report; PDF still required for final reconciliation.
- **IMPLEMENTATION STEPS:** 1. Transcribe M1 entity rows from §4. 2. Specify six exact CSV header lines and field limits. 3. Specify business keys/upsert rules and row statuses. 4. List DDL feature substitutions required by the tenant. 5. Send data-contract diff to M2/M3/M4 for sign-off.
- **FIELDS / METHODS / ACTIONS:** Entity fields in §4; CSV kinds and header names recorded in §8.
- **EXPECTED BEHAVIOR:** All consumers refer to `ObjectUUID`, `SourceSystemUUID`, `SourceRecordKey`, `ImportBatchUUID` exactly.
- **TEST CASES:** Contract check: every FK points to a declared key; each CSV header maps to one field/type; repeated key behavior is explicit.
- **ACCEPTANCE CRITERIA:** M2–M4 approve v1; no open type/name question; unresolved PDF conflict is marked, not guessed.
- **DEPENDENCIES:** Day-1 profile; M2/M3/M4 contract sections.
- **COMMIT / DELIVERABLE:** Data contract section and M1 transport/work item for Day 3.

M2-D02-T01 — Freeze rule, classifier, and analytics contract

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** `docs/clean-core/contracts/rules-v1.md`.
- **PURPOSE:** Publish identical recommendation values, input structures, missing-data policy, demo priorities, formula, and output schema.
- **PRECONDITIONS:** M1 field list; Day-1 supported types/current-user/time API; all proposed policy values explicitly marked pending PDF.
- **IMPLEMENTATION STEPS:** 1. Define classifier input/result/match structures. 2. Add rule IDs/priorities/conditions/outcomes/explanation and missing/conflict behavior from §4. 3. Freeze the risk formula and freshness test boundary. 4. Define portfolio grouping keys/metric names. 5. Review with M1/M3/M4.

- **FIELDS / METHODS / ACTIONS:** `EVALUATE_OBJECT`; outputs `OutcomeCode`, `EvidenceStatus`, `RiskScore`, `PriorityBand`, `Rationale`, `MissingEvidenceCodes`, `MatchedRuleIDs` via match rows.
- **EXPECTED BEHAVIOR:** A consumer can build a caller without inventing a type/value.
- **TEST CASES:** `PASS + usage 25 + candidate NO → KEEP`; `FAIL + complete evidence → REMEDIATE`; missing/stale → `MANUAL_REVIEW`; tie conflict → `MANUAL_REVIEW`.
- **ACCEPTANCE CRITERIA:** M1 confirms fields exist; M3 accepts recommendation snapshot link; M4 accepts displayed labels; policy is visibly tagged demo-only.
- **DEPENDENCIES:** M1 data contract; PDF reconciliation is a named blocker if absent.
- **COMMIT / DELIVERABLE:** Rules contract and decision log entries for every proposed policy.

M3-D02-T01 — Freeze lifecycle, roles, and action contract

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** `docs/clean-core/contracts/workflow-v1.md`.
- **PURPOSE:** Make state transitions, roles, validations, audit events, and stale-update behavior unambiguous.
- **PRECONDITIONS:** Day-1 RAP identity/authorization report; M1 ObjectUUID and M2 RecommendationUUID contracts.
- **IMPLEMENTATION STEPS:** 1. Define DRAFT→SUBMITTED→APPROVED transitions. 2. Define action inputs/errors in §5. 3. Define draft owner, override reason, authenticated actor, reviewer ≠ submitter. 4. Map roles only to supported tenant role mechanism. 5. Mark unsupported gates explicitly.
- **FIELDS / METHODS / ACTIONS:** `assignOwner`, `overrideRecommendation`, `submitDecision`, `approveDecision`; status/outcome/audit fields.
- **EXPECTED BEHAVIOR:** No ambiguous status/action/role spelling remains for M4.
- **TEST CASES:** Valid path; override without reason; approval by submitter; illegal transition; role-less call; stale ETag.
- **ACCEPTANCE CRITERIA:** M1/M2/M4 sign off; no hard-coded identity; production acceptance blocked if platform role mapping is absent.
- **DEPENDENCIES:** Day-1 identity/locking profile and M1/M2 contracts.
- **COMMIT / DELIVERABLE:** Workflow contract and verified role/identity decision.

M4-D02-T01 — Merge and publish canonical contract

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** `docs/clean-core/contracts/mvp-contract-v1.md`, `docs/clean-core/contracts/change-log.md`, `docs/clean-core/requirements-traceability.md`.
- **PURPOSE:** Publish one canonical contract and map every visible field/action to the service that supplies it.
- **PRECONDITIONS:** M1 data, M2 rules, M3 workflow sections; Day-1 UI profile.
- **IMPLEMENTATION STEPS:** 1. Merge agreed fields/enums/operations without renaming. 2. Add screen→entity/field/action matrix. 3. Mark every proposed value as `PDF CONFIRMATION REQUIRED`. 4. Add approvals and freeze timestamp. 5. Create blocker log.
- **FIELDS / METHODS / ACTIONS:** Inventory columns, object sections, portfolio metric IDs, workflow button/action IDs and route names.
- **EXPECTED BEHAVIOR:** UI consumers can bind without guessed OData property names.
- **TEST CASES:** Search the merged file for duplicate/mismatched enum spellings; verify all displayed fields occur in §4 and every action in §5.
- **ACCEPTANCE CRITERIA:** All four owners approve one v1; no coding against unapproved local schemas.
- **DEPENDENCIES:** Three owner sections; Day-1 Fiori path.
- **COMMIT / DELIVERABLE:** Canonical contract, traceability matrix, `blockers.md`, Git commit.

DAY 3 — Start the first thin backend/UI slice from the frozen contract

M1-D03-T01 — Create source-system and custom-object persistence

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** ADT tables `ZCC_SRCSYS`, `ZCC_OBJECT`; CDS interface entities `ZI_CC_SOURCE_SYSTEM`, `ZI_CC_CUSTOM_OBJECT`; global read class `ZCL_CC_INVENTORY_REPOSITORY` with local ABAP Unit class `ltcl_inventory_repository`.
- **PURPOSE:** Persist and read the minimum object inventory using the frozen schema.
- **PRECONDITIONS:** Day-1 table/CDS pass; Day-2 contract v1; source system is the parent.
- **IMPLEMENTATION STEPS:** 1. Create tables with UUID primary keys and contract fields. 2. Add association from object to source system and authorization-check annotation. 3. Activate dependency order (table→source CDS→object CDS). 4. Implement `GET_SOURCE_SYSTEM`, `GET_OBJECT_BY_BUSINESS_KEY` and `CHECK_OBJECT_NATURAL_KEY` in the repository. 5. Add local tests for association and duplicate-key result.
- **FIELDS / METHODS / ACTIONS:** Fields exactly as §4; association `_SourceSystem`; repository methods named above; no guessed columns.

- **EXPECTED BEHAVIOR:** Object row resolves to one source system; missing parent/duplicate key is returned as a typed validation result for the owning write service.
- **TEST CASES:** `SYS1/CLAS/ZCL_DEMO` resolves; same source/type/name is flagged duplicate; absent `SYS1` returns `UNKNOWN_SOURCE_SYSTEM`.
- **ACCEPTANCE CRITERIA:** Tables, CDS and repository activate; local tests pass; field names match v1 byte-for-byte.
- **DEPENDENCIES:** Contract v1 and package assignment; M2/M3/M4 consume the activated CDS entities.
- **COMMIT / DELIVERABLE:** ADT transport/task ID; activation/test evidence; update object inventory.

M2-D03-T01 — Create versioned rules and classifier/recommendation contracts

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** ADT tables `ZCC_RULESET`, `ZCC_RULE`, `ZCC_RECOMM`, `ZCC_RMATCH`; CDS `ZI_CC_RECOMMENDATION`, `ZI_CC_RULE_MATCH`; interfaces `ZIF_CC_TYPES`, `ZIF_CC_CLASSIFIER`, `ZIF_CC_RECOMMENDATION_SERVICE`.
- **PURPOSE:** Publish the stable rule configuration, typed pure-evaluation contract, and minimal recommendation read shape needed for Day 5.
- **PRECONDITIONS:** Contract v1; M1 type names; ABAP Unit/type probe and persistence passed.
- **IMPLEMENTATION STEPS:** 1. Create RuleSet/Rule, Recommendation and RecommendationMatch tables with fields from §4. 2. Define shared outcome/status constants in `ZIF_CC_TYPES`. 3. Define classifier `TY_*` structures/table types and `EVALUATE_OBJECT` signature. 4. Define `RECALCULATE_OBJECTS` taking object UUIDs and explicit AsOfUtc and returning typed summaries. 5. Create the Recommendation/Match CDS read entities; activate tables before CDS/interfaces; do not implement dynamic expressions.
- **FIELDS / METHODS / ACTIONS:** Inputs include ObjectUUID, AsOfUtc, evidence, usage, finding/dependency counts and rules; outputs include outcome, version, evidence status, score/band, rationale, missing codes and match table.
- **EXPECTED BEHAVIOR:** M1/M3 compile against public names; missing-evidence recommendation has a persisted read model for the first slice.
- **TEST CASES:** Compile tests constructing `TY_EVAL_INPUT`/recalculation result; assert constants equal the v1 enum strings.
- **ACCEPTANCE CRITERIA:** Tables/view/interfaces activate; M1/M3 acknowledge signatures; outputs match contract; snapshot rows are immutable through service policy.
- **DEPENDENCIES:** M1 data contract; Day-1 released type/persistence support.
- **COMMIT / DELIVERABLE:** Table/view/interface transport, test evidence and signatures published to team.

M3-D03-T01 — Create minimum decision root

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** ADT table `ZCC_DECISION`; CDS root `ZI_CC_DECISION`; local `docs/clean-core/contracts/workflow-v1.md` lifecycle acceptance table.
- **PURPOSE:** Persist a decision linked to the inventory object and future recommendation without enabling unreviewed transitions.
- **PRECONDITIONS:** M1 `ZI_CC_CUSTOM_OBJECT` activated; workflow contract v1; UUID/time types verified.
- **IMPLEMENTATION STEPS:** 1. Create table fields from §4, including recommendation snapshot reference and ETag timestamps. 2. Define root view/key and association to object. 3. Activate; keep action/authorization implementation for Days 11–13. 4. Record initial-state and link constraints.
- **FIELDS / METHODS / ACTIONS:** DecisionUUID, ObjectUUID, RecommendationUUID, status, outcome, override/owner, submit/approve fields, created/last-changed fields.
- **EXPECTED BEHAVIOR:** CDS exposes a decision row with `DRAFT`; no approve/override action is exposed yet.
- **TEST CASES:** Create a fixture against existing object; missing ObjectUUID validation fails; invalid state string is rejected by agreed validation seam.
- **ACCEPTANCE CRITERIA:** Table and root activate and names match M2/M4 contract; no auth bypass or premature public write operation.
- **DEPENDENCIES:** M1 object CDS; M2 RecommendationUUID contract; Day-1 ETag type.
- **COMMIT / DELIVERABLE:** ADT transport/task ID and decision root metadata.

M4-D03-T01 — Generate application skeleton and integration checklist

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** Supported-profile artifact `ui/cc-migration-cockpit/` (generator-owned) **or**, if the tenant has no local app workflow, `docs/clean-core/ui/fiori-preview-runbook.md`; integration checklist `docs/clean-core/ui/integration-checklist.md`.
- **PURPOSE:** Create only the Fiori Elements structure proven on Day 1 and list its exact metadata dependencies.
- **PRECONDITIONS:** Frozen contract, Day-1 Fiori profile, M1/M3 entity names.
- **IMPLEMENTATION STEPS:** 1. Generate V4 FE List Report/Object Page from the probe/service metadata; retain generator output. 2. Configure app namespace/route and service placeholder. 3. Add integration checklist for object

list/detail/evidence/recommendation/decision/audit/portfolio. 4. If preview-only, document the exact preview launch steps instead of fabricating local paths.

- **FIELDS / METHODS / ACTIONS:** Initial route `CustomObjects`; fields `ObjectName`, `ObjectType`, `SourceSystemUUID`; no custom upload UI yet.
- **EXPECTED BEHAVIOR:** Generated app or supported preview opens and displays probe metadata; no mock React backend.
- **TEST CASES:** Build/preview generated app; direct route refresh; note exact missing entity/action metadata.
- **ACCEPTANCE CRITERIA:** Actual generator paths/template/version are recorded; app config does not point at localhost.
- **DEPENDENCIES:** Day-1 UI profile and M1/M3 probe service.
- **COMMIT / DELIVERABLE:** UI skeleton/runbook and checklist; Git commit if files are local.

DAY 4 — Activate the first inventory read service and connect the first list

M1-D04-T01 — Add inventory projections and UI metadata

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** CDS projections `ZC_CC_SOURCE_SYSTEM`, `ZC_CC_CUSTOM_OBJECT`; metadata extension `ZC_CC_OBJECT_MDE` (ADT DDLX, proposed).
- **PURPOSE:** Provide stable UI-ready inventory fields without exposing persistence tables directly.
- **PRECONDITIONS:** Day-3 tables/interface entities active; M4 generated app path known.
- **IMPLEMENTATION STEPS:** 1. Project contract fields and `_SourceSystem`. 2. Add labels/selection/search annotations for object name/type/source/component. 3. Keep projections read-only until importer owns writes. 4. Activate and inspect service metadata.
- **FIELDS / METHODS / ACTIONS:** `ObjectUUID`, `ObjectName`, `ObjectType`, `SystemCode`, `PackageName`, `SoftwareComponent`, `Description`, `OwnerUser`.
- **EXPECTED BEHAVIOR:** One OData entity row carries the agreed names and searchable object field.
- **TEST CASES:** Query `CL_DEMO` and filter by `ObjectType=CLAS`; check source association and empty result.
- **ACCEPTANCE CRITERIA:** Both projections/MDE activate; no renamed CDS property; metadata is consumable by RAP service.
- **DEPENDENCIES:** M1 Day-3 CDS; M4 app route.
- **COMMIT / DELIVERABLE:** ADT transport/task ID and `$metadata` field checklist.

M2-D04-T01 — Implement safe classifier skeleton and first ABAP Unit tests

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** Global class `ZCL_CC_CLASSIFIER`; local ABAP Unit class `ltcl_classifier` in its class pool.
- **PURPOSE:** Implement a deterministic baseline before importing all evidence types.
- **PRECONDITIONS:** `ZIF_CC_CLASSIFIER`, `ZIF_CC_TYPES`, contract v1 active.
- **IMPLEMENTATION STEPS:** 1. Implement method `EVALUATE_OBJECT` using only passed input. 2. Return `MANUAL_REVIEW` with `SYSTEM_FALLBACK` when required evidence is absent. 3. Build rationale/missing-code output. 4. Do not read system time/database in the pure method.
- **FIELDS / METHODS / ACTIONS:** `EVALUATE_OBJECT`; exact `TY_EVAL_INPUT/RESULT/MATCH` signature.
- **EXPECTED BEHAVIOR:** Empty evidence never yields `KEEP/RETIRE/REPLACE/REMEDIATE`.
- **TEST CASES:** Empty required fields → `MANUAL_REVIEW`, `EvidenceStatus=INCOMPLETE`, missing codes listed; same input twice → identical output.
- **ACCEPTANCE CRITERIA:** Class activates; tests pass; no SAP-specific rule claim appears in explanation.
- **DEPENDENCIES:** M2 Day-3 interface; M1 entity names in test input.
- **COMMIT / DELIVERABLE:** Class/test transport and test-run evidence.

M3-D04-T01 — Expose the first inventory read service

- **OWNER:** M3 — RAP Workflow & Security (service binding integration); M1 remains owner of data/projections.
- **OBJECT TO CREATE OR MODIFY:** Service definition `ZUI_CC_COCKPIT` and binding `ZUI_CC_COCKPIT_V4`, or fallback `ZUI_CC_COCKPIT_V2` if that is the Day-1 supported FE-compatible profile (ADT types `SRVD`/`SRVB`).
- **PURPOSE:** Expose inventory CDS through the verified RAP OData service so M4 can bind the real backend.
- **PRECONDITIONS:** M1 projections activated; Day-1 OData profile; no unsupported entity included.
- **IMPLEMENTATION STEPS:** 1. Expose `ZC_CC_SOURCE_SYSTEM` and `ZC_CC_CUSTOM_OBJECT`. 2. Activate definition and binding. 3. Publish/enable service in the approved tenant mechanism. 4. Record URL through secure team channel, never put credentials in Git.

- **FIELDS / METHODS / ACTIONS:** Inventory projections only; read/list/search supported by binding.
- **EXPECTED BEHAVIOR:** Service returns source/object data and metadata with the contract names.
- **TEST CASES:** Call list, key lookup, filter and `$metadata` using authorized tenant client; unauthorized access result recorded.
- **ACCEPTANCE CRITERIA:** M4 confirms consumable metadata; service profile exactly matches Day-1 choice; no localhost endpoint.
- **DEPENDENCIES:** M1 Day-4 projections and M3 Day-1 service capability.
- **COMMIT / DELIVERABLE:** ADT transport/task ID, service binding name, sanitized request/response evidence.

M4-D04-T01 — Bind the first Fiori inventory List Report

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** Generated app `ui/cc-migration-cockpit/webapp/manifest.json` and generator-owned List Report annotations/config; preview-only profile uses `ZC_CC_OBJECT_MDE` and `fiori-preview-runbook.md`.
- **PURPOSE:** Render live backend inventory data, not hard-coded frontend data.
- **PRECONDITIONS:** M3 service binding active; M1 projection metadata; Day-1 selected frontend profile.
- **IMPLEMENTATION STEPS:** 1. Replace probe URI with cockpit service root. 2. Configure `CustomObjects` entity set/route. 3. Show object name/type/source/component. 4. Add basic object-name search and source/type filters using metadata. 5. Verify authentication is platform-managed.
- **FIELDS / METHODS / ACTIONS:** `ObjectName`, `ObjectType`, `SystemCode`, `SoftwareComponent`; no workflow actions yet.
- **EXPECTED BEHAVIOR:** List renders backend rows; empty/error state is distinguishable from a valid empty dataset.
- **TEST CASES:** Existing row appears; search `CL_DEMO` returns one; unknown query returns empty state; service 401/500 shows error.
- **ACCEPTANCE CRITERIA:** List runs on supported preview/deployment, calls the actual binding, and uses no local mock/localhost URL.
- **DEPENDENCIES:** M1 projections → M3 binding → M4 app.
- **COMMIT / DELIVERABLE:** UI commit or preview screenshot and binding trace.

DAY 5 — Demonstrate the first thin end-to-end slice

M1-D05-T01 — Provide one safe synthetic inventory fixture

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** `ZCL_CC_DEMO_SEED` (ADT class, restricted to the verified non-production seed mechanism), `docs/clean-core/demo/day5-source-systems.csv`, and `docs/clean-core/demo/day5-object.csv`.
- **PURPOSE:** Supply one deterministic source and object for the first real service-to-UI demonstration before full CSV import is complete.
- **PRECONDITIONS:** M1 persistence and M3 read service active; Day-1 confirms approved dev-only seed/write mechanism. If none exists, do not add a backdoor: mark the persisted slice blocked and seed via the earliest supported Day-7 importer instead.
- **IMPLEMENTATION STEPS:** 1. Define synthetic system `DEMO01` and object `ZCL_DEMO_UNKNOWN`. 2. Insert both only through the approved dev-only mechanism. 3. Keep seed idempotent; never load in production. 4. Record cleanup/disable procedure.
- **FIELDS / METHODS / ACTIONS:** Source `SystemCode=DEMO01`; object `ObjectType=CLAS`, `ObjectName=ZCL_DEMO_UNKNOWN`, `PackageName=ZDEMO`, stable source key `DAY5-OBJ-001`.
- **EXPECTED BEHAVIOR:** One object is returned by the live inventory service; missing evidence is explicit, not fabricated.
- **TEST CASES:** Seed twice → one business-key row; list query returns one object; unapproved environment → seed refuses.
- **ACCEPTANCE CRITERIA:** Demo record is synthetic, repeatable, isolated to dev, and attributable; no real-system scan is implied.
- **DEPENDENCIES:** Supported dev seed path and M3 binding.
- **COMMIT / DELIVERABLE:** CSV fixture, seed evidence and transport; record blocker if no approved seed path.

M2-D05-T01 — Seed the demo policy and persist the first recommendation

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** ADT class `ZCL_CC_RULESET_SEEDER`, `ZCL_CC_RECOMMENDATION_SERVICE`, existing `ZCC_RECOMM`/`ZCC_RMATCH` and CDS `ZI_CC_RECOMMENDATION`/`ZI_CC_RULE_MATCH`; local classifier/service tests.
- **PURPOSE:** Complete the first recommendation slice safely while evidence ingestion is not yet available.
- **PRECONDITIONS:** Day-3 recommendation/rule tables; M2 classifier skeleton; M1 Day-5 object fixture; Day-1-approved non-production rule-data seed path.

- **IMPLEMENTATION STEPS:** 1. Implement idempotent `SEED_DEMO_V1` to insert `CC_DEMO` version `1.0`, set it ACTIVE with server CreatedAt/CreatedBy/PublishedAt/PublishedBy, and insert all five v1 rules (`D00_MISSING`, `D10_READINESS_FAIL`, `D20_ALT_CONFIRMED`, `D30_ZERO_USAGE`, `D40_READINESS_PASS`) exactly as specified in §4; Day 5 classifier may execute only the missing-evidence gate. 2. Implement the minimum M2-owned recalculation path for an object with no evidence. 3. Persist one recommendation and `D00_MISSING` trace row with RuleSet UUID/version. 4. Expose read-only CDS association. 5. Never auto-seed production or infer a positive recommendation.
- **FIELDS / METHODS / ACTIONS:** `SEED_DEMO_V1`, `RECALCULATE_OBJECTS`; outcome `MANUAL_REVIEW`, `EvidenceStatus=INCOMPLETE`, missing `CLOUD_READINESS`/usage.
- **EXPECTED BEHAVIOR:** Day-5 object displays a transparent manual-review recommendation tied to `CC_DEMO/1.0` and selected `D00_MISSING` trace.
- **TEST CASES:** Seed twice → one RuleSet/version and exactly five v1 rule rows; no evidence → MANUAL_REVIEW via D00_MISSING; repeat evaluation at same AsOfUtc → same result.
- **ACCEPTANCE CRITERIA:** Snapshot/match read is real persisted data; seeding is restricted to verified non-prod; M4 can display exact result; absent seed capability is reported as a blocker.
- **DEPENDENCIES:** M1 object key and approved dev seed path; M2 tables/interfaces/classifier.
- **COMMIT / DELIVERABLE:** M2 transport, seed/recalc test output and field mapping to M4.

M3-D05-T01 — Add safe decision read shape to service metadata

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** `ZC_CC_DECISION` projection and update `ZUI_CC_COCKPIT` service definition only if M2/M3 entities are active; otherwise `docs/clean-core/contracts/day5-service-gap.md`.
- **PURPOSE:** Let the first object detail page distinguish “no decision” from “no recommendation” without exposing incomplete actions.
- **PRECONDITIONS:** M3 decision root; M2 recommendation CDS; M3 service binding.
- **IMPLEMENTATION STEPS:** 1. Project DecisionUUID/status/outcome/owner/read-only. 2. Expose only safe read fields. 3. Add association to object/recommendation; do not expose approve/override. 4. Capture missing dependencies if either CDS is not ready.
- **FIELDS / METHODS / ACTIONS:** `DecisionStatus`, `DecisionOutcome`, `OwnerUser`, `RecommendationUUID`; no actions.
- **EXPECTED BEHAVIOR:** No decision is an empty association; it is not rendered as an approved decision.
- **TEST CASES:** Object with no decision → empty status; object with DRAFT fixture → `DRAFT`; unavailable recommendation → request fails visibly, not silently.
- **ACCEPTANCE CRITERIA:** Service metadata is consistent or a dated blocker names its owner; no premature mutation endpoint.
- **DEPENDENCIES:** M2 recommendation projection and M3 root/service.
- **COMMIT / DELIVERABLE:** Transport or gap record and sanitized metadata snapshot.

M4-D05-T01 — Run and capture the first E2E slice

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** Existing List Report route and `webapp/manifest.json`; generated Object Page annotations from CDS; `docs/clean-core/test-evidence/day05-e2e.md`.
- **PURPOSE:** Demonstrate synthetic object → OData inventory → Fiori list → object details → safe recommendation.
- **PRECONDITIONS:** M1 fixture; M3 binding; M2 recommendation read result; supported UI profile.
- **IMPLEMENTATION STEPS:** 1. Navigate from list to object page. 2. Show object identity and recommendation outcome/rationale/missing evidence. 3. Capture network/service entity names and screenshots. 4. Label any not-yet-built evidence/workflow sections “not available in MVP slice.”
- **FIELDS / METHODS / ACTIONS:** ObjectName/ObjectType/SystemCode; Recommendation OutcomeCode/EvidenceStatus/Rationale; no enabled workflow action.
- **EXPECTED BEHAVIOR:** `ZCL_DEMO_UNKNOWN` shows `MANUAL_REVIEW` and missing evidence; no fake recommendation or approval.
- **TEST CASES:** Click row; reload detail route; recommendation present; no decision/action shown as completed.
- **ACCEPTANCE CRITERIA:** One repeatable E2E demonstration or explicit seed/service blocker; evidence contains build/profile/version and sanitized screenshot.
- **DEPENDENCIES:** M1→M3 live object; M2 recommendation; all Day-1 gates.
- **COMMIT / DELIVERABLE:** `day05-e2e.md`, screenshot references and any UI config commit.

DAY 6 — Complete persistence, provenance, CSV template, and synthetic dataset

M1-D06-T01 — Create evidence, import, usage, finding, dependency persistence and templates

- **OWNER:** M1 — ABAP Data & Import.

- **OBJECT TO CREATE OR MODIFY:** Tables `ZCC_USAGE`, `ZCC_EVIDENCE`, `ZCC_FINDING`, `ZCC_DEPEND`, `ZCC_BATCH`, `ZCC_IMPROW`; CDS `ZI_CC_USAGE_FACT`, `ZI_CC_EVIDENCE`, `ZI_CC_FINDING`, `ZI_CC_DEPENDENCY`, `ZI_CC_IMPORT_BATCH`, `ZI_CC_IMPORT_ROW`; local `docs/clean-core/import-template/*.csv` and `docs/clean-core/demo/*.csv`.
- **PURPOSE:** Complete source facts, row status/provenance, and a reproducible CSV contract.
- **PRECONDITIONS:** M1 schema frozen; UUID/timestamp DDL profile known.
- **IMPLEMENTATION STEPS:** 1. Create fields exactly as §4 and associations. 2. Add validations for count ≥0, period start ≤ end, required source keys, value-code typing. 3. Create six template headers and the five-object synthetic dataset in §4, including usage/evidence/findings/dependency rows. 4. Use fixed synthetic timestamps and never live customer data. 5. Activate tables before CDS.
- **FIELDS / METHODS / ACTIONS:** All §4 fields; relationships to source/object/batch; `QualityCode` computed/validated from value presence.
- **EXPECTED BEHAVIOR:** Facts resolve to an existing object and retain source name/reference, source record key, batch and capture time.
- **TEST CASES:** Usage count `-1` rejected; period end before start rejected; blank evidence value yields incomplete quality; unknown object reference rejected.
- **ACCEPTANCE CRITERIA:** All table/views activate; each CSV header matches `data-v1.md`; data set contains the five objects and CSV rows enumerated in §4 (5 usage facts, 9 evidence rows, one finding, one dependency).
- **DEPENDENCIES:** M1 Day-3 object/system entities; contract and M2 input codes.
- **COMMIT / DELIVERABLE:** ADT transport/task ID; exact CSV templates/data under docs; update contract registry.

M2-D06-T01 — Persist recommendation snapshots, rule matches, and editable rule BO

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** Complete CDS `ZI_CC_RULESET`, `ZI_CC_RULE`, `ZI_CC_RECOMMENDATION`, `ZI_CC_RULE_MATCH`; BDEF for root `ZI_CC_RULESET`; behavior pool `ZBP_I_CC_RULESET`.
- **PURPOSE:** Complete auditable recommendation-match projections and allow draft rule configuration with immutable published versions.
- **PRECONDITIONS:** M2 Day-3 tables/interfaces; M1 RecommendationUUID/object keys; RAP behavior probe passed.
- **IMPLEMENTATION STEPS:** 1. Complete snapshot/match CDS fields and associations exactly per §4. 2. Define RuleSet composition to rules. 3. Permit create/update/delete only while parent is DRAFT. 4. Add `publish` action stub; publication validation arrives Day 9. 5. Keep published versions and recommendation snapshots immutable.
- **FIELDS / METHODS / ACTIONS:** RuleSet `RuleSetCode/Version/Status`; Rule `RuleID/Priority/ConditionCode/InputCode/Operator/CompareValue/OutcomeCode`; recommendation/match snapshot fields.
- **EXPECTED BEHAVIOR:** Draft rules are editable; active versions and existing recommendation snapshots are read-only and retain old version identifiers.
- **TEST CASES:** Draft rule create/update succeeds; attempt to modify rule under ACTIVE version fails; recommendation+match read keys resolve.
- **ACCEPTANCE CRITERIA:** Tables/CDS/BDEF activate; no dynamic expression execution; M1/M3 can compile against IDs and values.
- **DEPENDENCIES:** M2 Day-3 public types; M1 IDs; RAP capabilities.
- **COMMIT / DELIVERABLE:** ADT transport/task ID and RuleSet lifecycle test evidence.

M3-D06-T01 — Add audit persistence and decision projection metadata

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** Table `ZCC_AUDIT`; CDS `ZI_CC_DECISION_AUDIT`, `ZC_CC_DECISION`, `ZC_CC_DECISION_AUDIT`; metadata extension `ZC_CC_DECISION_MDE`.
- **PURPOSE:** Make decision/audit data queryable for the later workflow page without yet granting transitions.
- **PRECONDITIONS:** M3 decision root; M2 recommendation entity active; §4 audit schema frozen.
- **IMPLEMENTATION STEPS:** 1. Create audit table with event fields and decision association. 2. Expose read-only audit projection. 3. Project decision/recommendation links and display labels. 4. Ensure no ordinary update/delete operation is offered for audit. 5. Add access-control annotations required by the selected profile.
- **FIELDS / METHODS / ACTIONS:** AuditUUID, DecisionUUID, EventCode, actor/time, before/after values, Reason; projection fields status/outcome/owner.
- **EXPECTED BEHAVIOR:** An audit query can read events in time order; no exposed write/delete path exists.
- **TEST CASES:** Read empty audit; read synthetic `CREATED` row; attempt audit update through exposed service is denied/not generated.
- **ACCEPTANCE CRITERIA:** Views activate and service field names match contract; ordinary OData metadata contains no audit mutation operation.
- **DEPENDENCIES:** M2 recommendation CDS; M3 decision table; chosen DCL profile.
- **COMMIT / DELIVERABLE:** ADT transport/task ID and service metadata evidence.

M4-D06-T01 — Add object-page evidence and recommendation sections

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** Generated FE app `webapp/manifest.json`; CDS metadata extensions consumed by the app; add generated Object Page sections via annotation configuration. If preview-only, update `fiori-preview-runbook.md`.
- **PURPOSE:** Define the actual detail-page layout and data bindings for facts becoming available on Day 6.
- **PRECONDITIONS:** M1 fact CDS, M2 recommendation/match CDS, M3 decision projection activated or documented as pending.
- **IMPLEMENTATION STEPS:** 1. Bind object association sections for evidence, usage, findings, dependencies. 2. Add recommendation header and rationale section. 3. Add value-help/display labels from annotations. 4. Show provenance fields in a collapsible section, not as an unlabeled JSON blob.
- **FIELDS / METHODS / ACTIONS:** EvidenceCode/ValueText/ValueNumber/QualityCode/CapturedAt/SourceName; usage period/count; finding code/severity/summary; dependency target; recommendation/rationale.
- **EXPECTED BEHAVIOR:** Empty association renders “No imported … yet”; missing evidence is not rendered as a zero measurement.
- **TEST CASES:** One object with facts shows rows; one object without facts shows empty state; long explanation remains readable.
- **ACCEPTANCE CRITERIA:** Bindings use service properties from metadata; no static evidence values embedded in UI.
- **DEPENDENCIES:** M1 fact views; M2 recommendation view; M3 decision projection as available.
- **COMMIT / DELIVERABLE:** UI commit or annotated preview evidence and field-to-section map.

DAY 7 — Implement initial parser, row validation, and import error reporting

M1-D07-T01 — Implement CSV parse, stage, validate, and batch process

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** Classes `ZCL_CC_CSV_PARSER`, `ZCL_CC_IMPORT_VALIDATOR`, `ZCL_CC_IMPORT_SERVICE`; interface `ZIF_CC_IMPORTER`; BDEF/root behavior pool for `ZI_CC_IMPORT_BATCH` / `ZBP_I_CC_IMPORT_BATCH`; class-pool local ABAP Unit tests.
- **PURPOSE:** Turn a submitted batch into persisted rows plus deterministic row-level outcomes.
- **PRECONDITIONS:** Day-6 import tables/CSV headers; Day-1 RAP parameter/upload profile; M2 classifier interface exists.
- **IMPLEMENTATION STEPS:** 1. Parse UTF-8 comma/double-quote CSV and LF/CRLF; reject unmatched quotes/wrong columns/row >512. 2. Compare exact versioned header per ImportKind. 3. Create batch/row results. 4. Validate required cells, enums, numeric/date ranges, and references. 5. Persist accepted rows only; attach error code/text and row number to rejected rows. 6. In staging fallback, allow client create only for BatchUUID, RowNumber, EntityKind and RawRow; server owns RowStatus/ErrorCode/ErrorText/ResolvedUUID. 7. Expose `processBatch`; add direct `CsvText` action/file UI only if Day-1 proved it.
- **FIELDS / METHODS / ACTIONS:** `PARSE_CSV`, `VALIDATE_ROW`, `PROCESS_BATCH`; statuses from §4; `ImportKind`, `RawRow`, `ErrorCode`, counts.
- **EXPECTED BEHAVIOR:** Valid row persists; malformed/missing-field/invalid-value/unknown-reference rows are rejected with a stable error; batch completes with counts.
- **TEST CASES:** One valid object row → `PERSISTED`; missing `object_name` → `MISSING_REQUIRED_FIELD`; negative count → `INVALID_EXECUTION_COUNT`; unmatched quote → `MALFORMED_CSV`; unknown system → `UNKNOWN_SOURCE_SYSTEM`; unknown object → `UNKNOWN_OBJECT`.
- **ACCEPTANCE CRITERIA:** Tests pass; invalid row never partially persists; every input row has one stored result; importer cap/header/errors are documented.
- **DEPENDENCIES:** M1 Day-6 entities/templates; service/action mode confirmed Day 1; M3 service exposure pattern.
- **COMMIT / DELIVERABLE:** Parser/importer transport, unit test output, sanitized sample batch/result.

M2-D07-T01 — Add evidence completeness gate

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** Class `ZCL_CC_EVIDENCE_GATE` and its local ABAP Unit class; update `ZIF_CC_CLASSIFIER` only through contract change if necessary.
- **PURPOSE:** Distinguish absent, incomplete, unknown, and stale required evidence before any positive rule can match.
- **PRECONDITIONS:** M1 Evidence/Usage CDS active; M2 input structure frozen; fixed-clock test seam available.
- **IMPLEMENTATION STEPS:** 1. Implement `CHECK_COMPLETENESS` over typed input. 2. Require latest CLOUD_READINESS and a valid 90-day usage period per proposed policy. 3. Classify unknown/future timestamps safely. 4. Return status and sorted missing/stale code list. 5. Keep the threshold clearly marked project-defined.
- **FIELDS / METHODS / ACTIONS:** `CHECK_COMPLETENESS`; outputs `EvidenceStatus`, `MissingEvidenceCodes`.
- **EXPECTED BEHAVIOR:** Missing/unknown/stale required input blocks recommendation and lists why; optional unknown evidence does not falsely pass a rule.

- **TEST CASES:** Missing readiness → INCOMPLETE; 91-day-old capture → STALE; exactly 90-day-old → COMPLETE; future timestamp → UNKNOWN.
- **ACCEPTANCE CRITERIA:** Boundary tests pass under fixed clock; no positive outcome bypasses the gate.
- **DEPENDENCIES:** M1 evidence field names and M2 rule policy contract.
- **COMMIT / DELIVERABLE:** Class/test transport and boundary-case report.

M3-D07-T01 — Add decision create validation and ETag skeleton

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** BDEF for `ZI_CC_DECISION`, behavior pool `ZBP_I_CC_DECISION`, local workflow ABAP Unit test class.
- **PURPOSE:** Permit a valid draft to be created and protect it against stale updates before workflow actions arrive.
- **PRECONDITIONS:** Decision root/audit schema; RAP managed behavior and ETag/lock support verified.
- **IMPLEMENTATION STEPS:** 1. Define root behavior with create/read and protected update fields. 2. Add create validation for existing object/current recommendation. 3. Configure ETag or supported lock field. 4. Keep transition actions disabled until Day 12. 5. Add test doubles for current principal and recommendation lookup.
- **FIELDS / METHODS / ACTIONS:** `validateCreate`, ETag `LastChangedAt`; initial status forced to `DRAFT`; client cannot set `APPROVED`/actor/timestamps.
- **EXPECTED BEHAVIOR:** Create produces a draft from a current recommendation; forged state/audit fields are ignored/rejected.
- **TEST CASES:** Valid object+recommendation creates DRAFT; missing link fails; caller-supplied APPROVED fails.
- **ACCEPTANCE CRITERIA:** Behavior activates; no unrestricted update/delete; tests assert server-owned fields.
- **DEPENDENCIES:** M1 object keys; M2 recommendation CDS/service; Day-1 RAP identity/ETag.
- **COMMIT / DELIVERABLE:** BDEF/pool transport and test output.

M4-D07-T01 — Connect import batch list/error feedback under supported mode

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** Fiori route `ImportBatches` in `ui/cc-migration-cockpit/webapp/manifest.json`; for profile A only, generator-registered `ui/cc-migration-cockpit/webapp/ext/controller/ImportUpload.controller.js` and `ui/cc-migration-cockpit/webapp/ext/fragment/ImportUpload.fragment.xml`; for profile B, use standard ImportBatch/ImportRow OData create/list operations and document no file picker.
- **PURPOSE:** Show batch status and row errors without claiming an unavailable file-upload control.
- **PRECONDITIONS:** M1 Batch/Row projection and `processBatch` metadata; Day-1 import mode.
- **IMPLEMENTATION STEPS:** 1. Add batch list with kind/status/counts/time. 2. Add row-error table with row number/code/text. 3. If direct upload is supported, read file as UTF-8 and call the agreed action; otherwise stage rows through supported entity CRUD. 4. Disable processing while request is pending.
- **FIELDS / METHODS / ACTIONS:** BatchUUID, ImportKind, BatchStatus, counts; RowNumber, RowStatus, ErrorCode, ErrorText; `processBatch`.
- **EXPECTED BEHAVIOR:** User can see accepted/rejected counts and exact rejected row; unsupported upload is never shown as a working button.
- **TEST CASES:** Valid batch completes; one invalid row shows `MISSING_REQUIRED_FIELD`; empty list and service error show separate states.
- **ACCEPTANCE CRITERIA:** UI invokes only service metadata actions; profile and limitations visible in runbook.
- **DEPENDENCIES:** M1 batch service and M3 binding; Fiori upload extension only if proven supported.
- **COMMIT / DELIVERABLE:** UI commit or preview evidence with import-mode verification.

DAY 8 — Make imports idempotent and connect persisted evidence to classifier input

M1-D08-T01 — Implement duplicate detection, upsert, and provenance

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** `ZCL_CC_IMPORT_SERVICE`, `ZCL_CC_IMPORT_VALIDATOR`, ImportBatch BDEF implementation, importer local test class.
- **PURPOSE:** Make retries safe and retain traceability from each accepted fact back to its source row/batch.
- **PRECONDITIONS:** Day-7 single-row import works; import key/serialization decision recorded.
- **IMPLEMENTATION STEPS:** 1. Normalize key fields. 2. Detect same key + same payload in-batch/across batches and mark `SKIPPED_DUPLICATE`. 3. Same source key + changed payload updates the existing UUID and marks `UPDATED`. 4. Same natural key with incompatible payload and no stable source key returns `DUPLICATE_CONFLICT`. 5. Preserve latest ImportBatchUUID, SourceReference, CapturedAt, row number. 6. Serialize import if unique index/lock is unavailable.

- **FIELDS / METHODS / ACTIONS:** `UPSERT_SOURCE_ROW`; entity UUID, SourceRecordKey, ImportBatchUUID, ImportedAt, row status/counts.
- **EXPECTED BEHAVIOR:** Replaying identical CSV changes no inventory/fact rows; corrected prior rejection can be imported in a new batch.
- **TEST CASES:** Same valid file twice → second `SKIPPED_DUPLICATE`; changed value same key → `UPDATED`, same UUID; duplicate conflicting row within one file → one accepted/one rejected; corrected bad row → accepted.
- **ACCEPTANCE CRITERIA:** No duplicate persisted business rows; counters reconcile; every persisted row has provenance; concurrency limitation, if any, is explicit.
- **DEPENDENCIES:** M1 Day-7 importer; Day-1 locking/index support.
- **COMMIT / DELIVERABLE:** Updated transport/test report and before/after row counts.

M2-D08-T01 — Complete freshness and missing-evidence behavior

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** `ZCL_CC_EVIDENCE_GATE`, `ZCL_CC_CLASSIFIER`, local ABAP Unit class; publish `ZIF_CC_RECOMMENDATION_SERVICE` signature for M1.
- **PURPOSE:** Provide one stable post-import API that can calculate and persist a recommendation without M1 writing M2 tables.
- **PRECONDITIONS:** M1 read CDS and idempotent write behavior; recommendation table/view; M2 completeness gate.
- **IMPLEMENTATION STEPS:** 1. Define `RECALCULATE_OBJECTS(object UUID table, AsOfUtc)` and result summary. 2. Load evidence through released CDS/repository, call pure classifier, persist snapshot and match rows through the M2-owned RAP-safe write path. 3. Return IDs/outcomes to caller. 4. Ensure active RuleSet is selected unambiguously.
- **FIELDS / METHODS / ACTIONS:** Interface method `RECALCULATE_OBJECTS`; output RecommendationUUID/Outcome/EvidenceStatus; rule version and trace included.
- **EXPECTED BEHAVIOR:** One call yields a versioned snapshot; old snapshots remain readable; an import caller need not know M2 tables.
- **TEST CASES:** Fresh pass; stale readiness; missing usage; no active rule set → explicit error; same input/as-of → same outcome.
- **ACCEPTANCE CRITERIA:** Interface compiles for M1; no separate commit from behavior pool; M2 owns all recommendation persistence.
- **DEPENDENCIES:** M1 read CDS; M2 persistence and rules; agreed RAP transaction approach.
- **COMMIT / DELIVERABLE:** Interface/class transport and M1 integration handoff.

M3-D08-T01 — Enforce open-decision and snapshot-link invariants

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** `ZBP_I_CC_DECISION` validation/determination implementation and workflow local tests.
- **PURPOSE:** Prevent draft creation against missing/stale object references and avoid two simultaneously open decisions for one object.
- **PRECONDITIONS:** M2 recommendation service; M1 object read; Day-7 create behavior.
- **IMPLEMENTATION STEPS:** 1. Validate ObjectUUID and RecommendationUUID association. 2. Check no DRAFT/SUBMITTED decision exists for that object. 3. Snapshot `DecisionOutcome` from the linked recommendation. 4. Reject client alteration of snapshot UUID/outcome on update except explicit override action later.
- **FIELDS / METHODS / ACTIONS:** `validateCreate`, `validateUpdate`; status, ObjectUUID, RecommendationUUID, DecisionOutcome.
- **EXPECTED BEHAVIOR:** One open decision per object; creating a second open record returns stable `OPEN_DECISION_EXISTS`.
- **TEST CASES:** First DRAFT succeeds; second simultaneous DRAFT/SUBMITTED fails; an APPROVED historical record permits a new DRAFT because only DRAFT/SUBMITTED are open; missing recommendation fails.
- **ACCEPTANCE CRITERIA:** Validations are server-side and covered; historical decision behavior is written in the workflow contract.
- **DEPENDENCIES:** M1/M2 read services; M3 Day-7 BDEF.
- **COMMIT / DELIVERABLE:** Transport, state-invariant test output and contract note.

M4-D08-T01 — Render row-level import diagnostics

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** Import batch Object Page/list annotations and generated app routing in `ui/cc-migration-cockpit/webapp/manifest.json` (or supported preview annotations).
- **PURPOSE:** Make duplicates, updates, incomplete evidence warnings, and corrected re-import outcomes explainable to an importer.
- **PRECONDITIONS:** M1 `RowStatus` and error fields finalized; batch/row projections exposed.
- **IMPLEMENTATION STEPS:** 1. Add status filters. 2. Display row number, source key, status, error code/text, resolved UUID. 3. Highlight `ACCEPTED_WARNING` separately from `REJECTED`. 4. Link batch back to its file name and counts.

- **FIELDS / METHODS / ACTIONS:** `RowStatus` values in §4; no client-side status reclassification.
- **EXPECTED BEHAVIOR:** Incomplete evidence can be accepted with a warning and later shown as missing to M2; duplicate is not shown as a rejection.
- **TEST CASES:** Render one persisted, warning, rejected, duplicate, and updated row; click filter and verify returned rows.
- **ACCEPTANCE CRITERIA:** UI labels map one-to-one to API statuses; no error message is lost/truncated silently.
- **DEPENDENCIES:** M1 import row service; M3 OData binding.
- **COMMIT / DELIVERABLE:** UI commit/preview screenshot and status mapping table.

DAY 9 — Implement deterministic rules, explanations, and conflict handling

M1-D09-T01 — Harden reference validation and import failure atomicity

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** `ZCL_CC_IMPORT_VALIDATOR`, `ZCL_CC_IMPORT_SERVICE`; local importer test class; `docs/clean-core/import-template/README.md`.
- **PURPOSE:** Finish object/source/reference checks and make failure behavior predictable across all six CSV kinds.
- **PRECONDITIONS:** Idempotent importer; active entity CDS; M2 input-code list.
- **IMPLEMENTATION STEPS:** 1. Validate SOURCE_SYSTEM before object; object before usage/evidence/finding/dependency. 2. Reject nonexistent source/source object/target object with distinct codes. 3. Do not persist a dependency with an unresolved target. 4. Define per-row atomicity: rejected row writes only ImportRow, not partial facts. 5. Document import order and retry procedure.
- **FIELDS / METHODS / ACTIONS:** `RESOLVE_SOURCE`, `RESOLVE_OBJECT`, `VALIDATE_DEPENDENCY`; error codes `UNKNOWN_SOURCE_SYSTEM`, `UNKNOWN_OBJECT`, `UNKNOWN_TARGET_OBJECT`, `MALFORMED_CSV`.
- **EXPECTED BEHAVIOR:** A malformed dependency row cannot create a dangling link; corrected re-import succeeds.
- **TEST CASES:** Missing system; missing source object; missing dependency target; valid corrected target; malformed row with valid neighboring row (neighbor still persists).
- **ACCEPTANCE CRITERIA:** All reference failures are distinguishable; batch is row-atomic and count-balanced.
- **DEPENDENCIES:** M1 tables/importer; M2 evidence code contract.
- **COMMIT / DELIVERABLE:** Transport, import README, test evidence.

M2-D09-T01 — Implement versioned evaluator, rules, trace, and formulas

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** `ZCL_CC_CLASSIFIER`, `ZCL_CC_RECOMMENDATION_SERVICE`, RuleSet behavior pool `ZBP_I_CC_RULESET`; classifier/recommendation ABAP Unit classes.
- **PURPOSE:** Produce a stable, inspectable outcome from the agreed rules and typed evidence.
- **PRECONDITIONS:** M1 fact CDS; Day-8 completeness gate; RuleSet/Rule tables; frozen demo policy (or PDF-reconciled replacement).
- **IMPLEMENTATION STEPS:** 1. Implement allowlisted operators/conditions only. 2. Select active version and sort priority ascending then RuleID. 3. Gate missing/stale/unknown required evidence. 4. Resolve same-priority conflict to manual review; no match also manual review. 5. Persist selected/all matched rule IDs and explanation, policy version, risk score/band and missing codes. 6. Validate/publish only supported rule codes and outcomes.
- **FIELDS / METHODS / ACTIONS:** `EVALUATE_OBJECT`, `RECALCULATE_OBJECTS`, `publish`; rule inputs/outputs in §4–5; no dynamic `EVAL`/SQL.
- **EXPECTED BEHAVIOR:** Same input, rules and AsOfUtc always yield same result; missing evidence cannot produce a positive outcome.
- **TEST CASES:** `PASS+25+NO→KEEP`; `FAIL+complete→REMEDIATE`; candidate YES→REPLACE unless higher-priority rule; zero usage→RETIRE; missing/stale→MANUAL_REVIEW; equal-priority KEEP vs RETIRE→MANUAL_REVIEW; unknown/no-match→MANUAL_REVIEW; score boundary 39/40/69/70; repeat evaluation identical.
- **ACCEPTANCE CRITERIA:** All unit tests pass; trace contains rule ID/version/explanation; formulas match §4; demo-policy disclaimer in rationale/docs.
- **DEPENDENCIES:** M1 fact entities and import service; M2 contract/active rules.
- **COMMIT / DELIVERABLE:** ADT transport, test output, rule-set v1 record and rule decision table.

M3-D09-T01 — Prepare workflow against immutable recommendation snapshots

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** `ZI_CC_DECISION` association/projection and `ZBP_I_CC_DECISION` link checks.
- **PURPOSE:** Ensure a human decision points to the exact recommendation version that was reviewed.
- **PRECONDITIONS:** M2 persisted Recommendation/Match entities and published rule version.

- **IMPLEMENTATION STEPS:** 1. Add association to RecommendationUUID and immutable rule version display. 2. Store RecommendationUUID at decision creation. 3. Prevent recommendation refresh from silently overwriting a decision snapshot. 4. Define how a later recommendation requires a new draft per v1.
- **FIELDS / METHODS / ACTIONS:** RecommendationUUID, OutcomeCode, RuleSetVersion, EvaluatedAt; validation for a current snapshot.
- **EXPECTED BEHAVIOR:** A later evaluation creates a new recommendation but leaves existing decision/approval history unchanged.
- **TEST CASES:** Create decision against R1; create R2 after changed evidence; decision still references R1; attempt to alter linked recommendation rejected.
- **ACCEPTANCE CRITERIA:** Snapshot link is visible in CDS; behavior test proves recommendation immutability from decision path.
- **DEPENDENCIES:** M2 recommendation service; M3 decision root.
- **COMMIT / DELIVERABLE:** Transport and snapshot regression report.

M4-D09-T01 — Display matched rules, missing evidence, and rationale

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** Object Page annotations/section configuration in the generated app or CDS MDE consumed by FE; `ui/cc-migration-cockpit/webapp/manifest.json` only for generated routes/data sources.
- **PURPOSE:** Make recommendation decision logic auditable to the analyst.
- **PRECONDITIONS:** M2 recommendation and rule-match service metadata; M1 evidence associations.
- **IMPLEMENTATION STEPS:** 1. Display outcome, demo priority band/score label, rule-set version and evaluation time. 2. Add rationale and missing evidence text. 3. Add match table with rule ID/priority/selected/explanation. 4. Show a clear “demo triage score—not SAP risk rating” label while the proposed formula remains.
- **FIELDS / METHODS / ACTIONS:** OutcomeCode, EvidenceStatus, RiskScore, PriorityBand, RuleSetVersion, Rationale, MissingEvidenceCodes, RuleID/Priority/IsSelected/ExplanationText.
- **EXPECTED BEHAVIOR:** Missing-data and conflict fallbacks are understandable; a recommendation is not rendered as an approved decision.
- **TEST CASES:** One normal rule; missing evidence; two tied conflicts; empty match fallback; long rationale wrap.
- **ACCEPTANCE CRITERIA:** All displayed rule data comes from service rows, not UI constants; screen labels distinguish advice from workflow status.
- **DEPENDENCIES:** M2 recommendation service; M1 fact details; M3 service binding.
- **COMMIT / DELIVERABLE:** UI commit/preview screenshots and field-to-service trace.

DAY 10 — Integrate and verify import → recommendation

M1-D10-T01 — Trigger recommendation service after successful import

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** ImportBatch behavior implementation `ZBP_I_CC_IMPORT_BATCH`; `ZCL_CC_IMPORT_SERVICE`; integration test class.
- **PURPOSE:** Send the unique set of affected objects to M2 after accepted source facts are available.
- **PRECONDITIONS:** M2 `ZIF_CC_RECOMMENDATION_SERVICE` signature/implementation handed off; importer rows/provenance correct; transaction behavior agreed.
- **IMPLEMENTATION STEPS:** 1. Collect distinct ObjectUUIDs from accepted/updated CustomObject, UsageFact, Evidence, Finding or Dependency rows. 2. Call M2 interface once per batch with one fixed AsOfUtc. 3. Capture output counts/failed object IDs in batch summary. 4. Preserve row results if evaluation fails; set batch status/error accurately. 5. Use supported RAP LUW/EML pattern—no forbidden `COMMIT` in behavior pool.
- **FIELDS / METHODS / ACTIONS:** `PROCESS_BATCH`; `RecalculatedObjectCount` may be added only via contract change; batch error summary otherwise records failure.
- **EXPECTED BEHAVIOR:** Successful evidence import yields a persisted versioned recommendation per affected object; duplicate-only reimport does not create a different result unless explicit reevaluation is requested.
- **TEST CASES:** Two affected objects → two recalculations; duplicate batch → no extra facts; M2 service failure → batch reports warning/failure without lying about committed rows.
- **ACCEPTANCE CRITERIA:** Integration test passes in tenant; ownership remains M1 importer/M2 recommendation writer; transaction outcome is documented.
- **DEPENDENCIES:** M2 interface and transaction-safe persistence; M1 import action.
- **COMMIT / DELIVERABLE:** Transport, integration test output, handoff note.

M2-D10-T01 — Close the import-to-classifier integration contract

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** `ZCL_CC_RECOMMENDATION_SERVICE`, `ZIF_CC_RECOMMENDATION_SERVICE`, M2 integration ABAP Unit class.

- **PURPOSE:** Verify persisted importer facts become the exact expected recommendation and trace.
- **PRECONDITIONS:** M1 import batch action and five-object synthetic CSV set; active versioned rules; M2 class tests green.
- **IMPLEMENTATION STEPS:** 1. Read the latest committed rows for each affected ObjectUUID. 2. Map them into pure evaluator input. 3. Persist snapshot and match rows in supported LUW. 4. Return output summaries; do not modify import row status. 5. Record chosen clock/version.
- **FIELDS / METHODS / ACTIONS:** `RECALCULATE_OBJECTS`; RecommendationUUID, OutcomeCode, RuleSetVersion, match count.
- **EXPECTED BEHAVIOR:** Imported `PASS + usage 25 + candidate NO` produces KEEP; missing input produces MANUAL_REVIEW; every result has source rule version.
- **TEST CASES:** Import all five fixed-fixture objects; expect KEEP, REMEDIATE, REPLACE, RETIRE and MANUAL_REVIEW respectively; compare rationale, missing codes and match IDs.
- **ACCEPTANCE CRITERIA:** All five records match expected outputs; no classifier reads current time or bypasses missing-evidence gate.
- **DEPENDENCIES:** M1 object/evidence/usage service and M1 batch call; active M2 rules.
- **COMMIT / DELIVERABLE:** Transport, expected/actual table and integration test evidence.

M3-D10-T01 — Verify snapshot consumption before lifecycle actions

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** `ZBP_I_CC_DECISION` create validation and `docs/clean-core/contracts/day10-workflow-integration.md`.
- **PURPOSE:** Confirm a draft will bind to the recommendation M2 actually produced, not a locally reconstructed outcome.
- **PRECONDITIONS:** M2 import-triggered recommendations; M1 object data; decision create behavior.
- **IMPLEMENTATION STEPS:** 1. Create a DRAFT from returned RecommendationUUID. 2. Read back OutcomeCode/RuleSetVersion via association. 3. Add validation for missing/deleted recommendation. 4. Record create failure code for unsupported/stale snapshot.
- **FIELDS / METHODS / ACTIONS:** `RecommendationUUID`, `DecisionOutcome`, status `DRAFT`; no approval yet.
- **EXPECTED BEHAVIOR:** Decision begins with exactly the recommendation outcome and immutable snapshot ID.
- **TEST CASES:** Create from KEEP snapshot; nonexistent snapshot fails; recommendation re-evaluation leaves existing draft linked to original.
- **ACCEPTANCE CRITERIA:** RAP read result matches classifier output; no independent M3 rule logic.
- **DEPENDENCIES:** M2 recommendation IDs and M3 root.
- **COMMIT / DELIVERABLE:** Transport and handoff evidence for Day 11 lifecycle work.

M4-D10-T01 — Execute import-to-recommendation end-to-end checks

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** `docs/clean-core/test-evidence/day10-import-classification.md`; generated ImportBatch/Object Page routes/annotations if needed.
- **PURPOSE:** Verify service, UI, validation, recommendation and error states together on the synthetic dataset.
- **PRECONDITIONS:** M1/M2 integrated processBatch; M3 binding; M4 import and detail pages.
- **IMPLEMENTATION STEPS:** 1. Submit valid fixture via the selected supported import route. 2. Open batch counts/errors. 3. Open each object detail and verify recommendation/matched rules. 4. Correct one invalid row and re-import. 5. Capture sanitized request/response screenshots and log defects.
- **FIELDS / METHODS / ACTIONS:** Batch counts/status; ObjectName; outcome/rationale/matched rule IDs; row errors.
- **EXPECTED BEHAVIOR:** The five fixture objects map to their expected outcomes; corrected row succeeds without duplicating its object/fact.
- **TEST CASES:** Valid source/object; missing required CSV field; invalid target reference; same file twice; corrected re-import; stale/missing evidence.
- **ACCEPTANCE CRITERIA:** Import/recommendation path is reproducible; each mismatch gets an owner/severity in blocker log; no UI-only fake result.
- **DEPENDENCIES:** M1 import service→M2 recommendation; M3 OData binding; M4 app.
- **COMMIT / DELIVERABLE:** E2E evidence, screenshots, defect list and UI/config commit.

DAY 11 — Complete decision state model and legal-transition rules

M1-D11-T01 — Stabilize inventory read projections for decision context

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** `ZC_CC_CUSTOM_OBJECT`, `ZC_CC_SOURCE_SYSTEM`, `ZC_CC_USAGE`, `ZC_CC_EVIDENCE`, `ZC_CC_FINDING`, `ZC_CC_DEPENDENCY` projections and their association definitions.
- **PURPOSE:** Provide the stable object/evidence context M3 and M4 need on a decision page.

- **PRECONDITIONS:** Day-6 entity CDS active; M2 output fields frozen; no schema change without change-log approval.
- **IMPLEMENTATION STEPS:** 1. Check all projection property names against v1. 2. Add useful associations from object to source/facts. 3. Add server-side search/filter annotations only on supported fields. 4. Keep write operations behind importer. 5. Run CDS activation and service metadata diff.
- **FIELDS / METHODS / ACTIONS:** Object identity, source/component/owner; evidence quality/capture; usage period/count; findings/dependencies.
- **EXPECTED BEHAVIOR:** Detail query loads context without exposing raw persistence table names or allowing direct edits.
- **TEST CASES:** Filter by system/type/component; object with no facts returns empty associations; object with facts resolves correctly.
- **ACCEPTANCE CRITERIA:** Metadata diff has no unapproved changes; association links return correct fixture rows.
- **DEPENDENCIES:** M1 persistence; M2/M3 agreed consumer requirements.
- **COMMIT / DELIVERABLE:** ADT transport, metadata diff and filter query evidence.

M2-D11-T01 — Enforce rule version lifecycle and latest recommendation semantics

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** `ZBP_I_CC_RULESET`, `ZCL_CC_RECOMMENDATION_SERVICE`, RuleSet/classifier local test classes.
- **PURPOSE:** Make rule versions immutable after activation and prevent “latest recommendation” from erasing decision history.
- **PRECONDITIONS:** Day-9 rules active; recommendation snapshot persistence; M3 snapshot-link contract.
- **IMPLEMENTATION STEPS:** 1. Validate unique RuleSetCode+Version and exactly one active version per code. 2. Validate all rule IDs/operators/priorities/outcomes at publish. 3. Reject edits to ACTIVE/RETIRED versions. 4. Define latest recommendation as max EvaluatedAt then stable UUID tie-break, while decision keeps its own RecommendationUUID. 5. Return no active version as a typed error.
- **FIELDS / METHODS / ACTIONS:** `publish`, `Status`, `EffectiveFrom/To`, `RuleSetVersion`, `EvaluatedAt`.
- **EXPECTED BEHAVIOR:** Editing a published rule requires a new DRAFT version; old recommendation traces continue to show old values.
- **TEST CASES:** Duplicate version rejected; publish draft succeeds; edit active fails; recalc after v2 creates a new snapshot while v1 remains readable.
- **ACCEPTANCE CRITERIA:** Immutability/latest ordering deterministic; version lifecycle is unit tested and documented.
- **DEPENDENCIES:** M2 rule tables/service; M3 recommendation reference.
- **COMMIT / DELIVERABLE:** Transport, version tests and published v1 demo fixture.

M3-D11-T01 — Implement the legal transition core in RAP

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** BDEF for `ZI_CC_DECISION`, pool `ZBP_I_CC_DECISION`, action parameter entities `ZA_CC_SUBMIT` and `ZA_CC_APPROVE`, workflow contract.
- **PURPOSE:** Implement the minimum server-side lifecycle before Day-12 action hardening and UI wiring.
- **PRECONDITIONS:** Decision root, snapshot link, Day-1 RAP/authorization/ETag capabilities.
- **IMPLEMENTATION STEPS:** 1. Define `submitDecision` and `approveDecision` with v1 parameter entities. 2. Implement `DRAFT→SUBMITTED` (require linked recommendation and assigned owner) and `SUBMITTED→APPROVED` state changes with current principal/time from supported platform APIs. 3. Reject every other transition. 4. Defer the `assignOwner` and `overrideRecommendation` BDEF actions and parameter entities to Day 12. 5. Leave full role/separation-of-duties checks and audit insertion for Days 12–13.
- **FIELDS / METHODS / ACTIONS:** `submitDecision`, `approveDecision`, transition validators; status and actor/timestamp fields are server-owned.
- **EXPECTED BEHAVIOR:** Legal basic transitions work only from their initial states; client cannot set state, actor or timestamp directly.
- **TEST CASES:** Assigned DRAFT→SUBMITTED succeeds; SUBMITTED→APPROVED succeeds with test principal; DRAFT→APPROVED and APPROVED→SUBMITTED fail; no owner fails submit.
- **ACCEPTANCE CRITERIA:** BDEF activates and transition tests pass; Day 12 adds override-reason/input/reviewer-separation checks before UI enables actions.
- **DEPENDENCIES:** M2 snapshot entity; M3 workflow contract and behavior pool; verified user/time API.
- **COMMIT / DELIVERABLE:** BDEF/pool transport and transition test output.

M4-D11-T01 — Bind decision status and state-dependent page controls

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** Object Page annotations and generated app action configuration in `webapp/manifest.json` where required by the generator.
- **PURPOSE:** Show current decision state correctly while the backend action implementation is completed.

- **PRECONDITIONS:** M3 decision metadata/action signatures; M2 recommendation snapshot; actual FE profile.
- **IMPLEMENTATION STEPS:** 1. Add decision status/outcome/owner/snapshot section. 2. Configure action placement from service metadata. 3. Hide/disable not-yet-enabled actions until backend capability/state is verified. 4. Show “No decision created” separately from DRAFT.
- **FIELDS / METHODS / ACTIONS:** DecisionStatus, DecisionOutcome, OwnerUser, RecommendationUUID, RuleSetVersion; action IDs from v1.
- **EXPECTED BEHAVIOR:** A DRAFT is visibly distinct from a recommendation; APPROVED is not shown until server confirms it.
- **TEST CASES:** No decision, DRAFT, SUBMITTED and APPROVED fixtures; action visibility follows backend metadata/state.
- **ACCEPTANCE CRITERIA:** No front-end-only transition; service response is source of truth.
- **DEPENDENCIES:** M3 projection/BDEF; M2 recommendation fields.
- **COMMIT / DELIVERABLE:** UI config commit or state screenshots and action mapping.

DAY 12 — Implement assignment, override, submit, and approval operations

M1-D12-T01 — Complete import-service read exposure and post-batch summary

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** ImportBatch/ImportRow projection views and M1 contribution to `ZUI_CC_COCKPIT`; importer service methods.
- **PURPOSE:** Make every import result consumable to both operations and M4 without bypassing validation.
- **PRECONDITIONS:** Idempotent importer; batch behavior; M3 service definition/binding active.
- **IMPLEMENTATION STEPS:** 1. Expose batch header/row results read-only after processing. 2. Ensure batch totals equal the sum of row statuses. 3. Expose supported `processBatch` action, not a direct table write. 4. Add batch status/failure semantics for parser and recommendation-service errors. 5. Review access controls with M3.
- **FIELDS / METHODS / ACTIONS:** Batch counts/status/time; row number/status/error/resolved UUID; `processBatch` result.
- **EXPECTED BEHAVIOR:** Batch report is complete, reconciled, and read-only after processing except a supported retry of an unprocessed batch.
- **TEST CASES:** Totals sum; duplicate count distinct from warning; failed parse reports error summary; unauthorized read follows selected policy.
- **ACCEPTANCE CRITERIA:** M4 can read all batch diagnostics; no direct unvalidated entity create is exposed as the production ingestion path.
- **DEPENDENCIES:** M1 importer; M3 service; M3 access-control profile.
- **COMMIT / DELIVERABLE:** Transport, service metadata diff and count reconciliation output.

M2-D12-T01 — Finalize decision-facing recommendation service contract

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** `ZIF_CC_RECOMMENDATION_SERVICE`, `ZCL_CC_RECOMMENDATION_SERVICE`, recommendation projection and M3 consumer example.
- **PURPOSE:** Provide M3 a stable snapshot read that includes advice, rationale and version but no workflow mutation.
- **PRECONDITIONS:** Rule lifecycle/test coverage; M3 decision actions now being implemented.
- **IMPLEMENTATION STEPS:** 1. Finalize recommendation-by-UUID and latest-by-object query signatures. 2. Include match rows, completeness, missing codes, score/band, ruleset version. 3. Define no-result/error behavior. 4. Ensure refresh creates a new snapshot, never updates a prior one. 5. Provide M3 a typed test fixture.
- **FIELDS / METHODS / ACTIONS:** `GET_LATEST_FOR_OBJECT`, `GET_BY_UUID`, match association; exact §4 fields.
- **EXPECTED BEHAVIOR:** M3 can initialize a decision from a single explicit snapshot without reproducing the classifier.
- **TEST CASES:** No rec → `NOT_FOUND`; two recs → deterministic latest; by-UUID returns historical version and matches.
- **ACCEPTANCE CRITERIA:** Interface activated, fixture consumed by M3, no private DB type leaks across package boundary.
- **DEPENDENCIES:** M1 object UUID; M2 recommendation persistence; M3 root/action implementation.
- **COMMIT / DELIVERABLE:** Interface transport and consumer handoff note.

M3-D12-T01 — Implement all four lifecycle actions and validations

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** `ZBP_I_CC_DECISION`; abstract action parameter entities `ZA_CC_ASSIGN_OWNER`, `ZA_CC_OVERRIDE`, `ZA_CC_SUBMIT`, `ZA_CC_APPROVE`; local ABAP Unit tests.
- **PURPOSE:** Implement the promised workflow state changes with mandatory owner/reason and server-side invariants.

- **PRECONDITIONS:** Day-11 BDEF stubs; current-user/role API and ETag/lock path verified; M2 snapshot service.
- **IMPLEMENTATION STEPS:** 1. `assignOwner`: DRAFT only, resolve user, set OwnerUser. 2. `overrideRecommendation`: DRAFT only, validate outcome enum and 1–255 nonblank reason, set override flag/outcome. 3. `submitDecision`: DRAFT only, require owner and override reason if applicable; set actor/time/status. 4. `approveDecision`: SUBMITTED only, reviewer role, reviewer ≠ submitter; set actor/time/status. 5. Return RAP reported/failed messages and preserve transaction rollback on validation failure.
- **FIELDS / METHODS / ACTIONS:** Actions and parameters in §5; server-managed actor/timestamp/state; ETag/lock check.
- **EXPECTED BEHAVIOR:** Valid transitions update one decision; any invalid input leaves state and outcome unchanged.
- **TEST CASES:** Valid owner/override/submit/approve; blank reason; invalid outcome; unassigned submit; unauthorized approve; submitter self-approval; repeated/illegal action; stale ETag.
- **ACCEPTANCE CRITERIA:** All action tests pass; no action trusts a client-supplied actor/status; unsupported role profile is visibly gated.
- **DEPENDENCIES:** M2 recommendation contract; verified Day-1 identity/ETag; M3 Day-11 BDEF.
- **COMMIT / DELIVERABLE:** BDEF/pool/action-param transport and ABAP Unit output.

M4-D12-T01 — Add standard Fiori workflow dialogs/actions

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** Decision Object Page annotations, service action metadata configuration, and generated `webapp/manifest.json`; extension only if standard FE action parameter dialog cannot render the frozen parameters.
- **PURPOSE:** Let an authorized user assign, override, submit and approve using server actions.
- **PRECONDITIONS:** M3 action metadata active; M3 role mapping test user; M4 object page bound to decision root.
- **IMPLEMENTATION STEPS:** 1. Place action buttons from RAP metadata. 2. Render Override parameter dialog with `NewOutcome` and mandatory `Reason`. 3. Require confirmation on submit/approve. 4. Display backend validation messages inline; refresh entity after action. 5. Never locally set status/outcome.
- **FIELDS / METHODS / ACTIONS:** `assignOwner`, `overrideRecommendation`, `submitDecision`, `approveDecision`; status/outcome/reason.
- **EXPECTED BEHAVIOR:** Override cannot be submitted with blank reason; successful action refreshes server state; denied/failed action leaves page unchanged and shows message.
- **TEST CASES:** Valid override; empty reason; valid submit; valid approval; forbidden approval; service timeout.
- **ACCEPTANCE CRITERIA:** Buttons invoke RAP actions; action parameters match CDS metadata; no custom UI sends actor/status.
- **DEPENDENCIES:** M3 actions, M2 recommendation association, selected FE generator/runtime.
- **COMMIT / DELIVERABLE:** UI commit/preview video or screenshots and action-to-service trace.

DAY 13 — Implement role authorization, reviewer separation, and audit events

M1-D13-T01 — Verify inventory/import read access under M3 authorization policy

- **OWNER:** M1 — ABAP Data & Import (data owner); M3 owns security rules.
- **OBJECT TO CREATE OR MODIFY:** M1 projection annotations (`@AccessControl.authorizationCheck`) and import service checks specified by M3; update `docs/clean-core/security-data-scope.md` with M3.
- **PURPOSE:** Ensure import rows and provenance are neither accidentally public nor inaccessible to the authorized cockpit roles.
- **PRECONDITIONS:** M3 provides role/DCL mapping from supported tenant identity; no custom auth mechanism invented by M1.
- **IMPLEMENTATION STEPS:** 1. Apply the agreed DCL to inventory/import projections. 2. Keep raw row text limited to authorized importer/admin roles if policy allows. 3. Execute read tests as analyst/reviewer/no-role principal. 4. Report missing platform role assignment; do not weaken DCL to make the UI pass.
- **FIELDS / METHODS / ACTIONS:** SourceSystem/Object/Batch/Row read scope; owner/principal values remain data, not login secrets.
- **EXPECTED BEHAVIOR:** Only the agreed roles see the data; no-role test follows the documented deny rule.
- **TEST CASES:** Analyst inventory read; reviewer batch read per policy; no-role read denied; unauthorized processBatch rejected.
- **ACCEPTANCE CRITERIA:** Security tests match the role matrix; any unavailable role mechanism is a release blocker.
- **DEPENDENCIES:** M3 role/DCL contract; M1 projections/import service.
- **COMMIT / DELIVERABLE:** Transport plus sanitized role-matrix test evidence.

M2-D13-T01 — Protect published rules and audit recommendation provenance

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** RuleSet BDEF `ZBP_I_CC_RULESET`, rule CDS access annotations, recommendation/match projections; M2 unit tests.
- **PURPOSE:** Prevent a non-admin/draft editor from changing active policy and preserve who/which version produced a result.
- **PRECONDITIONS:** M3 supported role names/authorization context; M2 version lifecycle.
- **IMPLEMENTATION STEPS:** 1. Require configured rule-admin role to create/publish a ruleset using supported RAP authorization. 2. Keep recommendation/match read permission aligned with analyst/reviewer. 3. Persist exact RuleSetUUID/version on every result. 4. Reject edit/delete on active rules and old recommendation matches.
- **FIELDS / METHODS / ACTIONS:** RuleSet status/version, recommendation version and match fields; no auth credentials in tables.
- **EXPECTED BEHAVIOR:** Unauthorized policy mutation is rejected; existing recommendation explanation remains tied to its original ruleset version.
- **TEST CASES:** Admin publishes draft; analyst publish denied; edit active denied; recalc after v2 retains old result trace.
- **ACCEPTANCE CRITERIA:** M3 confirms tenant-supported role mapping; tests prove access and immutability.
- **DEPENDENCIES:** M3 role profile; M2 ruleset behavior and recommendation service.
- **COMMIT / DELIVERABLE:** Transport and authorization/immutability test output.

M3-D13-T01 — Implement authorization handlers and append-only audit writes

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** DCL objects `ZP_CC_OBJECT`, `ZP_CC_DECISION`, `ZP_CC_AUDIT` where supported; `ZBP_I_CC_DECISION`; `ZI_CC_DECISION_AUDIT` access definition; local security/audit tests.
- **PURPOSE:** Enforce authenticated role/instance permissions, separation of duties, and immutable event history.
- **PRECONDITIONS:** Day-1 principal/role mechanism; M1/M2 entities; Day-12 action methods.
- **IMPLEMENTATION STEPS:** 1. Map platform roles (proposed aliases analyst/reviewer/admin) to actual tenant assignments. 2. Implement global/instance auth for owner edit, reviewer approve, admin rule operations as supported. 3. Compare the actual caller with SubmittedBy before approval. 4. Append audit event in the same successful RAP LUW for create/assign/override/submit/approve. 5. Expose audit read only; no update/delete action; actor/time obtained server-side.
- **FIELDS / METHODS / ACTIONS:** `get_global_authorizations`, `get_instance_authorizations` if supported; DecisionAudit fields in §4; actions from §5.
- **EXPECTED BEHAVIOR:** Unauthorized/self-approval attempts fail with no state change/audit success event; successful changes append exactly one event with before/after/reason/actor/time.
- **TEST CASES:** Analyst submits; different reviewer approves; submitter approves own request → denied; blank override → denied/no audit; valid override → reason recorded; audit update/delete denied.
- **ACCEPTANCE CRITERIA:** Tests run with supported role injection/tenant principals; no hard-coded user allowlist; history is append-only through service.
- **DEPENDENCIES:** M1 scope; M2 rule roles; verified auth/current-user APIs and ETag/lock.
- **COMMIT / DELIVERABLE:** DCL/BDEF transport, role mapping note, audit/security test output.

M4-D13-T01 — Exercise UI authorization and audit-read states

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** Object Page action visibility/configuration; `docs/clean-core/test-evidence/day13-security-ui.md`.
- **PURPOSE:** Verify UI reflects backend permissions without being treated as the enforcement layer.
- **PRECONDITIONS:** M3 role assignments and audit projection; test identities from SAP admin.
- **IMPLEMENTATION STEPS:** 1. Log in as analyst, reviewer, submitter and no-role test principal using platform-supported SSO. 2. Check action visibility and attempt direct service action for each forbidden case. 3. Render audit events newest/oldest in deterministic order. 4. Ensure UI hides secrets and shows useful 403/validation messages.
- **FIELDS / METHODS / ACTIONS:** Action buttons; DecisionStatus; Audit EventCode/ActorUser/OccurredAt/Reason.
- **EXPECTED BEHAVIOR:** Hidden buttons are not relied upon; backend rejects a direct unauthorized action; successful history is visible.
- **TEST CASES:** Analyst cannot approve if policy says reviewer-only; submitter cannot approve; reviewer can approve another user’s submitted decision; no-role denied; audit event visible.
- **ACCEPTANCE CRITERIA:** UI behavior and raw service behavior agree; no unhandled browser error; actor/time displayed from server data.
- **DEPENDENCIES:** M3 auth/audit; M4 workflow page.
- **COMMIT / DELIVERABLE:** Security UI report, screenshots and defect records.

DAY 14 — Prove legal/illegal transitions, authorization, audit integrity, and stale-update behavior

M1-D14-T01 — Run full import regression suite

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** Local test classes in `ZCL_CC_CSV_PARSER`, `ZCL_CC_IMPORT_VALIDATOR`, `ZCL_CC_IMPORT_SERVICE`; `docs/clean-core/test-evidence/import-matrix.md`.
- **PURPOSE:** Close every required import edge case before the UI/portfolio freeze.
- **PRECONDITIONS:** Import service and error codes frozen; M2 call path stable.
- **IMPLEMENTATION STEPS:** 1. Run parser and persistence tests in a clean test client. 2. Verify counter totals. 3. Run same-file replay and corrected re-import. 4. Verify row references/provenance. 5. Re-run after defects and record results.
- **FIELDS / METHODS / ACTIONS:** CSV kinds, RowStatus, error code, source key, batch counts, ResolvedUUID.
- **EXPECTED BEHAVIOR:** Each source line has one outcome and no invalid row leaves partial entity data.
- **TEST CASES:** Valid; missing field; invalid enum/value; duplicate; repeat import; invalid reference; malformed row; corrected re-import; incomplete evidence warning.
- **ACCEPTANCE CRITERIA:** All cases pass with exact codes/counts; cross-batch idempotency passes; no failing critical import test.
- **DEPENDENCIES:** M1 importer; M2 recommendation trigger handled in integrated test.
- **COMMIT / DELIVERABLE:** Test output and import matrix.

M2-D14-T01 — Run classifier/rule regression and conflict suite

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** Local ABAP Unit classes for `ZCL_CC_CLASSIFIER`, `ZCL_CC_EVIDENCE_GATE`, `ZCL_CC_RECOMMENDATION_SERVICE`; `docs/clean-core/test-evidence/rules-matrix.md`.
- **PURPOSE:** Prove every rule result, missing/stale path, tie policy, no-match behavior and score formula.
- **PRECONDITIONS:** M1 integration data; active demo rule set; injected fixed `AsOfUtc`.
- **IMPLEMENTATION STEPS:** 1. Build minimal typed fixtures per case. 2. Assert recommendation, version, selected/matched rules, rationale and missing list. 3. Assert risk score/band boundaries. 4. Re-evaluate identical input. 5. Run old/new rule-version snapshot regression.
- **FIELDS / METHODS / ACTIONS:** `EVALUATE_OBJECT`, `RECALCULATE_OBJECTS`, EvidenceStatus/Outcome/RiskScore/PriorityBand/RuleID.
- **EXPECTED BEHAVIOR:** Deterministic output and safe fallback for unknown/conflict/no rule.
- **TEST CASES:** Meets/fails rule; missing; stale; tied conflict; no match; unknown value; repeated deterministic result; score thresholds.
- **ACCEPTANCE CRITERIA:** All exact expected output assertions pass; no hidden date/system dependency; no unsupported factual rule.
- **DEPENDENCIES:** M1 data; M2 policy and tests.
- **COMMIT / DELIVERABLE:** Rules matrix and test-run evidence.

M3-D14-T01 — Run full workflow, SoD, audit, and concurrency tests

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** Local workflow/security ABAP Unit classes and tenant integration test notes `docs/clean-core/test-evidence/workflow-matrix.md`.
- **PURPOSE:** Verify every legal/illegal action and audit integrity under the real auth/lock path.
- **PRECONDITIONS:** M3 actions/DCL/audit; two principals with different roles; supported ETag/lock.
- **IMPLEMENTATION STEPS:** 1. Execute valid DRAFT→SUBMITTED→APPROVED. 2. Execute every negative case. 3. Read audit before/after. 4. Attempt ordinary audit update/delete. 5. Send an action with stale ETag or verify single-writer fallback. 6. Confirm failed calls leave no event/state mutation.
- **FIELDS / METHODS / ACTIONS:** All four workflow actions, status/outcome, ActorUser, OccurredAt, Reason, ETag.
- **EXPECTED BEHAVIOR:** Only authorized legal transitions succeed; each success writes one immutable event; stale updates fail or documented lock serializes.
- **TEST CASES:** Valid/invalid submit; valid/unauthorized approval; override blank/valid; self-approval; illegal transition; stale update; audit immutability.
- **ACCEPTANCE CRITERIA:** Every expected response/state/event asserted; no production auth claim without tenant-role evidence.
- **DEPENDENCIES:** M2 snapshot; M3 auth mapping/behavior; SAP admin test users.
- **COMMIT / DELIVERABLE:** Workflow matrix, sanitized request/event traces, transport/test output.

M4-D14-T01 — Run browser-level UI regression for all current pages

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** `docs/clean-core/test-evidence/ui-matrix.md`; supported FE app annotations/routes and browser test artifacts used by the team.
- **PURPOSE:** Verify user-visible list/details/actions/error behavior against real services.
- **PRECONDITIONS:** Stable test tenant, synthetic data, M1–M3 service versions, role-based test accounts.
- **IMPLEMENTATION STEPS:** 1. Run inventory search/filter. 2. Open evidence/recommendation/matches. 3. Perform valid workflow action and inspect audit. 4. Trigger invalid/unauthorized action. 5. Capture console/network errors and responsive viewport screenshots.
- **FIELDS / METHODS / ACTIONS:** Inventory fields; recommendation fields; decision actions; audit fields; portfolio placeholders.
- **EXPECTED BEHAVIOR:** Every displayed value reflects service response; errors are actionable; direct route reload works.
- **TEST CASES:** Inventory appears; search/filter; details; explanation; submit/approve; override reason required; audit visible; loading/empty/403/500 states.
- **ACCEPTANCE CRITERIA:** No high-severity UI defect; every route/action has a recorded pass/fail and backend owner for defects.
- **DEPENDENCIES:** All M1–M3 services; supported browser/FE runtime.
- **COMMIT / DELIVERABLE:** UI regression matrix, screenshots and bug tickets/rows in blocker log.

DAY 15 — Integrate workflow actions and audit history into the Fiori Object Page

M1-D15-T01 — Finalize read-only evidence/import navigation in cockpit service

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** M1 projections and contribution to `ZUI_CC_COCKPIT`; association links from object to facts and batch provenance.
- **PURPOSE:** Ensure M4's Object Page can navigate from an evidence/finding row to the originating import batch.
- **PRECONDITIONS:** M1 import regression green; M3 service binding; M4 screen field matrix.
- **IMPLEMENTATION STEPS:** 1. Add/verify `_ImportBatch` links on imported rows. 2. Expose read-only provenance fields authorized by M3. 3. Ensure source reference is text, not a clickable untrusted URL. 4. Verify pagination and stable row keys.
- **FIELDS / METHODS / ACTIONS:** SourceName, SourceReference, SourceRecordKey, CapturedAt, ImportBatchUUID, BatchUUID/filename/status.
- **EXPECTED BEHAVIOR:** Analyst can trace one value to source row and batch without editing the fact.
- **TEST CASES:** Evidence→batch link; missing SourceReference; unauthorized batch access; multi-page facts stable key.
- **ACCEPTANCE CRITERIA:** Provenance association works and is read-only; no private data appears to unauthorized role.
- **DEPENDENCIES:** M1 batch/fact views; M3 DCL; M4 navigation configuration.
- **COMMIT / DELIVERABLE:** Transport and sanitized provenance query evidence.

M2-D15-T01 — Expose recommendation match and portfolio read services

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** `ZC_CC_RECOMMENDATION`, `ZC_CC_RULE_MATCH`, initial `ZC_CC_PORTFOLIO` CDS/read model; M2 service fields in `ZUI_CC_COCKPIT` handoff.
- **PURPOSE:** Give M4 stable, read-only OData data for rule detail and later portfolio KPIs.
- **PRECONDITIONS:** M2 recommendation service and match persistence; M1 object/source associations; Day-1 CDS aggregate branch.
- **IMPLEMENTATION STEPS:** 1. Project snapshots/matches. 2. Define portfolio grouping keys and count fields from v1. 3. Implement CDS aggregates or documented class-query fallback. 4. Apply access annotations. 5. Validate metadata and deterministic grouping.
- **FIELDS / METHODS / ACTIONS:** Recommendation and matches from §4; portfolio keys `SystemCode`, `SoftwareComponent`, `ObjectType`; counts per outcome/evidence/status.
- **EXPECTED BEHAVIOR:** Each detail row ties to one recommendation version; portfolio totals describe the same object population.
- **TEST CASES:** Query recommendation and match by UUID; no match returns empty; portfolio counts for 3 fixtures equal expected.
- **ACCEPTANCE CRITERIA:** M4 can bind without private types; counts reconcile with direct test-fixture count.
- **DEPENDENCIES:** M1 projections; M2 data; service definition owner M3.
- **COMMIT / DELIVERABLE:** Transport, metadata and reconciliation output.

M3-D15-T01 — Expose decision actions and audit history in final service

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** `ZC_CC_DECISION`, `ZC_CC_DECISION_AUDIT`, service definition `ZUI_CC_COCKPIT`, supported binding; BDEF action annotations.
- **PURPOSE:** Publish the verified workflow contract to M4 as a supported UI service.
- **PRECONDITIONS:** M3 Day-14 action/auth/audit tests pass; M1/M2 projections activate.
- **IMPLEMENTATION STEPS:** 1. Expose decision root, action parameter entities and audit projection. 2. Include M1/M2 projections only after owner activation. 3. Publish/restart the selected binding using tenant process. 4. Capture `$metadata` and run valid/invalid action calls with role test users.
- **FIELDS / METHODS / ACTIONS:** `assignOwner`, `overrideRecommendation`, `submitDecision`, `approveDecision`; read-only audit projection.
- **EXPECTED BEHAVIOR:** Service metadata matches v1; successful actions change server state and create audit; invalid calls return RAP messages.
- **TEST CASES:** Action metadata check; valid override call; blank reason; self-approval denied; audit read only.
- **ACCEPTANCE CRITERIA:** All action imports in M4 resolve; no stale service metadata; authorization tests remain green.
- **DEPENDENCIES:** M1/M2 projections and M3 RAP/security.
- **COMMIT / DELIVERABLE:** Service transport, metadata snapshot, action test evidence.

M4-D15-T01 — Integrate workflow and audit on Object Page

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** Generated Fiori app `webapp/manifest.json`, object-page annotations/config, and only generator-supported extension files if required.
- **PURPOSE:** Complete the decision workspace with server-backed state transitions and history.
- **PRECONDITIONS:** M3 combined service metadata; M2 recommendation projection; M1 evidence sections; test principals.
- **IMPLEMENTATION STEPS:** 1. Bind decision status/owner/snapshot. 2. Wire all four RAP actions. 3. Show override dialog with mandatory reason. 4. Add audit table ordered by OccurredAt. 5. Refresh the page after action; preserve selection/error messages. 6. Keep recommendation separate from decision outcome.
- **FIELDS / METHODS / ACTIONS:** Decision status/outcome/owner; action IDs; audit actor/time/event/reason; recommendation/rule version.
- **EXPECTED BEHAVIOR:** Override, submit, approve cause server refresh and corresponding audit row; denied action does not alter UI state.
- **TEST CASES:** Valid workflow; blank reason; unauthorized approval; self-approval; audit order; expired session/service error.
- **ACCEPTANCE CRITERIA:** UI actions call the combined service and all statuses are server-derived; no action button bypasses role checks.
- **DEPENDENCIES:** M1/2 projections; M3 service/actions/auth; supported FE runtime.
- **COMMIT / DELIVERABLE:** UI commit and full workflow screenshot set.

DAY 16 — Complete inventory filters, evidence display, recommendation detail, and UI errors

M1-D16-T01 — Add supported server-side inventory and evidence filters

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** `ZC_CC_CUSTOM_OBJECT`, `ZC_CC_EVIDENCE`, `ZC_CC_USAGE_FACT`, `ZC_CC_FINDING`, `ZC_CC_DEPENDENCY` metadata/projection definitions.
- **PURPOSE:** Make the UI filters use actual indexed/service-supported properties and return consistent inventory rows.
- **PRECONDITIONS:** M1 entities/projections and M3 DCL active; M4 field/filter list frozen.
- **IMPLEMENTATION STEPS:** 1. Verify filters for system, object type, component, owner, evidence status where exposed. 2. Add `@Search`/selection annotations only for supported OData fields. 3. Verify sort/page behavior. 4. Preserve authorization filters; never load all rows to filter in the browser.
- **FIELDS / METHODS / ACTIONS:** SystemCode, ObjectType, SoftwareComponent, OwnerUser, ObjectName; evidence QualityCode/CapturedAt.
- **EXPECTED BEHAVIOR:** Search/filters constrain the backend query and retain object associations.
- **TEST CASES:** Filter by each field singly and combined; unknown filter value returns empty; unauthorized source remains hidden.
- **ACCEPTANCE CRITERIA:** Service query results match expected fixture IDs; no metadata/filter error on supported binding.
- **DEPENDENCIES:** M1 CDS and M3 DCL; M4 UI filters.

- **COMMIT / DELIVERABLE:** Transport and filter-result evidence.

M2-D16-T01 — Finalize recommendation explanation and input provenance

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** `ZCL_CC_CLASSIFIER`, `ZCL_CC_RECOMMENDATION_SERVICE`, recommendation/match CDS projections.
- **PURPOSE:** Make each recommendation explain both the selected rule and the evidence data used, without claiming unsupported source facts.
- **PRECONDITIONS:** M1 provenance links; Day-14 rules green; M4 detail fields.
- **IMPLEMENTATION STEPS:** 1. Include matched-rule explanation and stable missing-code ordering. 2. Ensure original SourceRecordKey/ImportBatchUUID can be navigated via M1 association. 3. Mark demo rule-set description/policy version in output. 4. Test no-match/system-fallback explanation.
- **FIELDS / METHODS / ACTIONS:** Rationale, MissingEvidenceCodes, RuleID, Priority, IsSelected, RuleSetVersion, EvidenceStatus, source/batch links.
- **EXPECTED BEHAVIOR:** Analyst can see why a rule matched and which evidence is missing, stale or unknown.
- **TEST CASES:** KEEP rule explanation; MANUAL_REVIEW missing reason; conflict explanation lists both IDs; unknown optional value does not appear as asserted fact.
- **ACCEPTANCE CRITERIA:** Every outcome has a nonblank rationale; fallback has no fake RuleUUID; evidence provenance resolves.
- **DEPENDENCIES:** M1 source records; M2 evaluator/match schema; M4 page.
- **COMMIT / DELIVERABLE:** Transport and rationale/match regression table.

M3-D16-T01 — Verify authorization on every read and action path

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** DCL/access-control objects `ZP_CC_OBJECT`, `ZP_CC_IMPORT`, `ZP_CC_DECISION`, `ZP_CC_AUDIT`, `ZP_CC_RULESET` where supported; RAP auth handlers.
- **PURPOSE:** Close direct-URL/service-call gaps after frontend wiring.
- **PRECONDITIONS:** Day-13 role matrix; M1/M2 all exposed projections; Day-14 tests.
- **IMPLEMENTATION STEPS:** 1. Enumerate every entity set and action in `$metadata`. 2. Apply role/instance check to each protected path. 3. Test direct OData calls bypassing UI visibility. 4. Confirm read-only audit and immutable active rules. 5. Document any entity intentionally public within authenticated tenant.
- **FIELDS / METHODS / ACTIONS:** Entity sets and action list from §5; current-principal and role API per actual platform.
- **EXPECTED BEHAVIOR:** No API path grants more capability than its UI role; unauthorized calls fail with no persisted side effect.
- **TEST CASES:** Direct approve as analyst; read audit as no-role; edit active rule; processBatch without import role; all must follow role matrix.
- **ACCEPTANCE CRITERIA:** Every exposed operation has a test and owner; missing platform authorization remains a blocker, not a waiver.
- **DEPENDENCIES:** M1/M2 service metadata; M3 auth implementation; SAP role admin.
- **COMMIT / DELIVERABLE:** Updated authorization matrix, transport and direct-call traces.

M4-D16-T01 — Finish inventory, evidence, recommendation, and error UX

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** Generated List Report/Object Page annotations/config and supported `webapp/ext/**` files only where the selected generator requires them.
- **PURPOSE:** Finish the analyst's end-to-end read flow and distinguish loading, empty, stale/missing and service-failure states.
- **PRECONDITIONS:** M1 filters/provenance; M2 recommendation match fields; M3 security test pass.
- **IMPLEMENTATION STEPS:** 1. Bind filters to server metadata. 2. Show missing/stale/unknown separately. 3. Add batch/source navigation. 4. Add loading/empty/403/404/500 states. 5. Use concise text and accessible labels; do not portray a missing numeric count as zero.
- **FIELDS / METHODS / ACTIONS:** Inventory, facts, provenance, recommendation, match, decision fields from §4; no client classification logic.
- **EXPECTED BEHAVIOR:** Each state tells user whether data is absent, incomplete, unauthorized or unavailable.
- **TEST CASES:** Inventory filter; evidence present/empty; missing evidence; stale record; unknown object; 403/500; narrow/mobile viewport.
- **ACCEPTANCE CRITERIA:** No unhandled UI exception; filter values equal service output; no fake zero/default outcome.
- **DEPENDENCIES:** M1/M2 services and M3 access control.
- **COMMIT / DELIVERABLE:** UI commit, screenshots and updated UI test matrix.

DAY 17 — Build portfolio metrics and filters from supported read models

M1-D17-T01 — Validate the portfolio source population and import freshness

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** `ZC_CC_CUSTOM_OBJECT/ZC_CC_USAGE_FACT` read projections and `docs/clean-core/test-evidence/portfolio-source-counts.md`.
- **PURPOSE:** Supply M2 one authoritative inventory population and confirm filters exclude duplicate/import-error rows.
- **PRECONDITIONS:** Import regression green; M2 portfolio grouping keys frozen.
- **IMPLEMENTATION STEPS:** 1. Define the object population query (all persisted unique objects in selected systems). 2. Verify only persisted facts count; rejected/staged rows do not. 3. Add supported captured-at fields for any freshness display. 4. Produce a fixture count by system/component/type.
- **FIELDS / METHODS / ACTIONS:** ObjectUUID, SourceSystemUUID/SystemCode, SoftwareComponent, ObjectType, Usage CapturedAt.
- **EXPECTED BEHAVIOR:** Same filter set returns the same object count to M2 and M4.
- **TEST CASES:** Five-object fixture gives 5 total; rejected object row adds 0; duplicate re-import leaves total at 5; system filter yields the five expected IDs.
- **ACCEPTANCE CRITERIA:** Fixture population and grouping are documented and query results reconcile.
- **DEPENDENCIES:** M2 analytics keys; M1 import service.
- **COMMIT / DELIVERABLE:** Transport if projection changes; source-count evidence.

M2-D17-T01 — Implement portfolio aggregate read model and formula tests

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** `ZC_CC_PORTFOLIO` CDS view entity and `ZC_CC_PORTFOLIO_MDE`; conditional fallback class `ZCL_CC_PORTFOLIO_QUERY` if CDS aggregation failed Day 1.
- **PURPOSE:** Return portfolio counts and priority totals from the same filtered object/recommendation population.
- **PRECONDITIONS:** M1 source counts; M2 recommendation snapshots; Day-1 aggregate capability branch.
- **IMPLEMENTATION STEPS:** 1. Group by SystemCode, SoftwareComponent, ObjectType. 2. Calculate total objects; each outcome count; incomplete evidence; open decisions; priority band counts. 3. Ensure one current recommendation per object for current-state counts; preserve historical snapshots only in detail. 4. Add filter annotations. 5. Test sum-of-buckets equals total population.
- **FIELDS / METHODS / ACTIONS:** Contracted `TotalObjectCount`, `KeepCount`, `RemediateCount`, `ReplaceCount`, `RetireCount`, `ManualReviewCount`, `IncompleteEvidenceCount`, `OpenDecisionCount`, `HighPriorityCount`, `MediumPriorityCount`, `LowPriorityCount`.
- **EXPECTED BEHAVIOR:** A selected portfolio slice’s outcome counts reconcile to its object total; an object without a latest recommendation is counted as `MANUAL_REVIEW` and incomplete with demo score 50/`MEDIUM` until reevaluated.
- **TEST CASES:** Five synthetic objects with one of each outcome → total 5 and KEEP/REMEDIATE/REPLACE/RETIRE/MANUAL_REVIEW counts 1/1/1/1/1; incomplete=1; priority counts HIGH=0/MEDIUM=1/LOW=4; open decisions=0 after clean reset; system filter yields 5; empty scope returns zeros.
- **ACCEPTANCE CRITERIA:** Aggregate query/class output matches independent expected fixture calculation; no double count from multiple recommendation history rows.
- **DEPENDENCIES:** M1 population/read views; M2 recommendation & decision association; M3 decision status read.
- **COMMIT / DELIVERABLE:** Transport/query class and metric reconciliation report.

M3-D17-T01 — Verify open-decision portfolio count and access behavior

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** `ZC_CC_DECISION` projection and service authorization definitions; `docs/clean-core/test-evidence/portfolio-security.md`.
- **PURPOSE:** Ensure OpenDecisionCount uses the agreed DRAFT/SUBMITTED states and is subject to the same authorization policy.
- **PRECONDITIONS:** M2 portfolio query; M3 state/auth contract; M4 overview filters.
- **IMPLEMENTATION STEPS:** 1. Define OpenDecisionCount as objects whose sole open decision is DRAFT or SUBMITTED. 2. Exclude historical APPROVED decisions from open count. 3. Verify no object is counted twice if it has one approved history and one new open decision. 4. Test direct query with each role.
- **FIELDS / METHODS / ACTIONS:** DecisionStatus, ObjectUUID; portfolio `OpenDecisionCount` only (v1 excludes approved-history KPI).
- **EXPECTED BEHAVIOR:** An approved historical decision plus a current submitted decision yields one open object, not two; an approved-only object yields zero open.
- **TEST CASES:** Approved-only → 0; one approved history + one current SUBMITTED → 1; two open rows blocked by M3 invariant; no-role query denied per policy.
- **ACCEPTANCE CRITERIA:** Definition is in canonical contract; counts reconcile with direct RAP query.
- **DEPENDENCIES:** M2 portfolio model; M3 decision lifecycle and DCL.

- **COMMIT / DELIVERABLE:** State-count decision, transport if needed, test evidence.

M4-D17-T01 — Build portfolio overview and interactive filters

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** Fiori Elements portfolio route/page in generated app; if FE cannot render the read model, use a generator-supported custom page in `webapp/ext/` (path frozen from generator) and document it.
- **PURPOSE:** Present the agreed KPIs and allow drill-down to the filtered inventory list.
- **PRECONDITIONS:** M2 portfolio read model; M1 source/component fields; M3 authorized decision counts.
- **IMPLEMENTATION STEPS:** 1. Add KPI cards for total/outcomes/incomplete evidence/open decisions and priority bands. 2. Add filters for system/component/type/outcome/priority/status supported by service. 3. Make KPI selection navigate with the same filter set to Inventory. 4. Label score as project demo triage only.
- **FIELDS / METHODS / ACTIONS:** Portfolio metric properties from M2; route `Portfolio`; navigation target `CustomObjects`.
- **EXPECTED BEHAVIOR:** Filtered cards and detail list share one population; no stale count after filter change.
- **TEST CASES:** Unfiltered fixture metrics; system filter; click `ManualReviewCount`; empty state; service error; browser refresh on portfolio route.
- **ACCEPTANCE CRITERIA:** Every displayed count equals M2 test fixture; filters persist in navigation; no hard-coded KPI values.
- **DEPENDENCIES:** M1/M2 read services, M3 decision metric policy.
- **COMMIT / DELIVERABLE:** UI commit, screenshot and metric-to-service mapping.

DAY 18 — Integration/regression, fix high-priority defects, and retest

M1-D18-T01 — Run import/data regression after integration fixes

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** All M1 importer/parser/validator test classes; `docs/clean-core/test-evidence/day18-backend-regression.md`.
- **PURPOSE:** Verify end-to-end data integrity after service, security and UI integration changes.
- **PRECONDITIONS:** Defect list triaged; no unreviewed contract change.
- **IMPLEMENTATION STEPS:** 1. Clean/reseed synthetic test data using approved method. 2. Run all parser, validator, idempotency and integration tests. 3. Compare row, entity, provenance and recommendation counts. 4. Retest only after fixes and mark commit/transport ID.
- **FIELDS / METHODS / ACTIONS:** All import kinds/row statuses, source keys, batch references, affected object list.
- **EXPECTED BEHAVIOR:** Re-run produces same entity counts and expected outcomes; rejected rows never leak.
- **TEST CASES:** Entire Import test matrix in §10 plus repeat import after service restart.
- **ACCEPTANCE CRITERIA:** No critical/high data-loss or duplicate defect; all counts reconcile.
- **DEPENDENCIES:** M2/M3 integration changes; M1 defect owners.
- **COMMIT / DELIVERABLE:** Regression run log, fixture versions and transport IDs.

M2-D18-T01 — Run classifier and portfolio regression on the frozen fixture

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** M2 ABAP Unit classes, `ZC_CC_PORTFOLIO`/fallback query, `docs/clean-core/test-evidence/day18-analytics-regression.md`.
- **PURPOSE:** Verify rules, recommendation history and metrics remain consistent after UI/workflow connections.
- **PRECONDITIONS:** Active v1 rules; frozen fixture; any policy change approved in contract log.
- **IMPLEMENTATION STEPS:** 1. Run full rule/freshness/conflict suite. 2. Recalculate batch fixtures. 3. Compare recommendation snapshots/matches and portfolio counts. 4. Verify existing decision snapshot unchanged after new recommendation. 5. Retest all analytics defects.
- **FIELDS / METHODS / ACTIONS:** Outcomes, missing codes, version, score, bands, match IDs, KPI properties.
- **EXPECTED BEHAVIOR:** Same frozen fixture and time yield byte-for-byte stable business result; portfolio totals reconcile.
- **TEST CASES:** Full rule suite, deterministic repeat, historical snapshot, all metric filters, empty dataset.
- **ACCEPTANCE CRITERIA:** No high-priority analytics defect; all expected/actual outputs recorded.
- **DEPENDENCIES:** M1 fixtures; M3 decision state/query; M2 implementation.
- **COMMIT / DELIVERABLE:** Test log and transport IDs.

M3-D18-T01 — Run workflow/security/audit regression in target tenant

- **OWNER:** M3 — RAP Workflow & Security.

- **OBJECT TO CREATE OR MODIFY:** Workflow/security ABAP Unit and integration evidence `docs/clean-core/test-evidence/day18-security-regression.md`.
- **PURPOSE:** Prove behavior remains secure and append-only after final service binding and UI integration.
- **PRECONDITIONS:** Tenant test users/roles; M3 auth profile; Day-14 test baseline.
- **IMPLEMENTATION STEPS:** 1. Replay legal path. 2. Replay every forbidden action over direct OData, not only UI. 3. Check audit rows and before/after values. 4. Replay stale ETag/lock contention. 5. Retest any role/binding change.
- **FIELDS / METHODS / ACTIONS:** Four decision actions; audit fields; lock/ETag and DCL results.
- **EXPECTED BEHAVIOR:** Unauthorized or invalid call causes no state transition/success audit; successful call has one correct event.
- **TEST CASES:** Workflow and audit matrix in §10, including self-approval and ordinary history update/delete.
- **ACCEPTANCE CRITERIA:** Zero open critical security finding; unsupported instance/concurrency feature has explicit limitation and disabled path.
- **DEPENDENCIES:** M1/M2 stable services, SAP admin role configuration.
- **COMMIT / DELIVERABLE:** Sanitized test traces and security sign-off/remaining blocker.

M4-D18-T01 — Run full UI/API E2E regression and close P1 defects

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** `docs/clean-core/test-evidence/day18-ui-regression.md`; frontend generated app files only for approved fixes.
- **PURPOSE:** Execute the full user journey and verify live UI values against backend test oracles.
- **PRECONDITIONS:** M1–M3 regression builds; Day-17 portfolio; synthetic data and roles.
- **IMPLEMENTATION STEPS:** 1. Reset to frozen fixture. 2. Run import, inventory search, object detail, recommendation, decision actions, audit, portfolio. 3. Compare KPI/detail data. 4. Test loading, empty, 403/404/500, expired login. 5. Fix P1/P2 defects and rerun the affected test plus full smoke suite.
- **FIELDS / METHODS / ACTIONS:** Every route/entity/action in §10; no local mocks in release profile.
- **EXPECTED BEHAVIOR:** All core criteria pass in the actual target runtime; no console/network error masks failure.
- **TEST CASES:** Complete UI matrix; viewport 390/768/1440 px where the supported app permits.
- **ACCEPTANCE CRITERIA:** No open P1 defect; every P2 is resolved or accepted with owner/impact; regression evidence timestamped.
- **DEPENDENCIES:** M1/M2/M3 release candidates.
- **COMMIT / DELIVERABLE:** UI regression report, screenshots, fixed-app Git commit.

DAY 19 — Finalize setup, architecture, test evidence, screenshots, and demo dataset

M1-D19-T01 — Write data/import setup and recovery instructions

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** `docs/clean-core/setup/import-runbook.md`, CSV templates/demo files, `docs/clean-core/architecture/data-model.mmd`.
- **PURPOSE:** Enable another team member to load the same safe dataset, inspect provenance, and correct/retry a row.
- **PRECONDITIONS:** Day-18 importer pass; final import mode/profile known.
- **IMPLEMENTATION STEPS:** 1. List import order, header version, row limits, batch process steps. 2. Explain every status/error and corrected re-import. 3. Document safe cleanup/reset method and no-production-seed rule. 4. Draw entity relationships and provenance path. 5. Include exact transport/dependency setup.
- **FIELDS / METHODS / ACTIONS:** CSV kinds/header names, `processBatch`, statuses/error codes, FKs.
- **EXPECTED BEHAVIOR:** A fresh test operator reproduces the fixture without undocumented access or live SAP data.
- **TEST CASES:** Follow runbook from blank test dataset; compare expected row counts and provenance.
- **ACCEPTANCE CRITERIA:** Runbook steps match actual service and generated errors; diagram has no undocumented entity.
- **DEPENDENCIES:** M1 release candidate; M4 canonical docs structure.
- **COMMIT / DELIVERABLE:** Docs/CSV Git commit and runbook walkthrough evidence.

M2-D19-T01 — Document rule semantics, versions, and limits

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** `docs/clean-core/rules/rule-catalog-v1.md`, `docs/clean-core/architecture/classification.mmd`, test-output table.
- **PURPOSE:** Let reviewers trace each recommendation to explicit synthetic input and prevent demo thresholds being misrepresented.
- **PRECONDITIONS:** Day-18 rules green; final policy reconciliation recorded.

- **IMPLEMENTATION STEPS:** 1. List each rule ID/priority/preconditions/evidence/output/explanation/missing/conflict behavior. 2. Document freshness clock, priority tie rule, no-match fallback, score formula. 3. Label project-defined policies and synthetic data. 4. Show recommendation/version/match data flow. 5. Attach exact ABAP Unit expected/actual summary.
- **FIELDS / METHODS / ACTIONS:** RuleSet code/version, RuleID, InputCode, operator, OutcomeCode, score fields.
- **EXPECTED BEHAVIOR:** A demo viewer cannot mistake the rule set for SAP migration authority.
- **TEST CASES:** Cross-check every catalog case against a passing test ID and stored match row.
- **ACCEPTANCE CRITERIA:** All active rules appear once in docs; no rule lacks test/explanation/version.
- **DEPENDENCIES:** M1 demo input, M3 workflow snapshot, M2 test outputs.
- **COMMIT / DELIVERABLE:** Rule catalog/diagram Git commit.

M3-D19-T01 — Document security model, transition diagram, and platform limits

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** `docs/clean-core/security/authorization-matrix.md`, `docs/clean-core/architecture/workflow.mmd`, `docs/clean-core/setup/backend-activation.md`.
- **PURPOSE:** Document exactly which platform identity/roles protect each action and how audit/concurrency behave.
- **PRECONDITIONS:** Day-18 security test and Day-1 capability evidence.
- **IMPLEMENTATION STEPS:** 1. Separate platform authentication from RAP authorization. 2. Map actual tenant role names to analyst/reviewer/admin aliases. 3. Draw DRAFT→SUBMITTED→APPROVED and action validations. 4. Document append-only guarantee scope and tenant-side limits. 5. List activation/package/service sequence and blocker alternatives.
- **FIELDS / METHODS / ACTIONS:** Role/action matrix, ActorUser/OccurredAt/Reason, DCL/BDEF names, ETag/lock mode.
- **EXPECTED BEHAVIOR:** Setup steps reproduce the protected service and never suggest app-managed passwords.
- **TEST CASES:** Reviewer matrix matches Day-18 traces; every action maps to exactly one test; diagram transition labels equal BDEF.
- **ACCEPTANCE CRITERIA:** No unsupported auth/concurrency claim; admin confirms role setup; open limitation has owner.
- **DEPENDENCIES:** M3 tenant roles and backend transport; M4 docs navigation.
- **COMMIT / DELIVERABLE:** Security/setup docs commit and admin sign-off or explicit blocker.

M4-D19-T01 — Assemble release guide, screenshots, and test evidence index

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** `docs/clean-core/README.md`, `docs/clean-core/demo/demo-script.md`, `docs/clean-core/test-evidence/index.md`, screenshots under `docs/clean-core/test-evidence/screenshots/`.
- **PURPOSE:** Produce a single entry point for setup, screens, regression evidence, limitations and presentation.
- **PRECONDITIONS:** M1–M3 documentation and Day-18 regression results; final Fiori profile.
- **IMPLEMENTATION STEPS:** 1. Link ABAP activation/import/rules/workflow runbooks. 2. Capture Inventory, Object Page, recommendation explanation, decision/audit, portfolio, import error views. 3. Record screen size/runtime/date and synthetic-data label. 4. Link test matrices and accepted defects. 5. Verify screenshots contain no usernames, tokens, real SAP data or sensitive URLs.
- **FIELDS / METHODS / ACTIONS:** Route names, service names, displayed fields/actions, acceptance IDs.
- **EXPECTED BEHAVIOR:** New reviewer can follow the demo and locate evidence for each acceptance claim.
- **TEST CASES:** Open every relative link; compare screenshots with release build; scan for secrets/real data.
- **ACCEPTANCE CRITERIA:** All required screenshots/docs exist and are internally linked; no broken path or secret.
- **DEPENDENCIES:** M1/M2/M3 docs; release-candidate frontend.
- **COMMIT / DELIVERABLE:** Documentation/screenshots Git commit and polished demo script.

DAY 20 — Run the demonstration, verify acceptance, and hand off limitations

M1-D20-T01 — Demonstrate repeatable, provenance-safe import

- **OWNER:** M1 — ABAP Data & Import.
- **OBJECT TO CREATE OR MODIFY:** Final `import-runbook.md` and Day-20 run log `docs/clean-core/test-evidence/day20-demo.md`.
- **PURPOSE:** Prove the same synthetic CSV can be ingested, diagnosed, retried and audited without duplicate rows.
- **PRECONDITIONS:** Release-candidate backend, approved non-production tenant, Day-19 fixture/runbook.

- **IMPLEMENTATION STEPS:** 1. Load source systems→objects→facts. 2. Show one malformed/missing-field/reference row error. 3. Re-import identical file and show duplicate/no-change counts. 4. Correct rejected row and re-import. 5. Trace an evidence row to source/batch and report final object counts.
- **FIELDS / METHODS / ACTIONS:** Batch/row statuses, source keys, provenance, counts.
- **EXPECTED BEHAVIOR:** Valid data persists once; malformed/reference rows never create partial records; repeat import is idempotent.
- **TEST CASES:** Valid, missing field, invalid value/reference, duplicate, repeat, malformed, corrected re-import, incomplete-evidence warning.
- **ACCEPTANCE CRITERIA:** Live results equal import matrix and fixture; no real customer data is used.
- **DEPENDENCIES:** M2 classifier callback and M3 authorization; approved seed/import route.
- **COMMIT / DELIVERABLE:** Day-20 run log, final counts and any blocker/limitation.

M2-D20-T01 — Demonstrate explainable recommendations and correct metrics

- **OWNER:** M2 — Rules Engine & Analytics.
- **OBJECT TO CREATE OR MODIFY:** Final `rule-catalog-v1.md` and Day-20 expected/actual results in `day20-demo.md`.
- **PURPOSE:** Prove recommendation is deterministic, versioned and evidence-linked—not a black-box SAP claim.
- **PRECONDITIONS:** M1 demo dataset and portfolio page; fixed active RuleSet; M4 screenshots.
- **IMPLEMENTATION STEPS:** 1. Open one object each for KEEP, REMEDIATE, and MANUAL_REVIEW. 2. Show matched rule/version and rationale. 3. Show missing/stale evidence behavior and tie fallback test evidence. 4. Reconcile portfolio counts against fixture. 5. State demo-policy disclaimer verbally and in UI.
- **FIELDS / METHODS / ACTIONS:** Outcome, EvidenceStatus, RuleSetVersion, RuleID/priority/explanation, score/band, portfolio metrics.
- **EXPECTED BEHAVIOR:** Outputs equal frozen test oracle; no unknown input yields a positive recommendation.
- **TEST CASES:** Day-9 rule cases; Day-17 portfolio totals; repeated evaluation with same AsOfUtc.
- **ACCEPTANCE CRITERIA:** All expected/actual results match; no recommendation is described as SAP-authored guidance.
- **DEPENDENCIES:** M1 persisted facts; M3 decision snapshot; M4 app.
- **COMMIT / DELIVERABLE:** Final result table, Day-20 test log and any accepted policy limitations.

M3-D20-T01 — Demonstrate authorized lifecycle and immutable history

- **OWNER:** M3 — RAP Workflow & Security.
- **OBJECT TO CREATE OR MODIFY:** Final `authorization-matrix.md` and Day-20 workflow/audit run log.
- **PURPOSE:** Prove an assigned user can submit, a distinct authorized reviewer can approve, and history cannot be changed via the normal service.
- **PRECONDITIONS:** Two platform test principals with verified roles; release-candidate RAP service; Day-14/18 security passes.
- **IMPLEMENTATION STEPS:** 1. Create DRAFT, assign owner, override with a reason, submit as owner. 2. Attempt approval as submitter and show denial. 3. Approve as separate reviewer. 4. Read event sequence/actor/time/reason. 5. Attempt ordinary audit update/delete; show denial. 6. State any platform limitation.
- **FIELDS / METHODS / ACTIONS:** DecisionStatus/Outcome, owner, SubmittedBy/ApprovedBy, four actions, audit fields and ETag.
- **EXPECTED BEHAVIOR:** Valid route reaches APPROVED; unauthorized/self-approval and invalid reason do not mutate state; audit contains successful events only.
- **TEST CASES:** Valid path, override without reason, unauthorized approval, self-approval, illegal transition, stale update, audit tamper attempt.
- **ACCEPTANCE CRITERIA:** Actual service enforcement and audit test evidence; no authentication method invented or credentials shown.
- **DEPENDENCIES:** M2 recommendation; M4 UI; platform role assignment.
- **COMMIT / DELIVERABLE:** Final security sign-off or explicit blocked acceptance item; Day-20 run log.

M4-D20-T01 — Deliver final demo, acceptance review, and handoff

- **OWNER:** M4 — Fiori UI, QA & Integration.
- **OBJECT TO CREATE OR MODIFY:** `docs/clean-core/demo/demo-script.md`, `docs/clean-core/test-evidence/day20-demo.md`, final supported app build/preview.
- **PURPOSE:** Run the integrated product demonstration and hand off reproducible artifacts and limitations.
- **PRECONDITIONS:** M1/M2/M3 demo setup; all critical defects resolved; Day-19 docs complete.
- **IMPLEMENTATION STEPS:** 1. Run the sequence in §12. 2. Verify inventory, search/filter, detail/evidence, explanation/matched rules, decision/override/approval, audit and portfolio. 3. Run at least one import error and corrected retry. 4. Check screenshots and acceptance checklist. 5. Publish final build/transport identifiers and list remaining blockers/stretch items.
- **FIELDS / METHODS / ACTIONS:** Every route and action in the final contract; metrics from M2; decision state from M3.

- **EXPECTED BEHAVIOR:** App is usable through the verified tenant/runtime and every displayed value is service-backed.
- **TEST CASES:** Full E2E and UI matrix; direct route refresh; 390/768/1440 widths if supported; error/empty/unauthorized states.
- **ACCEPTANCE CRITERIA:** §11 checklist is signed by all four owners; demo completed from clean synthetic fixture; unresolved items are explicit, not hidden.
- **DEPENDENCIES:** All team deliverables, platform roles and service deployment.
- **COMMIT / DELIVERABLE:** Final UI/docs Git commit, backend transport list, screenshots, demo recording if approved, and signed limitation list.

8. Exact artifact inventory (proposed names; ADT names finalized Day 1)

All backend objects below are **ADT repository objects in package** `ZCC_MIG_COCKPIT` **(or the verified assigned package)**, not filesystem files. Activate tables before CDS; activate interface/root CDS before projections; activate behavior definitions/pools before service metadata; activate service definition before binding. Each object is verified by activation, its consumer query/action, and the named ABAP Unit/integration case.

Persistence tables — ADT `Define Table` / DDL source

Technical name	Description	Owner	Dependencies / activation and verification
`ZCC_SRCSYS`	Source system	M1	Day-3 package/type check; verify source CDS lookup
`ZCC_OBJECT`	Custom-object inventory	M1	`ZCC_SRCSYS`; unique business-key validation; verify object/source association
`ZCC_USAGE`	Period usage snapshot	M1	`ZCC_OBJECT`; nonnegative count and ordered dates
`ZCC_EVIDENCE`	Typed evidence/provenance	M1	`ZCC_OBJECT`,`ZCC_BATCH`; value/quality validation
`ZCC_FINDING`	Imported finding	M1	`ZCC_OBJECT`,`ZCC_BATCH`; source severity preserved/normalized per contract
`ZCC_DEPEND`	Directed object dependency	M1	Both object FKs; reject missing target
`ZCC_BATCH`	Import batch and totals	M1	No record writes until valid process action
`ZCC_IMPROW`	Per-row import result/raw bounded row	M1	`ZCC_BATCH`; unique row number per batch
`ZCC_RULESET`	Versioned rule-set header	M2	`ZCC_RULE`; only one active version per code
`ZCC_RULE`	Allowlisted classification rules	M2	`ZCC_RULESET`; active version immutable
`ZCC_RECOMM`	Immutable evaluation snapshot	M2	`ZCC_OBJECT`,`ZCC_RULESET`; verify historical snapshot read
`ZCC_RMATCH`	Per-rule match/fallback trace	M2	`ZCC_RECOMM`, optional `ZCC_RULE`; verify selected/conflict rows
`ZCC_DECISION`	Decision root	M3	Object and recommendation links; behavior validates open-decision invariant
`ZCC_AUDIT`	Append-only workflow event	M3	`ZCC_DECISION`; no service update/delete

CDS, RAP, security, and service repository objects

Object type	Proposed technical name(s)	Owner	Purpose / dependencies / verification
CDS interface/root view entities (DDLS)	ZI_CC_SOURCE_SYSTEM, ZI_CC_CUSTOM_OBJECT, ZI_CC_USAGE_FACT, ZI_CC_EVIDENCE, ZI_CC_FINDING, ZI_CC_DEPENDENCY, ZI_CC_IMPORT_BATCH, ZI_CC_IMPORT_ROW, ZI_CC_RULESET, ZI_CC_RULE, ZI_CC_RECOMMENDATION, ZI_CC_RULE_MATCH, ZI_CC_DECISION, ZI_CC_DECISION_AUDIT	M1 for data/import; M2 rules/recommendation; M3 decision/audit	Direct table associations; activate/inspect projected element names and key/FK queries
CDS projection/consumption view entities (DDLS)	ZC_CC_SOURCE_SYSTEM, ZC_CC_CUSTOM_OBJECT, ZC_CC_USAGE_FACT, ZC_CC_EVIDENCE, ZC_CC_FINDING, ZC_CC_DEPENDENCY, ZC_CC_IMPORT_BATCH, ZC_CC_IMPORT_ROW, ZC_CC_RULESET, ZC_CC_RULE, ZC_CC_RECOMMENDATION, ZC_CC_RULE_MATCH, ZC_CC_DECISION, ZC_CC_DECISION_AUDIT, ZC_CC_PORTFOLIO	Same underlying entity owner; M3 integrates service	Expose only v1 fields; verify OData $metadata, association, filter and portfolio counts
CDS metadata extensions (DDLX)	ZC_CC_OBJECT_MDE, ZC_CC_DECISION_MDE, ZC_CC_PORTFOLIO_MDE	M1 for inventory labels, M3 for workflow labels, M2 for metrics; M4 supplies display map	UI labels/line items/facets/search; activate and verify FE renders correct sections
RAP behavior definitions (BDEF)	Root ZI_CC_IMPORT_BATCH; root ZI_CC_RULESET with rule composition; root ZI_CC_DECISION	M1, M2, M3 respectively	Managed root/action/validation/authorization configuration; verify behavior preview and negative actions
RAP behavior pools	ZBP_I_CC_IMPORT_BATCH, ZBP_I_CC_RULESET, ZBP_I_CC_DECISION	M1, M2, M3 respectively	Import processing, rule publication, decision actions; local tests and tenant action tests
DCL access controls (DCLS)	ZP_CC_OBJECT, ZP_CC_IMPORT, ZP_CC_RULESET, ZP_CC_DECISION, ZP_CC_AUDIT	M3	Use only tenant-supported principal/role mapping; verify allowed and denied direct OData queries
CDS abstract action-parameter entities (DDLS)	ZA_CC_ASSIGN_OWNER, ZA_CC_OVERRIDE, ZA_CC_SUBMIT, ZA_CC_APPROVE; ZA_CC_CSV_UPLOAD only if Day-1 mode A is proven	M3; upload entity only by M1	Typed action dialogs; verify generated parameter metadata, mandatory reason, payload cap
Service definition (SRVD)	ZUI_CC_COCKPIT	M3 (integrator); projections owned by M1/M2/M3	Expose only activated consumer projections and actions; check $metadata
Service binding (SRVB)	ZUI_CC_COCKPIT_V4 if supported; otherwise proposed ZUI_CC_COCKPIT_V2	M3	OData UI binding tested with actual FE runtime; no assumed endpoint/deployment

ABAP global classes/interfaces and tests

Object type/name	Owner	Required methods/use; dependency and done test
Interface ZIF_CC_TYPES	M2	Shared constants and public types; compile assertion against contract enum strings
Interface ZIF_CC_CLASSIFIER	M2	Pure EVALUATE_OBJECT; compile consumer and run deterministic tests
Interface ZIF_CC_RECOMMENDATION_SERVICE	M2	RECALCULATE_OBJECTS, GET_LATEST_FOR_OBJECT, GET_BY_UUID; M1/M3 test consumer
Interface ZIF_CC_IMPORTER	M1	PARSE_CSV, VALIDATE_ROW, PROCESS_BATCH typed contracts; all import row tests
Class ZCL_CC_CSV_PARSER	M1	Quoted-field/line ending/column/size handling; parser unit tests
Class ZCL_CC_IMPORT_VALIDATOR	M1	Required fields/enums/date/count/reference checks; import matrix
Class ZCL_CC_IMPORT_SERVICE	M1	Row-level atomic upsert/idempotency/provenance/recalc handoff; integration tests
Class ZCL_CC_EVIDENCE_GATE	M2	Missing/unknown/stale rules with explicit AsOfUtc; boundary tests
Class ZCL_CC_CLASSIFIER	M2	Pure deterministic policy evaluation, conflict/no-match, risk/priority; rule suite
Class ZCL_CC_RECOMMENDATION_SERVICE	M2	Load input, evaluate, persist snapshot/matches, latest lookup; integration tests
Class ZCL_CC_RULESET_SEEDER	M2, conditional non-production setup	Idempotently create synthetic CC_DEMO v1 rules; must not run in production
Class ZCL_CC_PORTFOLIO_QUERY	M2, only if Day-1 CDS aggregate capability fails	Same metrics as ZC_CC_PORTFOLIO; reconciliation against fixture
Class ZCL_CC_DEMO_SEED	M1, conditional non-production only	Idempotent Day-5 fixture seed; must not be exposed or runnable in production
Class ZCL_CC_INVENTORY_REPOSITORY	M1	GET_SOURCE_SYSTEM, GET_OBJECT_BY_BUSINESS_KEY, CHECK_OBJECT_NATURAL_KEY; local ltcl_inventory_repository tests
Class ZCL_CC_D1_PROBE	M1, temporary Day-1 probe	READ_PROBE and local ltcl_probe; remove from runtime dependencies after capability gate
Probe ZCL_CC_D1_RULE_PROBE, RAP ZBP_I_CC_D1_PROBE	Each owner Day 1	Temporary capability evidence; do not make them runtime dependencies
ABAP Unit classes ltcl_* in owning pools	M1/M2/M3	Local test classes; use named tests in §10; no imaginary standalone test-object files

Frontend files (only when Day-1 selects local SAP Fiori Tools/BAS Fiori Elements)

Use the generator output as the source of truth. Proposed project root is `ui/cc-migration-cockpit/`. Expected standard artifacts for the selected generated application are `ui/cc-migration-cockpit/package.json`, `ui/cc-migration-cockpit/webapp/manifest.json`, `ui/cc-migration-cockpit/webapp/Component.js`, `ui/cc-migration-cockpit/webapp/index.html`, and the generator-created annotation/config files under `ui/cc-migration-cockpit/webapp/annotations/` (retain the exact generated filename from Day 1; do not create a second arbitrary annotations XML). Routes/pages are generated Fiori Elements List Report/Object Page plus a portfolio route/page supported by that workflow. Override/submit/approve use RAP metadata action parameter dialogs first; a custom extension is created only if the selected generator/runtime cannot render an agreed action.

For direct file upload **only if Day 1 proves both service parameter and FE extension support**, proposed generator-registered files are `webapp/ext/controller/ImportUpload.controller.js` and `webapp/ext/fragment/ImportUpload.fragment.xml`, with registration in `webapp/manifest.json`. If unsupported, these files do not exist; the app uses ImportRow staging CRUD and labels the limitation. Never make an arbitrary React app or hard-code a local service URL.

Local project documentation/data files

Local path	Owner	Why it exists / acceptance
`docs/clean-core/environment/day1-*.md`	Each member	Actual capability evidence and selected fallback; no credentials
`docs/clean-core/contracts/data-v1.md`, `rules-v1.md`, `workflow-v1.md`, `mvp-contract-v1.md`, `change-log.md`	M1/M2/M3 sections; M4 canonical	Frozen shared field/action/schema contract
`docs/clean-core/requirements-traceability.md`, `blockers.md`	M4	Prompt/PDF-source caveat, requirements, acceptance and current blockers
`docs/clean-core/import-template/*.csv`, `docs/clean-core/demo/*.csv`	M1	Versioned six-kind header templates and synthetic dataset
`docs/clean-core/import-template/README.md`	M1	CSV encoding, fields, row order, errors, retry and size limits
`docs/clean-core/ui/fiori-preview-runbook.md`, `docs/clean-core/ui/integration-checklist.md`	M4	Used if local Fiori project unsupported; exact real UI profile
`docs/clean-core/architecture/*.mmd`	M1/M2/M3	Data, classifier, workflow diagrams checked against actual entities
`docs/clean-core/setup/*.md`, `docs/clean-core/security/authorization-matrix.md`	M1/M3	Activation, import, auth setup and limitations
`docs/clean-core/rules/rule-catalog-v1.md`	M2	Every active rule, threshold/source status and passing test ID
`docs/clean-core/test-evidence/*.md`	M1–M4	Exact expected/actual results and sanitized screenshots/traces
`docs/clean-core/README.md`, `docs/clean-core/demo/demo-script.md`	M4	Single handoff entry point and demo sequence

9. Integration dependency matrix and daily handoffs

Producer → consumer	Artifact/interface	Due day	Acceptance test / handoff evidence
Day-1 SAP admin/M1 → all	`day1-capability-matrix.md`: package, DDL, CDS, class, ABAP Unit, persistence and supported imports	1	All four probe results recorded; every unsupported feature has one consistent alternative/blocker
M2 → M1/M3/M4	`ZIF_CC_TYPES`, `ZIF_CC_CLASSIFIER`, `rules-v1.md`	3	Consumer compiles; enum strings/input/output names match canonical contract
M1 → M2/M3/M4	`ZI_CC_SOURCE_SYSTEM`, `ZI_CC_CUSTOM_OBJECT`, `ZC_CC_*` inventory	4	OData metadata contains exact IDs/fields and a real object query works
M3 → M4	`ZUI_CC_COCKPIT` + selected binding and probe URL/profile	4	M4 loads live inventory; metadata and host are from actual tenant
M1/M2/M3 → M4	Day-5 object+recommendation detail fields	5	Synthetic object opens; missing evidence shows `MANUAL_REVIEW`, not a fabricated success
M1 → M2	Usage/Evidence/Finding/Dependency CDS and type-code headers	6	One fixture per evidence type resolves to a stable ObjectUUID/provenance
M1 → M4	`ZI/ZC_CC_IMPORT_BATCH`, `ZI/ZC_CC_IMPORT_ROW`, `processBatch`	7	One valid and one invalid row produce persisted result and readable error
M1 → M2	Idempotent import outputs, source rows and affected object UUID list	8	Replay yields no duplicate; M2 receives only affected ObjectUUIDs
M2 → M1	`ZIF_CC_RECOMMENDATION_SERVICE~RECALCULATE_OBJECTS`	8	M1 compiles caller; M2 owns snapshots; fixed AsOfUtc and typed errors

Producer → consumer	Artifact/interface	Due day	Acceptance test / handoff evidence
M2 → M3/M4	Outcome/rule-version/match/explanation fields and immutable snapshots	9	M3 links snapshot; M4 shows exact matched rule IDs and rationale
M1 ↔ M2	Import-to-recommendation RAP-safe LUW/EML integration	10	All five synthetic outcomes match; failure counts are truthful
M2 → M3	Recommendation by UUID/latest read methods	12	M3 creates decision against an exact existing snapshot
M3 → M4	BDEF actions, parameter metadata, auth matrix and audit entity	12–15	Valid action succeeds; blank reason/self-approval direct call fails; audit is readable
M3 → M1/M2	Actual supported DCL/global/instance role mapping	13	Direct API allowed/denied cases match owner-authored policy
M2 + M3 → M4	Portfolio metrics and decision-state definition	17	KPI fixture sums reconcile and filters use same population
M1/M2/M3 → all	Release candidate, regression reports, open blocker list	18	No P1 defect; all negative tests pass or are marked blocked
All → M4	Setup docs, tests, screenshots, demo dataset and limitations	19	All relative links resolve; no secrets/real records; demo is reproducible
All → project sponsor	Day-20 acceptance/demo evidence	20	Checklist signed or specific unmet item/owner/due date documented

Daily cadence: At start, each owner names today's artifact and dependency status; by end, they report activation/test result/transport or Git commit and update `blockers.md`. Breaking shared contract requires a change-log entry, impact list, consumer approval, and same-day test update. Integration is continuous from Day 3; no member builds a parallel private schema.

10. Testing matrix (exact named inputs and expected results)

ID / feature	Owner	Test input/action	Expected result
IMP-01 valid object row	M1	`DEMO01,CLAS,ZCL_TEST,ZDEMO,DEMO, Fixture object,SYNTHETIC_OWNER,OBJ-TEST-001` with existing source system	One CustomObject persisted; status `PERSISTED`; SourceSystemUUID and batch provenance set
IMP-02 missing required field	M1	Object CSV row has blank `object_name`	Row `REJECTED`, error `MISSING_REQUIRED_FIELD`; no object persisted
IMP-03 invalid field value	M1	Usage `execution_count=-1` or period end before start	Row rejected with `INVALID_EXECUTION_COUNT` / `INVALID_PERIOD`; no UsageFact
IMP-04 duplicate within one CSV	M1	Same natural/source key and payload on lines 2 and 3	First persisted, second `SKIPPED_DUPLICATE`; count once
IMP-05 repeated import	M1	Submit exact same valid file as a new batch	Existing UUID unchanged; second row `SKIPPED_DUPLICATE`; `DuplicateRows` increments and no fact is added
IMP-06 invalid source/object reference	M1	Usage references `NO_SUCH_SYSTEM` or absent object	Row rejected with `UNKNOWN_SOURCE_SYSTEM` or `UNKNOWN_OBJECT`; no dangling FK
IMP-07 malformed CSV row	M1	Unmatched double quote or wrong column count	Row `REJECTED`, `MALFORMED_CSV/COLUMN_COUNT_MISMATCH`; neighboring valid row remains processable
IMP-08 corrected re-import	M1	Correct IMP-02 row under same stable `SourceRecordKey` in a new batch	Corrected row persists; rejected prior row remains in prior batch; no duplicate natural object
IMP-09 incomplete evidence	M1+M2	Valid object/evidence code and source but empty/UNKNOWN value	Row `ACCEPTED_WARNING`, quality incomplete/unknown; object evaluation `MANUAL_REVIEW` with missing code
IMP-10 dependency target missing	M1	Dependency source exists but target object does not	Row rejected `UNKNOWN_TARGET_OBJECT`; no Dependency record
RULE-01 rule met	M2	Fresh PASS, usage count 25 in 90-day period, candidate NO, no higher-priority match	`KEEP`, `D40_READINESS_PASS`, version recorded
RULE-02 rule failed	M2	Fresh FAIL, valid usage, no missing required facts	`REMEDIATE`, `D10_READINESS_FAIL` selected
RULE-03 replace indicator	M2	Complete required evidence, `ALTERNATIVE_CANDIDATE=YES`, no priority-10 failure	`REPLACE`, `D20_ALT_CONFIRMED` selected; synthetic indicator shown
RULE-04 zero usage	M2	Fresh required evidence and 90-day `ExecutionCount=0`	`RETIRE`, `D30_ZERO_USAGE` selected unless a higher-priority rule matches
RULE-05 missing evidence	M2	No `CLOUD_READINESS` row	`MANUAL_REVIEW`, `EvidenceStatus=INCOMPLETE`, missing code listed; no positive rule
RULE-06 stale evidence	M2	Capture is 91 days before fixed AsOfUtc	`MANUAL_REVIEW`, `EvidenceStatus=STALE`; exactly 90 days is fresh

ID / feature	Owner	Test input/action	Expected result
RULE-07 equal-priority conflict	M2	Test rules at same winning priority produce KEEP and RETIRE	`MANUAL_REVIEW`, both RuleIDs traced, neither selected as winner
RULE-08 no rule matches	M2	Complete input with test RuleSet containing only nonmatching active rule	`MANUAL_REVIEW`, `SYSTEM_FALLBACK`, no unsupported best-guess output
RULE-09 unknown value	M2	Required `CLOUD_READINESS=UNKNOWN`	`MANUAL_REVIEW`, EvidenceStatus UNKNOWN/INCOMPLETE per contract; no positive output
RULE-10 deterministic evaluation	M2	Same evidence/rules/AsOfUtc evaluated twice	Same Outcome, version, rationale, score, missing codes, and sorted match list
RULE-11 risk formula	M2	Score fixture: incomplete + 1 high + 2 deps = 80; then boundary fixtures at 39/40/69/70	Formula yields 80/HIGH; bands LOW/MEDIUM/MEDIUM/HIGH at boundaries
WF-01 valid submission	M3	DRAFT, assigned owner, required snapshot; owner calls submit	`SUBMITTED`; server SubmittedBy/time set; one SUBMITTED event
WF-02 invalid submission	M3	DRAFT with no owner or invalid recommendation	Rejected; remains DRAFT; no success audit event
WF-03 valid approval	M3	SUBMITTED decision; different reviewer role	`APPROVED`; ApprovedBy/time set; one APPROVED audit event
WF-04 unauthorized approval	M3	Analyst/no-role attempts approve	Authorization failure; state unchanged; no approval event
WF-05 override without reason	M3	DRAFT, NewOutcome valid, Reason blank/whitespace	Rejected `OVERRIDE_REASON_REQUIRED`; outcome/flag unchanged; no OVERRIDDEN event
WF-06 override with reason	M3	DRAFT, allowed outcome, `Reason=Reviewed by owner`	Outcome/IsOverride updated; exact reason stored in audit; status remains DRAFT
WF-07 self-approval	M3	Same authenticated actor is SubmittedBy and invokes approve	Denied by server; state remains SUBMITTED; no APPROVED event
WF-08 illegal transition	M3	Approve DRAFT or submit APPROVED	Denied with transition error; no change/event
WF-09 stale update	M3	Submit/approve with old ETag or concurrent update	Conflict/lock error; stale caller does not overwrite latest state; use documented serialized fallback if ETag absent
AUD-01 state event	M3	Successful assign/override/submit/approve	Exactly one expected EventCode, old/new state/outcome and actor/time recorded
AUD-02 actor/time	M3	Call action as test principal at fixed/observable server time	Actor is authenticated principal; timestamp is server-generated UTC, not client payload
AUD-03 override reason	M3	Valid override with reason	Decision and `OVERRIDDEN` audit both retain exact reason
AUD-04 append-only	M3	OData update/delete attempt against historical event	Service denies/no operation; stored event unchanged
UI-01 inventory display	M4	Load List Report with synthetic source/object	Rows and names match OData response
UI-02 search/filter	M4	Search object name and combine system/type/component filters	Exact expected rows; reset clears filters; server result used
UI-03 object details	M4	Open known ObjectUUID	Source, usage, evidence, finding, dependency sections match service
UI-04 recommendation explanation	M4	Open KEEP and MANUAL_REVIEW objects	Outcome, version, selected/matched rule, rationale and missing evidence visible
UI-05 decision action	M4	Assign/submit then reviewer approve	Server status refreshes and audit row appears
UI-06 mandatory reason	M4+M3	Open Override, submit blank then valid reason	Blank blocked by UI and RAP; valid call updates outcome/reason
UI-07 audit history	M4	Open approved decision	Actor/time/events/reason shown in chronological order
UI-08 portfolio metrics	M4+M2	Load all fixture objects then filter by system/component	KPI totals match total=5, outcome counts=1 each, incomplete=1, priority HIGH=0/MEDIUM=1/LOW=4, open=0; filtered list matches
UI-09 loading/empty/error	M4	Delay service, empty filter, deny role, return service error	Separate loading/empty/403/error states; no stale fake data
UI-10 unsupported upload mode	M4	Tenant selects staging fallback	No file-upload button is represented as functional; staging description and batch errors are visible

11. Day-20 acceptance checklist

Source and scope

- [ ] PDF has been attached and requirements reconciled, or deliverable is explicitly accepted as prompt-derived provisional plan.
- [ ] Day-1 matrix records actual SAP product/tenant, ADT package, supported object types, OData version, Fiori path, auth and concurrency capabilities.

- [ ] Unsupported capabilities are listed with the selected §3 alternative; no fake table, binding, upload, role or deployment claim remains.
- [ ] Synthetic demo policy and thresholds are labeled non-SAP guidance; no live scanner or automatic code changes are implied.

Backend/data/rules

- [ ] Agreed entities/fields/keys/relationships and enums match `mvp-contract-v1.md` and activated metadata.
- [ ] Import distinguishes valid, malformed, missing-required, invalid-value, duplicate, repeated, nonexistent-reference, incomplete-evidence and corrected-reimport rows.
- [ ] Accepted rows retain batch/source/row provenance; repeat import is idempotent; count totals reconcile.
- [ ] Classifier is deterministic for fixed inputs/time; missing/unknown/stale/no-match/conflict uses `MANUAL_REVIEW`; version and matched-rule trace persist.
- [ ] Risk score formula and portfolio values reconcile with the frozen synthetic fixture and carry the demo-policy disclaimer.

Workflow/security/UI

- [ ] Decision lifecycle permits only agreed transitions; override reason is mandatory; approval is role-authorized and separated from submitter.
- [ ] Actor/timestamp/reason are server-derived/stored; ordinary service calls cannot update/delete audit events.
- [ ] ETag/lock stale-update test passes or the approved serialized/single-writer limitation is visible and tested.
- [ ] Inventory, filters, object details, evidence, explanation, actions, audit, portfolio and loading/empty/error states work in the verified Fiori runtime.
- [ ] UI values are service-backed; no browser-only status, mock backend, localhost dependency or hidden upload limitation.

Delivery evidence

- [ ] Import, rules, workflow, audit and UI matrices in §10 have pass/fail results and owner.
- [ ] No open P1 defect; accepted P2/unsupported item has named owner, impact and next step.
- [ ] Runbooks, diagrams, setup, screenshots, synthetic dataset, Day-20 script and transport/Git IDs are present and sanitized.
- [ ] Each member signs the items within their ownership; remaining blockers are not described as completed.

12. Final demo sequence (10–12 minutes)

1. **Environment and scope (1 min, M4):** State the actual ABAP/Fiori profile chosen on Day 1; distinguish platform authentication from application authorization. Call out any missing PDF-derived confirmation and that demo rules/data are synthetic.
2. **Portfolio overview (1 min, M4/M2):** Show total objects and outcome/evidence/priority KPIs; apply a source-system filter; click a KPI into the matching inventory rows. M2 reconciles figures to the fixture.
3. **Inventory and provenance (2 min, M4/M1):** Search/filter a custom object; open its object page; show system/type/package, usage period, captured evidence, finding/dependency, source row and ImportBatch provenance.
4. **Recommendation explanation (2 min, M2/M4):** Show one KEEP/REMEDIATE or RETIRE example and one missing/stale MANUAL_REVIEW example. Expand matched RuleID, priority, rule-set version, rationale and missing code. Explicitly call the score and thresholds project-defined demo policy, not SAP guidance.
5. **Import behavior (1–2 min, M1):** Submit the synthetic CSV using the verified upload or staging mode. Show one malformed/invalid-reference row, error code, corrected retry and same-file replay with no duplicate entities. If direct upload is unsupported, show ImportRow staging and say so plainly.
6. **Decision workflow (2 min, M3/M4):** Create/assign a DRAFT, attempt override with blank reason (rejected), enter a reason, submit as owner, attempt self-approval (rejected), then approve as a different authorized reviewer.
7. **Audit and close (1 min, M3/M4):** Show server actor/time/reason and append-only history, point to the test evidence and list any unsupported capabilities/open limitations. End with the Day-20 acceptance checklist and deliverable/transport locations.

Final source note: This file is a detailed, internally consistent implementation blueprint based on the written request. It is not a PDF extraction because the PDF was not available in this turn. Once the PDF is uploaded, reconcile §1–§5 and the Day-2 contract before treating this as the authoritative implementation plan.
rt locations.

Final source note: This file is a detailed, internally consistent implementation blueprint based on the written request. It is not a PDF extraction because the PDF was not available in this turn. Once the PDF is uploaded, reconcile §1–§5 and the Day-2 contract before treating this as the authoritative implementation plan.
e malformed/invalid-reference row, error code, corrected retry and same-file replay with no duplicate entities. If direct upload is unsupported, show ImportRow staging and say so plainly.
6. **Decision workflow (2 min, M3/M4):** Create/assign a DRAFT, attempt override with blank reason (rejected), enter a reason, submit as owner, attempt self-approval (rejected), then approve as a different authorized reviewer.
7. **Audit and close (1 min, M3/M4):** Show server actor/time/reason and append-only history, point to the test evidence and list any unsupported capabilities/open limitations. End with the Day-20 acceptance checklist and deliverable/transport locations.

Final source note: This file is a detailed, internally consistent implementation blueprint based on the written request. It is not a PDF extraction because the PDF was not available in this turn. Once the PDF is uploaded, reconcile §1–§5 and the Day-2 contract before treating this as the authoritative implementation plan.
rt locations.

Final source note: This file is a detailed, internally consistent implementation blueprint based on the written request. It is not a PDF extraction because the PDF was not available in this turn. Once the PDF is uploaded, reconcile §1–§5 and the Day-2 contract before treating this as the authoritative implementation plan.